CKA — Certified Kubernetes Administrator

Core concepts, scheduling, storage, networking, security & troubleshooting

14 chapters · study & print edition

Contents

🚢 CKA Study Notes — Complete Course, Section by Section

> The entire **Certified Kubernetes Administrator** course, rebuilt on the **Learning Ladder** so you can skip the videos and actually *understand* — not memorize — Kubernetes. Every section climbs the same 8 rungs, leading with *why* and the *machinery* and putting the commands last. Source: Mumshad's CKA course.

🪜 How each section is built (the Learning Ladder)

Each file climbs the same rungs — read them in order and answer each ✅ **Check yourself** out loud before climbing on:

Rung	What it does
0 Setup	what you're learning & why it's on the CKA
1 The Pain	the problem that forced this to exist
2 The One Idea	the single sentence to memorize — everything derives from it
3 The Machinery ⚙️	how it *actually* works, with ASCII diagrams (the most important rung)
4 Vocabulary Map	every scary term pinned to a part of the machinery
5 The Trace	one concrete action followed end-to-end
6 The Contrast	how it differs from the alternative, and when *not* to use it
7 Prediction Test 🧪	the commands — as "if I do X, then Y, because [mechanism]" predictions you run & verify
🎁 Capstone	compress it to one sentence; find your shaky rung

Then each file closes with 🎯 **CKA exam tips** and a 📌 **command cheat sheet** for the exam.

How to use these: climb a section rung by rung, do its Rung-7 predictions on a live cluster (killercoda/minikube/kind) *before* reading the outcome, then answer the check-yourself questions. The test of readiness is that you can *predict* what a command does and do it fast — Istio_Learning_Ladder.md is the gold-standard example of the same method.

📚 The curriculum

#	Section	Covers	Exam weight
01	Core Concepts	architecture, pods, ReplicaSets/Deployments, Services, namespaces, imperative vs declarative	⭐⭐⭐
02	Scheduling	manual sched, taints/tolerations, node affinity, resources, DaemonSets, static pods, priority, admission	⭐⭐⭐
03	Logging & Monitoring	metrics-server, `kubectl top`, `kubectl logs`	⭐
04	Application Lifecycle Mgmt	rollouts/rollback, commands/args, env, ConfigMaps, Secrets, multi-container/init/sidecar	⭐⭐⭐
05	Cluster Maintenance	drain/cordon, version skew, **kubeadm upgrade**, **etcd backup/restore**	⭐⭐⭐
06	Security	TLS/PKI, CSR API, kubeconfig, **RBAC**, service accounts, image secrets, security contexts, NetworkPolicy	⭐⭐⭐
07	Storage	volumes, **PV/PVC**, access modes, reclaim policies, StorageClasses	⭐⭐
08	Networking	ports, **CNI**, kube-proxy, **CoreDNS**, **Ingress**	⭐⭐⭐
09	Design & Install a Cluster	HA topologies, leader election, **etcd quorum**	⭐⭐
10	Install the Kubeadm Way	runtime/cgroups, `kubeadm init`, CNI, `kubeadm join`	⭐⭐
11	Helm Basics	charts, releases, install/upgrade/rollback *(supplementary)*	⭐
12	Kustomize Basics	base/overlays, transformers, patches *(on the exam)*	⭐⭐
13	Troubleshooting	app / control-plane / worker-node / network failures	⭐⭐⭐⭐

Added beyond the course flow where useful: cross-links to the sibling **Linux** and **Networking** deep-dive guides (`../../Linux/`, `../../Networking/`) for the primitives under the hood (namespaces, iptables, TLS, DNS).

🎯 The CKA exam — what to expect

- **Format:** 100% **performance-based** — ~15–20 hands-on tasks on real clusters, **2 hours**, remotely proctored.
- **Passing score: 66%**. Partial credit exists — bank easy points, flag hard ones, move on.
- **Domains & weights** (current curriculum):

Domain	Weight	Mapped sections
Cluster Architecture, Installation & Configuration	**25%**	1, 5, 6, 9, 10
Workloads & Scheduling	**15%**	1, 2, 4
Services & Networking	**20%**	1, 6, 8
Storage	**10%**	7
Troubleshooting	**30%**	3, 13

- **Allowed docs:** the official **kubernetes.io/docs** (+ `/blog`), **helm.sh/docs**, and Kustomize docs — one browser tab. Learn to *search the docs fast* and copy YAML samples.
- You work across **multiple clusters** — always run the given `kubectl config use-context <ctx>` first.

⚡ Exam-day speed kit (set this up in the first 60 seconds)

```
alias k=kubectl
export do="--dry-run=client -o yaml"     # k run x --image=nginx $do > x.yaml
export now="--force --grace-period=0"    # k delete pod x $now
source <(kubectl completion bash)        # tab completion
complete -F __start_kubectl k            # completion for the alias too
# set vim: 2-space indent, expandtab   → ~/.vimrc:  set et ts=2 sw=2
```

The universal workflow: generate YAML, never hand-type it.

```
k run nginx --image=nginx $do > pod.yaml         # then edit + k apply -f
k create deploy web --image=nginx --replicas=3 $do > d.yaml
k expose deploy web --port=80 $do > svc.yaml
k create role/rolebinding/clusterrole ... $do    # RBAC fast
k explain pod.spec.containers --recursive        # forgot a field? no browser needed
```

🧭 Cross-cutting tips (worth more than any single section)

1. **Read the task's context switch.** Every question tells you which cluster — `kubectl config use-context` first, every time.
2. **Imperative first, YAML only when forced.** Simple pod/deploy/service/RBAC → imperative + `$do`. Complex (multi-container, volumes, probes) → generate + edit + `apply`.
3. **Know the file locations cold:** static pods `/etc/kubernetes/manifests/`, PKI `/etc/kubernetes/pki`, kubelet `/var/lib/kubelet/config.yaml` + `/etc/kubernetes/kubelet.conf`, etcd certs `/etc/kubernetes/pki/etcd`.
4. **Know the ports cold:** apiserver **6443**, kubelet **10250**, etcd **2379/2380**, scheduler **10259**, controller-mgr **10257**, NodePort **30000–32767**, CoreDNS **53**.
5. **Troubleshooting is 30%** — internalize Section 13's checklists (map→endpoints→logs; static-pod manifests; kubelet service→logs→config).
6. **`kubectl describe` + `kubectl logs --previous`** solve most "why is it broken" tasks.
7. **etcd backup/restore and a cluster upgrade** are near-guaranteed — practice each end-to-end until reflexive (Section 5).
8. Don't over-verify; if a task validates, move on. Flag the two hardest for the end.

🗓️ A 3-week study plan

```
WEEK 1 — Foundations & workloads
  Day 1  01 Core Concepts        Day 2  02 Scheduling
  Day 3  04 App Lifecycle        Day 4  03 Logging + 07 Storage
  Day 5  redo all Rung-7 labs imperatively (speed)

WEEK 2 — Cluster ops & networking (the heavy weights)
  Day 1  06 Security (RBAC + certs)   Day 2  08 Networking (CNI/CoreDNS/Ingress)
  Day 3  05 Cluster Maintenance (upgrade + etcd)   Day 4  09 Design + 10 Kubeadm
  Day 5  12 Kustomize + 11 Helm

WEEK 3 — Troubleshooting & exam simulation
  Day 1-2  13 Troubleshooting (break & fix clusters repeatedly)
  Day 3    full mock exam under 2h timer (killer.sh)
  Day 4    review misses; re-drill weak cheat sheets
  Day 5    second mock; polish speed kit + doc-search muscle
```

🐧 Under the hood

Every Kubernetes construct is a Linux/networking primitive with a fancy name — pods are namespaces + cgroups, Services are iptables, DNS is CoreDNS, certs are PKI. If a section's *machinery* feels fuzzy, drop to the sibling deep-dive guides: ../../Linux/ and ../../Networking/. Master the primitives once and CKA stops being memorization — you can *derive* the answer and *debug* anything.

Now go break some clusters and fix them. That's the exam. Good luck. 🍀

Core Concepts, Climbed the Ladder 🪜

Section 1 of the CKA — deriving how Kubernetes works, not memorizing kubectl

> This is the CKA "Core Concepts" section rebuilt on the **Learning Ladder**. Instead of leading with `kubectl`, we climb from **why Kubernetes exists** → **the one core idea** → **the machinery** → and only at the top, the commands. Each rung ends with a ✅ **Check yourself** question — answer it in your own words and climb on; if you can't, that rung is your next hands-on session.
>
> **The one rule:** every command in Rung 7 sits at the TOP of the ladder on purpose. You'll understand *what it does to the machinery* before you run it — which is exactly what the exam tests when it hands you a broken cluster.

RUNG 0 — The Setup 🎯

What am I learning? The core Kubernetes object model and cluster architecture — the foundation every other CKA section builds on: control plane vs worker nodes, and the primitives Pod → ReplicaSet → Deployment → Service → Namespace.

Why did it land on my desk? I'm sitting the **Certified Kubernetes Administrator** exam — 2 hours, 100% hands-on, and *Cluster Architecture* alone is 25% of the score. I can't debug a cluster I can't picture.

What do I already know about it? Probably that "pods run containers" and `kubectl get pods` lists them. What's usually fuzzy: *why* there's a control plane, what actually happens between `kubectl apply` and a running container, and why a Service exists at all. That fuzziness is what this ladder removes.

RUNG 1 — The Pain 🔥

Why does Kubernetes exist at all?

Sit with the problem before touching a single object. If you understand the pain, the whole API stops needing memorization — you can *derive* what Kubernetes must do to relieve it.

You have containers to run in production. A raw container runtime (Docker alone) gives you *nothing* for the hard parts:

```
LIFE WITHOUT AN ORCHESTRATOR (the pre-Kubernetes pain)

  You: docker run myapp          ┌─ container dies at 3AM ──▶ nobody notices, app is down
       docker run myapp          ├─ traffic spikes ─────────▶ you SSH in and manually run more
       docker run myapp          ├─ need a new version ─────▶ stop all / start all = downtime
                                 ├─ which host has room? ───▶ you track it in your head / a spreadsheet
                                 └─ container IP changed ───▶ whatever pointed at it is now broken
```

What people did before — and why it hurt:

- **Self-healing:** none. A crashed container stays dead until a human notices.

- **Scaling:** manual. You SSH to a box and `docker run` more copies, then wire up a load balancer by hand.
- **Placement:** manual. *You* decide which server has spare CPU/RAM — and get it wrong.
- **Rolling updates:** all-or-nothing. Stop v1 everywhere, start v2 everywhere = an outage window.
- **Networking:** container IPs are ephemeral. Anything hard-coded to an IP breaks on the next restart.

Who feels this pain most? The **platform / ops / SRE team** — you. Developers write features; when something falls over at 3 AM or a deploy needs zero downtime, it's the platform team holding the bag. Kubernetes is fundamentally a **platform team's tool**: it automates the operational survival kit so humans don't run it by hand.

What breaks without it: every operational guarantee — availability (nothing restarts dead apps), elasticity (no autoscaling), safe releases (no rolling update/rollback), and stable service discovery (no fixed address for a moving target).

> ✅ **Check yourself before Rung 2:** In one breath — name three things a bare container runtime can't do for you in production that forced an orchestrator to be invented. (Hint: what happens when a container dies, when traffic spikes, and when its IP changes?)

RUNG 2 — The One Idea 💡

The single sentence everything else hangs off

Memorize this exact sentence — the entire object model can be *derived* from it:

> **Kubernetes is a set of controllers that continuously reconcile the cluster's *actual* state toward the *desired* state you declare, with the API server + etcd as the single hub and source of truth.**

That's the whole trick. Everything else is detail. Watch how much falls out of it:

- *"you declare desired state"* → you write **YAML objects** (Pod, Deployment, Service). You never issue step-by-step orders; you describe the end goal.
- *"controllers continuously reconcile"* → there's a **control loop** watching: "desired = 3 replicas, actual = 2, so create 1." This is why Kubernetes **self-heals** — killing a pod just widens the gap the controller closes.
- *"actual toward desired"* → **ReplicaSets, Deployments, the node controller** are all the same pattern: watch a number, drive reality to match it.
- *"API server + etcd as the single hub"* → **every** action (yours, the scheduler's, the kubelet's) goes *through* the API server and is recorded in **etcd**. Nothing talks to etcd directly except the API server.

Once you see that **every object is just "desired state I handed to a controller,"** the API stops being a pile of unrelated YAML and becomes one pattern applied over and over.

> ✅ **Check yourself before Rung 3:** Cover the sentence and say it from memory. Then answer: if you `kubectl delete` a pod that belongs to a ReplicaSet, why does a new one appear almost instantly — and *which part of the one-sentence idea* explains it?

RUNG 3 — The Machinery ⚙️

How it ACTUALLY works under the hood — the most important rung. Go slow.

🧸 Plain-English first (read this before the technical version)

Picture a big shipping company. It has a **head office** (Kubernetes calls this the "control plane") that makes decisions and keeps records, and a fleet of **warehouses** (the "worker nodes") where the actual work — running your apps — gets done. This section covers three things: how those two halves are organized, what happens step by step when you ask for something, and the machine at the bottom that actually runs your apps.

The head office (the brain):

- **The record book (etcd):** one master ledger holding everything the company knows — every order, every worker, every status. A fact is only "real" once it's written here.
- **The front desk (the API server):** the ONLY person allowed to write in the ledger. Every request — from you or from any staff member — must go through this desk. It checks your ID, checks you're allowed to ask, checks the request makes sense, then records it.
- **The dispatcher (the scheduler):** looks at new jobs with no assigned warehouse and *decides* which warehouse should take each one. Important: it only decides and reports back to the front desk — it never delivers anything itself.
- **The supervisors (the controller manager):** a room full of clerks who constantly compare "what was promised" against "what's actually happening" and raise fixes whenever the two differ.

Each warehouse (the muscle) has three staff:

- **The foreman (kubelet):** receives jobs assigned to this warehouse, gets them running, and reports progress back to the front desk. Notably, the foreman is hired directly at the building (installed on the machine itself), not managed like the jobs he runs — so if *he* collapses, nobody automatically revives him.
- **The mailroom (kube-proxy):** sets up the local delivery routes so customer requests reach the right running copies of an app.
- **The forklift (the container runtime):** the actual machine that starts and runs your app's packages ("containers" — sealed boxes holding an app and everything it needs).

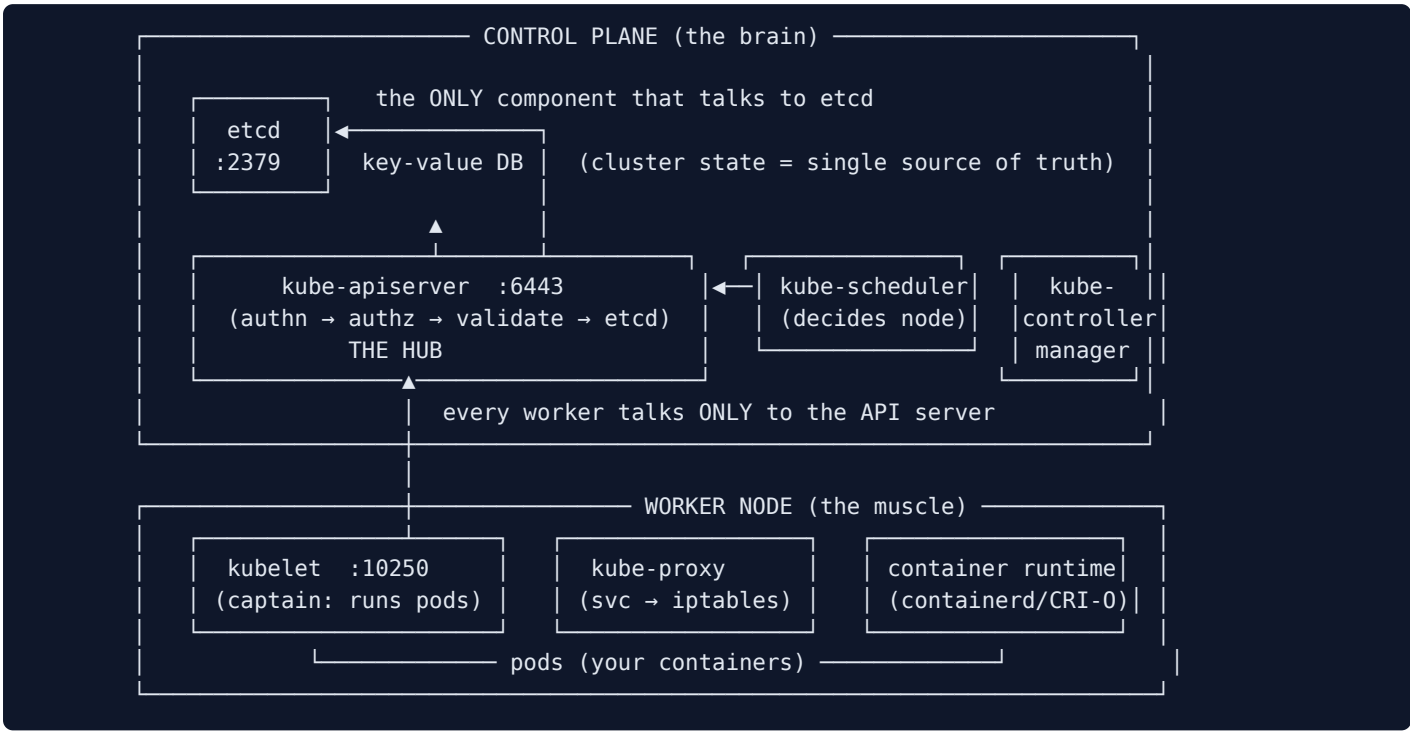
The step-by-step flow when you ask for something: you hand a request to the front desk → it's recorded in the ledger with "no warehouse assigned yet" → the dispatcher notices, picks a warehouse, and that choice is recorded → the foreman at that warehouse notices, has the forklift fetch and start the app → he reports "running," and that goes in the ledger too. Every change in Kubernetes follows this exact shape.

About the forklift brand: Kubernetes was originally built around one brand (Docker). It later created a **universal socket (called CRI)** so any compatible brand plugs in, and dropped the special adapter it had kept just for Docker. Apps packaged with Docker's tools still run fine — the boxes follow the universal standard even if that brand of forklift is gone. There are also three handheld inspection gadgets people confuse; the one worth knowing (**crictl**) is like a flashlight that lets you inspect a warehouse directly when the front desk isn't answering.

Now the original technical deep-dive — the same ideas, in precise form:

We open the hood. Three things to hold: **(A) the two-plane architecture, (B) how a pod actually gets created, and (C) what runs the containers (the runtime).**

(A) The two planes: brain vs muscle



The **control plane** decides and records; the **worker nodes** run the containers. The single most important structural fact: **everything flows through the API server, and only the API server touches etcd.** Analogy: worker nodes are cargo ships carrying containers; the control plane is the fleet of control ships directing them.

Control-plane components (memorize the ports — the exam asks):

Component	Role	Port	Where it lives (kubeadm)
etcd	Distributed key-value store; single source of truth for *all* state	`2379` client / `2380` peer	static pod `kube-system`
kube-apiserver	The hub — authenticates, validates, reads/writes etcd. **Only thing that talks to etcd.**	`6443`	static pod `kube-system`
kube-scheduler	*Decides* which node a pod goes on (it does **not** place it)	`10259`	static pod `kube-system`
kube-controller-manager	Runs all controllers (node, replication…) — the reconciliation engine	`10257`	static pod `kube-system`
cloud-controller-manager	Integrates with the cloud (LBs, nodes)	—	static pod (cloud)

Worker-node components:

Component	Role	Port	Deployed how
kubelet	The captain — registers the node, runs/monitors pods via the runtime, reports to the API server	`10250`	⚠️ **NOT a pod** — a **systemd service** you install per node
kube-proxy	Programs iptables/IPVS so Services route to pods	—	**DaemonSet** (one per node)
container runtime	Actually runs containers (containerd, CRI-O)	—	installed on every node

> 🎯 **CKA tip:** The most-tested architecture fact — **kubeadm runs the control plane as *static pods*** whose manifests live at `/etc/kubernetes/manifests/`. Edit a manifest and the kubelet auto-restarts that component. **The kubelet itself is a systemd service** (`systemctl status kubelet`), never a pod — which is why a broken kubelet can't self-heal.

(B) The golden flow: what actually happens on `kubectl create`

This is the machinery in motion — the reconcile loop from Rung 2, concretely:

```
kubectl create/apply
   └─▶ kube-apiserver:  authn → authz → validate → write "Pod (node: none)" to etcd → reply OK
          └─▶ kube-scheduler (watching): sees an unscheduled pod → filters+scores nodes → tells
apiserver
                 └─▶ apiserver writes the node assignment to etcd
                        └─▶ kubelet on that node (watching apiserver): sees a pod bound to it
                               └─▶ tells the container runtime to pull the image + start the
container
                                      └─▶ kubelet reports status back → apiserver → etcd
```

Every change in Kubernetes follows this shape: **write desired state → a controller notices the gap → it acts → the new actual state is recorded.** The API server is the center; etcd is the memory; nobody skips the hub.

(C) The runtime & CRI — what "runs the container"

Kubernetes was born coupled to Docker, then introduced the **CRI (Container Runtime Interface)** so *any* OCI-compliant runtime works. Docker predated CRI, so K8s used a shim (**dockershim**) — **removed in v1.24**. Docker is no longer a *runtime*, but Docker-*built images* still run (they follow the OCI image spec).

The three CLIs people confuse:

Tool	From	Talks to	Purpose
`ctr`	containerd	containerd only	low-level **debug only**, unfriendly — ignore
`nerdctl`	containerd	containerd	**Docker-like general CLI** (drop-in for `docker`)
`crictl`	Kubernetes	*any* CRI runtime	**debug pods/containers on a node** — the one CKA cares about

```
# On a node, when kubectl isn't enough (e.g. API server down):
crictl ps                # running containers (like docker ps)
crictl pods              # pod sandboxes
crictl logs <container>  # container logs
crictl images            # images
# crictl reads /etc/crictl.yaml for its runtime-endpoint
```

And **etcd**, the memory: a schema-free key-value store holding *everything* (nodes, pods, secrets, roles) under keys like `/registry/<resource>/<namespace>/<name>`. `kubectl get` is just reading etcd through the API server; a change is "real" only once etcd has it. (v3 API uses `put` / `get` / `del`; backup/restore is Section 5.)

> ✅ **Check yourself before Rung 4:** Draw the two-plane picture from memory, then trace `kubectl run nginx` : (1) which component writes to etcd? (2) which one picks the node, and does it *place* the pod or just *decide*? (3) which one actually starts the container? (4) if the scheduler is down, what state does the pod get stuck in?

RUNG 4 — The Vocabulary Map 🏷️

Pin every scary term to its role in the machinery above

Now the jargon has somewhere to land. Every term is just a label for a part of the picture you already hold.

Term	What it actually is	Which part of the machinery
Pod	Smallest deployable unit; wraps 1+ tightly-coupled containers sharing network + storage	What the kubelet runs on a node
Label	A key=value tag on an object	How selectors find things
Selector	A label query (`app=web`)	How Services/ReplicaSets pick their pods
ReplicaSet	Controller keeping *N* pod copies alive	The reconcile loop for pod count
Deployment	Controller managing ReplicaSets (adds rollout/rollback)	Sits above the RS
Service	Stable virtual IP + name load-balancing across pods	Programmed into nodes by kube-proxy
Endpoints	The actual pod IPs currently behind a Service	Proof the selector matched something
ClusterIP / NodePort / LoadBalancer	Service *scopes*: internal / node-port / cloud-LB	Where traffic can enter
Namespace	A virtual cluster for scoping names + policies	A partition of etcd's object tree
kubelet	Node agent that runs/reports pods	Worker node, systemd service
kube-proxy	Turns Services into iptables/IPVS rules	Worker node, DaemonSet
etcd	The key-value store of all state	Control plane, single source of truth
kube-apiserver	The hub every request flows through	Control plane, only etcd-talker
kube-scheduler	Decides pod → node	Control plane
kube-controller-manager	Houses all reconcile controllers	Control plane
CRI / containerd / crictl	Runtime interface / runtime / its debug CLI	Worker node, runs containers

The big unlock: which terms are the *same kind of thing*

```
GROUP 1 — "The workload hierarchy" (each manages the one below it):
   Deployment ──▶ ReplicaSet ──▶ Pod ──▶ container(s)
   (rollouts)     (N copies)     (unit)   (your app)

GROUP 2 — "Controllers" (all the SAME pattern: watch desired, drive actual):
   ReplicaSet · Deployment · node-controller · every *-controller in the manager

GROUP 3 — "Service discovery" (stable identity for moving pods):
   Service (virtual IP) + Selector (which pods) + Endpoints (the matched IPs)

GROUP 4 — "The control plane" (brain): apiserver + etcd + scheduler + controller-manager
GROUP 5 — "The node" (muscle): kubelet + kube-proxy + container runtime
```

Hold those five groups and you hold the Core Concepts vocabulary. Note **ReplicationController = old ReplicaSet** (same idea, `apiVersion: v1` vs `apps/v1`; the RS *requires* a `selector` and can adopt matching pods).

> ✅ **Check yourself before Rung 5:** Without looking — what's the chain of objects between a `Deployment` and a running container, and what does each link add? And: what does an empty **Endpoints** list tell you about a Service?

RUNG 5 — The Trace 🎬

Follow ONE concrete action end-to-end

Let's trace the exact thing the exam makes you do: **`kubectl apply -f deployment.yaml` (3 replicas), then a user hits the Service.** This single trace touches every component.

Step 1 — kubectl → API server. Your YAML (a Deployment, `replicas: 3`) is POSTed to `kube-apiserver:6443`. It **authenticates** you (client cert), **authorizes** you (RBAC), **validates** the object, and writes a Deployment to **etcd**. You get "created" back — but nothing is running yet.

Step 2 — Deployment controller (in controller-manager) notices. It's watching for Deployments. It sees one with no ReplicaSet and creates a **ReplicaSet** object (via the API server → etcd).

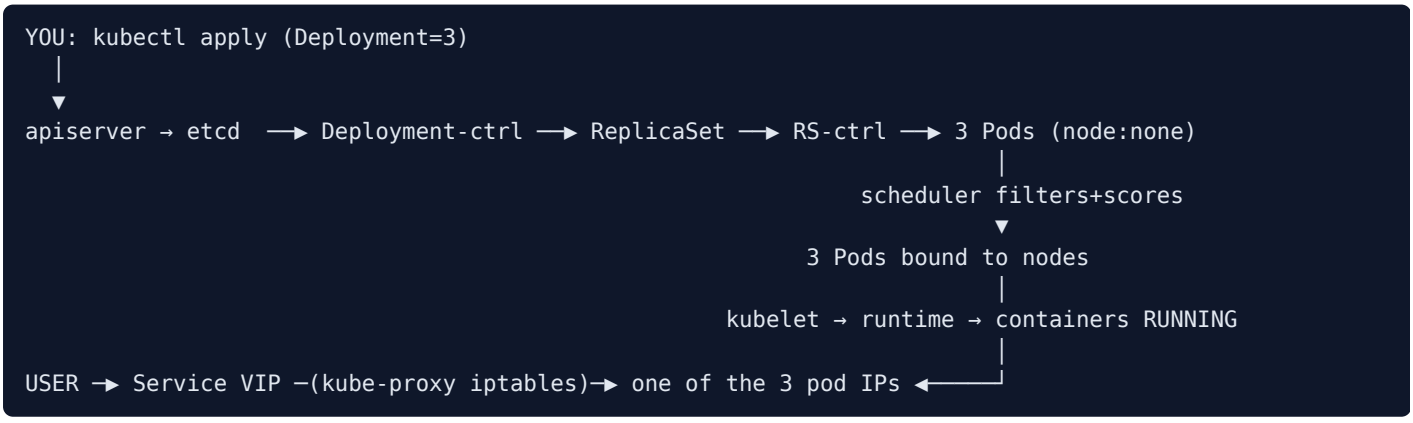
Step 3 — ReplicaSet controller notices. Desired = 3, actual = 0. It creates **3 Pod** objects, each with `node: <none>` — written through the API server to etcd.

Step 4 — Scheduler notices 3 unscheduled pods. For each: **filter** nodes (enough CPU/RAM? taints tolerated?) then **score** the survivors, pick the best, and tell the API server "bind pod → node." etcd now records each pod's node.

Step 5 — kubelet on each chosen node notices its pod. It calls the **container runtime** (via CRI) to pull the image and start the container, sets up the pod's network (CNI), and reports `Running` back → API server → etcd.

Step 6 — You expose it: `kubectl expose deployment`. A **Service** (ClusterIP) is created with `selector: app=web`. The **endpoints controller** finds the 3 pods whose labels match and records their IPs as the Service's **Endpoints**.

Step 7 — A request hits the Service. kube-proxy on every node has already turned the Service's virtual IP into **iptables/IPVS rules**. A packet to the ClusterIP is DNAT'd to one of the 3 pod IPs (random/round-robin). CoreDNS resolved the Service *name* to that virtual IP first (Section 8).



Every hop went through the API server; every fact lives in etcd; every "make it so" was a controller closing a gap.

> ✅ **Check yourself before Rung 6:** At Step 4 the scheduler did two distinct things before choosing a node — name them. And at Step 7, which component actually forwards the packet to a pod (not DNS, not the API server)?

RUNG 6 — The Contrast ⚖️

The boundary of a concept defines the concept

Kubernetes vs. running containers by hand (Docker / docker-compose):

	docker / compose	Kubernetes
Self-healing:	❌ you restart it	✅ controller reconciles
Scaling:	❌ manual docker run	✅ change `replicas`
Placement:	❌ you pick the host	✅ scheduler picks
Rolling update:	⚠️ stop-all/start-all	✅ Deployment does it live
Stable networking:	❌ IPs change	✅ Service = fixed VIP + name
Multi-host:	❌ single host	✅ whole cluster

The pattern in every row: Kubernetes replaces a *manual human action* with a *controller reconciling declared state*. That's not a bag of features — it's the one idea (Rung 2) applied everywhere.

Imperative vs declarative (the exam mindset):

- **Imperative** = *tell it how* (`run`, `create`, `expose`, `scale`, `set image`, `edit`, `delete`). Fast, one-off, not tracked.
- **Declarative** = *tell it the desired state* (`kubectl apply -f`). Idempotent (create *or* update), Git-trackable.

When NOT to use Kubernetes: a single small app on one box with no availability/scaling needs — the control-plane overhead (etcd, schedulers, a whole cluster to operate) outweighs the benefit. Kubernetes earns its keep when you have *many* services that must stay up, scale, and update safely.

One-sentence "why this over that":

> Use Kubernetes when you need containers to self-heal, scale, update safely, and find each other automatically across many hosts; run plain containers when you have one small workload and none of those guarantees matter.

> ✅ **Check yourself before Rung 7:** Explain to a colleague *why* docker-compose structurally can't self-heal a crashed container across a spike the way Kubernetes does — not "it lacks the feature," but what architectural piece it's missing. (Hint: is anything *watching desired vs actual* and allowed to act?)

RUNG 7 — The Prediction Test 🧪

Write the prediction BEFORE you run the command. A wrong prediction is your model repairing itself.

Here the commands finally arrive — reframed. Each is a **hypothesis you commit to first**, then verify. That single habit converts "I typed the command" into "I understand the system." First, the exam speed kit:

```
alias k=kubectl
export do="--dry-run=client -o yaml"     # generate YAML: k run x --image=nginx $do > x.yaml
export now="--force --grace-period=0"     # fast delete
source <(kubectl completion bash); complete -F __start_kubectl k
```

Prediction 1 — A ReplicaSet self-heals a deleted pod

> **My prediction:** "If I delete one pod owned by a 3-replica Deployment, then a replacement appears within seconds and the count returns to 3 — *because* the ReplicaSet controller is watching desired(3) vs actual, and a delete just widens the gap it exists to close."

```
k create deployment web --image=nginx --replicas=3
k get pods -l app=web                    # 3 Running
k delete pod <one-pod-name>              # kill one
k get pods -l app=web -w                 # watch
```

Verify: a new pod is created almost immediately; you end back at 3. If it *stays* at 2, your model of "controllers reconcile" needs repair — check the Deployment/RS actually owns those pods (`k get rs`).

Prediction 2 — `kubectl run` makes a Pod, not a Service

> **My prediction:** "If I `kubectl run nginx --image=nginx --port=80` , then a Pod exists but nothing is reachable from other pods by name — *because* `run` creates only a Pod; a Service is a separate object, and the `--port` flag just declares the container port."

```
k run nginx --image=nginx --port=80
k get pod nginx
k get svc                                # nginx is NOT here
```

Verify: the Pod exists; `get svc` does not list it. To make it reachable you must `k expose pod nginx --port=80` . If you expected a Service, repair: `run` = Pod only.

Prediction 3 — A selector/label mismatch = empty Endpoints = dead Service

> **My prediction:** "If a Service's `selector` doesn't match any pod's `labels` , then `describe svc` shows `Endpoints: <none>` and curling it fails — *because* the endpoints controller found no matching pods, so kube-proxy has nothing to forward to."

```
k create deployment web --image=nginx      # pods get label app=web
k expose deployment web --port=80          # selector app=web → MATCHES → endpoints populate
k describe svc web | grep Endpoints        # 1+ pod IPs
# now break it on purpose:
k patch svc web -p '{"spec":{"selector":{"app":"nope"}}}'
k describe svc web | grep Endpoints        # <none>
```

Verify: endpoints go from populated → `<none>` the instant the selector stops matching. This is the #1 real "my Service doesn't work" cause — and a classic Section 13 task.

Prediction 4 — Cross-namespace needs the FQDN

> **My prediction:** "If a pod in namespace `marketing` looks up a Service `db-service` that lives in `dev`, then the short name fails but `db-service.dev.svc.cluster.local` resolves — *because* the short name only searches the pod's *own* namespace; crossing namespaces requires the fully-qualified DNS name."

```
k create ns dev; k run redis --image=redis -n dev
k expose pod redis --name=db-service --port=6379 -n dev
k run test --image=busybox -n default -it --rm -- \
  sh -c 'nslookup db-service.dev.svc.cluster.local'
```

Verify: the FQDN resolves to the `dev` service's ClusterIP; the bare `db-service` would only resolve one in `default`. If both fail, CoreDNS may be down (Section 8).

The prediction habit, generalized

Prediction: "If I do X, then Y, because [mechanism]"	Ran it?	Right?	If wrong, what did my model miss?
1.			
2.			

🎁 CAPSTONE — Compress It

One sentence, no notes:

> Kubernetes is a control plane of reconciling controllers, fronted by one API server backed by etcd, that drives worker-node kubelets to run your declared Pods and stitches them together with label-selected Services.

Explain it to a beginner in 3 sentences:

> 1. You describe what you want (3 copies of my app, reachable at a stable address) as YAML, and hand it to the API server, which records it in etcd.
> 2. Background controllers constantly compare what you asked for against what's actually running and take action to close any gap — that's what makes Kubernetes self-heal and scale.
> 3. Because pods come and go, a Service gives them one stable virtual IP and name (matched by labels), so other apps never chase a moving target.

Which rung will I most likely need to revisit hands-on?

- **Rung 3B (the golden create-flow)** — under exam pressure people forget *which* component does what. Fix: run `k get events -A -w` while creating a Deployment and literally watch the hops.
- **Rung 4 (selectors/endpoints)** — the label↔selector↔endpoints chain trips people. Fix: do Prediction 3 twice.

🎯 CKA exam tips & quick notes (this section)

- **Set the speed aliases first thing** (`k`, `$do`, `$now`, completion) — worth minutes over the exam.
- **Always generate YAML, never type it:** `k run/create ... $do > f.yaml`, edit, `k apply -f`.
- `kubectl run` = a **Pod**; `kubectl create deployment` = a **Deployment**. Don't mix them up.
- `--dry-run=client` = print only; `--dry-run=server` = run admission but don't persist.
- Ports cold: apiserver **6443**, etcd **2379/2380**, kubelet **10250**, NodePort **30000–32767**, scheduler **10259**, controller-mgr **10257**.
- **Static pods** live in `/etc/kubernetes/manifests/`; the **kubelet is a systemd service**, not a pod.
- `Endpoints: <none>` ⇒ selector/label mismatch. `k describe` + `k get events` are your universal recon.
- `kubectl explain <res> --recursive` beats opening the docs when you forget a field.

📌 Imperative command cheat sheet

```
# PODS
k run nginx --image=nginx                                  # create a pod
k run nginx --image=nginx $do > pod.yaml                   # generate pod YAML
k run tmp --image=busybox -it --rm -- sh                   # throwaway debug pod
# DEPLOYMENTS
k create deployment web --image=nginx --replicas=3
k scale deployment web --replicas=5
k set image deployment/web nginx=nginx:1.26                # rolling image update
# SERVICES  (expose = auto-selector | create service = manual)
k expose deployment web --port=80 --type=NodePort
k expose pod redis --port=6379 --name=redis-service
# NAMESPACES
k create namespace dev
k config set-context --current --namespace=dev
# EDIT / REPLACE / DELETE
k edit deployment web                                      # live edit (NOT saved to file)
k replace --force -f pod.yaml                              # recreate (immutable-field change)
k delete pod nginx --force --grace-period=0
# INTROSPECTION
k get all -A ; k describe <res> <name> ; k explain <res> --recursive ; k api-resources
```

Related sections

- Section 2 — Scheduling — how the scheduler (Rung 3/5) picks nodes: taints, affinity, resources.
- Section 4 — Application Lifecycle Management — rollouts/rollback the Deployment adds on the RS.
- Section 8 — Networking — Services, kube-proxy, CNI and cluster DNS in depth.
- Section 6 — Security — how the API server authenticates/authorizes every request in the trace.
- Section 13 — Troubleshooting — `crictl`, `describe`, endpoints when the machinery breaks.
- ../../Linux/13-namespaces.md — the Linux namespaces/cgroups a Pod is really built from.

✅ Answers — "Check yourself before Rung N"

Before Rung 2

Q: Name three things a bare container runtime can't do for you in production that forced an orchestrator to be invented.

A: (1) **Self-healing** — when a container dies at 3 AM, nothing restarts it; it stays dead until a human notices. (2) **Scaling** — when traffic spikes, nobody starts more copies; you'd have to SSH in and `docker run` more by hand and wire up a load balancer yourself. (3) **Stable networking** — container IPs are ephemeral, so when a container restarts with a new IP, everything hard-coded to the old one breaks. (Placement decisions and zero-downtime rolling updates are two more valid answers from the same list.)

Before Rung 3

Q: Say the one-sentence idea from memory. Then: if you `kubectl delete` a pod that belongs to a ReplicaSet, why does a new one appear almost instantly — and which part of the sentence explains it?

A: The sentence: **Kubernetes is a set of controllers that continuously reconcile the cluster's actual state toward the desired state you declare, with the API server + etcd as the single hub and source of truth.** A new pod appears because the ReplicaSet controller is continuously watching desired (e.g. 3 replicas) vs actual; deleting a pod just widens the gap (actual = 2), and the controller exists to close that gap, so it immediately creates a replacement. The part of the sentence that explains it is *"controllers continuously reconcile the actual state toward the desired state"* — self-healing is not a separate feature, it's the reconcile loop doing its only job.

Before Rung 4

Q: Draw the two-plane picture, then trace `kubectl run nginx`: (1) which component writes to etcd? (2) which one picks the node — does it place the pod or just decide? (3) which one actually starts the container? (4) if the scheduler is down, what state is the pod stuck in?

A: The picture: a **control plane** (brain) holding etcd :2379, kube-apiserver :6443 as THE HUB, kube-scheduler :10259 and kube-controller-manager :10257, and **worker nodes** (muscle) each running the kubelet :10250, kube-proxy, and a container runtime — everything flows through the API server, and only the API server touches etcd. The trace: (1) **kube-apiserver** is the only component that writes to etcd — it authenticates, authorizes, validates, then records "Pod (node: none)". (2) The **kube-scheduler** picks the node, and it only *decides* — it tells the API server the binding, which is written to etcd; it never places anything itself. (3) The **kubelet** on the chosen node, watching the API server, sees the pod bound to it and tells the **container runtime** (via CRI) to pull the image and start the container. (4) With the scheduler down, the pod sits unscheduled with no node assignment — stuck in **Pending**.

Before Rung 5

Q: What's the chain of objects between a Deployment and a running container, and what does each link add? What does an empty Endpoints list tell you about a Service?

A: The chain is **Deployment → ReplicaSet → Pod → container(s)**. The Deployment adds rollouts and rollback (it manages ReplicaSets); the ReplicaSet adds the reconcile loop that keeps N pod copies alive; the Pod is the smallest deployable unit, wrapping one or more tightly-coupled containers that share network and storage; the containers are your actual app. An empty **Endpoints** list (`Endpoints: <none>`) tells you the Service's selector matched no pod labels — the endpoints controller found nothing, so kube-proxy has nothing to forward to and the Service is effectively dead. It's the #1 cause of "my Service doesn't work."

Before Rung 6

Q: At Step 4 the scheduler did two distinct things before choosing a node — name them. And at Step 7, which component actually forwards the packet to a pod?

A: The scheduler first **filters** the nodes (does the node have enough CPU/RAM? are its taints tolerated?) and then **scores** the survivors, binding the pod to the best-scoring node. At Step 7 the component that actually forwards the packet is **kube-proxy** — or more precisely the **iptables/IPVS rules** kube-proxy has programmed on every node, which DNAT a packet sent to the Service's ClusterIP to one of the matching pod IPs. CoreDNS only resolves the Service name to the virtual IP, and the API server isn't in the data path at all.

Before Rung 7

Q: Explain why docker-compose structurally can't self-heal a crashed container the way Kubernetes does — what architectural piece is it missing?

A: docker-compose is purely imperative: it starts what you told it to start and then nothing is left running that compares "what should exist" against "what does exist." Kubernetes has a **control loop** — a controller (backed by desired state recorded in the API server/etcd) that continuously watches desired vs actual and is *allowed to act* to close any gap. Compose has no reconciling controller, no stored desired state to reconcile against, and no watch mechanism, so a crashed container just stays crashed and a traffic spike changes nothing. It's not a missing feature toggle; it's the absence of the entire declare-watch-reconcile architecture that Rung 2's one sentence describes.

Scheduling, Climbed the Ladder 🪜

Section 2 of the CKA — deriving how pods land on nodes, and how you steer that

> The CKA "Scheduling" section on the Learning Ladder. Every feature here — taints, affinity, resources, DaemonSets, static pods, priority, profiles, admission — is *one lever on one machine*: the scheduler. We climb from **the pain of uncontrolled placement** → **the one matchmaker idea** → **the machinery** → then the commands as predictions. Each rung ends with a ✅ **Check yourself**.

RUNG 0 — The Setup 🎯

What am I learning? How the scheduler decides which node runs each pod, and every mechanism you use to influence or override that decision.

Why did it land on my desk? *Workloads & Scheduling* is 15% of the CKA, and "why is this pod **Pending**?" is one of the most common troubleshooting tasks (part of the 30% Troubleshooting domain). Placement bugs are everywhere.

What do I already know? Probably that "pods run on nodes" and maybe `nodeSelector`. What's fuzzy: *why* there are five different placement mechanisms, when to use which, and what actually makes a pod stick to (or bounce off) a node.

RUNG 1 — The Pain 🔥

Why does controllable scheduling exist at all?

Imagine the scheduler just dropped every pod on a random node with room. Real clusters break immediately:

```
UNCONTROLLED PLACEMENT (the pain)

  GPU training pod ──▶ lands on a cheap CPU-only node ──▶ crashes / runs 100x slow
  noisy batch job  ──▶ lands next to your prod API    ──▶ eats all RAM, API OOMs
  licensed workload ─▶ lands on any node              ──▶ license violation
  critical pod     ──▶ node is full of low-value pods ──▶ can't schedule, stays Pending
  log/monitoring agent ─▶ you must remember to run one on EVERY node, by hand
```

What people did before / without these levers: hard-wire pods to hosts by hand, or just hope. That doesn't survive node failures, autoscaling, or mixed hardware.

What breaks without control: hardware guarantees (GPU/SSD workloads), isolation (noisy neighbors), dedicated capacity (a node reserved for one team), critical-workload priority (important pods lose the race for space), and cluster-wide agents (one-per-node daemons).

Who feels it most? The **platform team** carving a shared cluster into safe, predictable slices for many teams and workload types.

> ✅ **Check yourself before Rung 2:** Give two concrete failures that happen if the scheduler ignores *what kind* of node a pod needs. (Hint: think special hardware, and think noisy neighbors.)

RUNG 2 — The One Idea 💡

The single sentence everything else hangs off

Memorize this — every feature in the section derives from it:

> **The scheduler is a matchmaker: for every pod with an empty `nodeName` it FILTERS out nodes that can't run it, SCORES the survivors, and BINDS the pod to the best one — and every scheduling feature is just a lever that biases that filter or score (or bypasses the matchmaker entirely).**

Watch the whole section fall out of it:

- *"filters out nodes that can't run it"* → **taints** (node says "keep out"), **nodeAffinity/nodeSelector** (pod says "only nodes like this"), **resource requests** (pod says "I need this much free") all remove candidate nodes.
- *"scores the survivors"* → **preferred** affinity, image locality — soft preferences that rank, not exclude.
- *"binds to the best one"* → the pod's `nodeName` gets set; that's literally all "scheduling" is.
- *"bypasses the matchmaker"* → **static pods** (kubelet runs them directly), **manual** `nodeName`, **DaemonSets** (one per node by design) skip normal scheduling.
- *"for every pod in the queue"* → **PriorityClass** decides *queue order* and can **preempt** (evict) lower-priority pods to make room.
- And **admission controllers** act *before* scheduling even starts — they can reject/mutate the pod on the way in.

> ✅ **Check yourself before Rung 3:** Cover the sentence and say it. Then: a taint and a nodeAffinity rule both keep a pod off the "wrong" node — but which one is a *node repelling pods* and which is a *pod requiring a node*? Why do you often need **both**?

RUNG 3 — The Machinery ⚙️

How placement ACTUALLY works — the most important rung. Go slow.

🧸 Plain-English first (read this before the technical version)

Think of a hotel front desk assigning guests (your apps, called "pods") to rooms (the computers, called "nodes"). The desk clerk is the **scheduler**.

(A) How assigning works. Every guest card has a blank "room number" line. The clerk watches for blank cards and does four things in order: take guests from the **waiting line**, **cross off** rooms that won't work at all, **rank** the rooms that remain, and finally **write the room number** on the card. That last pen-stroke is literally all "scheduling" is — someone else (the room's own staff) actually moves the guest in. If *every* room gets crossed off, the guest waits in the lobby indefinitely ("Pending"). If the clerk is off sick, everyone waits with no explanation. You can also skip the clerk by writing a room number on the card yourself before handing it in — but once a guest is settled, you can't move them; you check them out and check them in fresh.

(B) Three ways rooms get crossed off:

- **"Staff only" signs (taints):** a *room* can post a keep-out sign; only guests carrying a matching pass (a "toleration") may enter. Signs come in strengths: block newcomers, merely discourage them, or even evict guests already inside. Note: a sign only keeps others *out* — it doesn't pull your guest *in*.
- **Guest requirements (nodeSelector / nodeAffinity):** a *guest* can demand "only rooms with a sea view." You must first put the "sea view" sticker on the room (a label). Demands can be hard ("no such room? I'll wait forever") or soft ("nice to have, but I'll take anything"). Either way, once the guest is inside, a later sticker change doesn't kick them out.
- **Fitting (resource requests):** each guest declares the minimum space they need; the clerk only considers rooms with that much free. Guests also declare a maximum (a "limit"): overuse of shared time (CPU) just slows the guest down, but overuse of space (memory) gets them thrown out abruptly. The hotel can also set per-floor default and total space rules (LimitRange and ResourceQuota).
- The classic combo: a sign keeps strangers out of your room, AND a requirement keeps your guest in it — you need **both** to truly reserve a room for one guest.

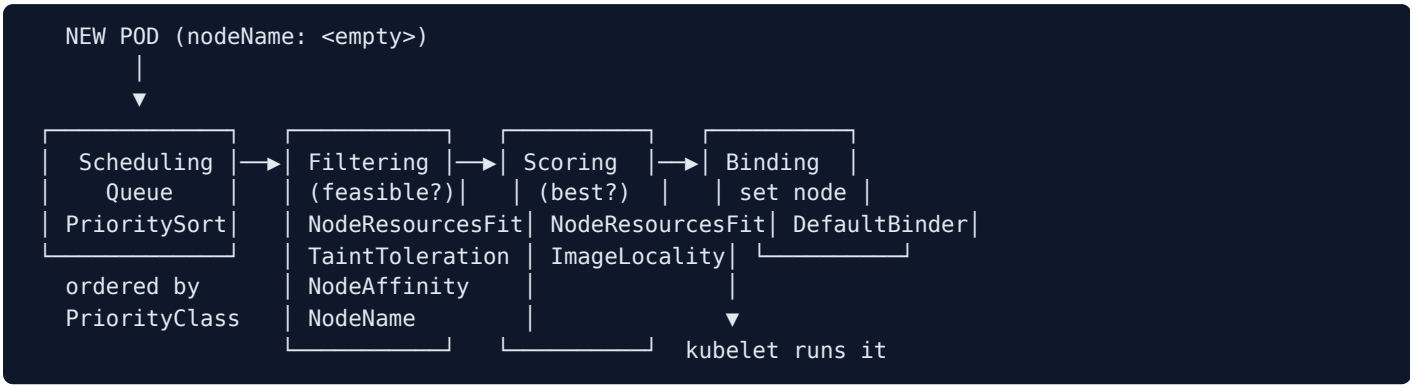
(C) Guests who skip the front desk: some staff must be in *every* room — cleaners, smoke detectors — placed automatically one-per-room (a **DaemonSet**). And a few live-in maintenance staff are hired directly by each room's caretaker from a folder of instructions on-site, with no front desk involved at all (**static pods** — this is actually how the hotel's own management offices get started).

(D) The line and the door: VIP status (**PriorityClass**) moves a guest up the waiting line and can even bump a lesser guest out of a full room (**preemption**). The hotel can employ several clerks with different rulebooks (multiple schedulers/profiles). And before any of this, a **doorman (admission controllers)** inspects each request at the entrance — he can adjust it or turn it away before it's even recorded.

Now the original technical deep-dive — the same ideas, in precise form:

(A) The core loop — what "scheduling" physically is

Every pod has `spec.nodeName`, normally empty. The scheduler watches for empty-`nodeName` pods and runs four phases, then writes the name:



If **no node survives filtering**, the pod stays **Pending** forever. If **no scheduler is running**, every pod stays Pending with `Node: <none>` and no events. That single fact drives most "Pending pod" triage.

Manual scheduling (no scheduler): set `spec.nodeName: node01` at creation (bypasses the matchmaker), or POST a **Binding object** to an existing pod's binding API (exactly what the scheduler does internally). You **cannot move a running pod** — delete + recreate to "reschedule."

(B) The filter levers — three ways to shrink the candidate set

1. Taints & Tolerations — the *node* repels pods. A taint on a node says "keep out unless you tolerate me"; a toleration on a pod says "I can withstand that."

```
node01 [taint: app=blue:NoSchedule]
   ▲ podA (no toleration)      → repelled → scheduled elsewhere
   └ podD (tolerates app=blue) → allowed through the filter
```

Effect	Meaning
`NoSchedule`	new intolerant pods won't schedule here
`PreferNoSchedule`	scheduler *tries* to avoid (soft)
`NoExecute`	intolerant new pods repelled **AND existing** ones **evicted**

```
kubectl taint nodes node01 app=blue:NoSchedule       # add
kubectl taint nodes node01 app=blue:NoSchedule-      # remove (trailing minus)
```

```
spec:
  tolerations:
  - key: "app"
    operator: "Equal"      # or "Exists" (then omit value)
    value: "blue"
    effect: "NoSchedule"
```

> 🎯 The **control-plane node is tainted** `node-role.kubernetes.io/control-plane:NoSchedule` — that's why your pods never land there (`kubectl describe node controlplane | grep -i taint`). **Taints only repel; they don't attract** — a tolerating pod can still go to any *other* untainted node.

2. nodeSelector / nodeAffinity — the *pod* requires a node. You must **label the node first** (`kubectl label nodes node01 size=large`).

```
# simple:
spec:
  nodeSelector: { size: large }
# advanced (operators nodeSelector can't do: In/NotIn/Exists/DoesNotExist/Gt/Lt):
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - { key: size, operator: In, values: [large, medium] }
```

Read the long type name **literally**:

Type	No matching node at scheduling	Label changes after it's running
`requiredDuringScheduling…IgnoredDuringExecution`	pod stays **Pending** (hard)	keeps running (ignored)
`preferredDuringScheduling…IgnoredDuringExecution`	placed **anywhere** (soft)	keeps running (ignored)

3. Resource requests — the pod must *fit*. The scheduler uses `requests` (not `limits`) to find a node with that much free.

```
spec:
  containers:
  - name: app
    image: nginx
    resources:
      requests: { memory: "256Mi", cpu: "250m" }   # for SCHEDULING (must fit)
      limits:   { memory: "512Mi", cpu: "500m" }   # runtime CEILING
```

- **CPU over limit → throttled** (never killed). **Memory over limit → OOMKilled** (memory can't be throttled).
- Units: CPU `1` = 1 core, `100m` = 0.1 core. Memory `Mi/Gi` = 1024-based, `M/G` = 1000-based (`256Mi ≠ 256M`).
- **LimitRange** = per-namespace default/min/max for pods that don't specify (affects **new** pods only). **ResourceQuota** = per-namespace *total* cap.

> 🎯 **Taints + affinity, the classic combo:** taints keep *other* pods **off** your node; affinity keeps *your* pod **on** it. Neither alone fully dedicates a node — **use both** for "only these pods here, and these pods only here."

(C) The bypass levers — pods that skip the matchmaker

- **DaemonSet** — one pod on **every** node, auto-added/removed as nodes join/leave (kube-proxy, CNI, log/monitoring agents). Uses node affinity internally but the scheduler otherwise ignores it. **No** `kubectl create daemonset` — make a Deployment YAML, flip `kind: DaemonSet`, drop `replicas` + `strategy`.

- **Static pod** — the **kubelet runs it with no API server/scheduler at all**, reading YAML from `/etc/kubernetes/manifests/` (path from `staticPodPath` in `/var/lib/kubelet/config.yaml`). This is **how the control plane bootstraps** (apiserver/etcd/scheduler are static pods). The API server shows a **read-only mirror** with the **node name appended** (`nginx-node01`); you delete it by **removing the file**, not via kubectl. Only *Pods* — no RS/Deployment/Service.

```
kubelet ──watches──▶ /etc/kubernetes/manifests/ ──creates──▶ static pods
   add file → created · edit → recreated · delete file → deleted · crash → restarted
```

(D) The queue & the gate — priority and admission

- **PriorityClass** (non-namespaced): higher `value` = scheduled sooner, and by default **preempts** (evicts) lower-priority pods to fit. `preemptionPolicy: Never` waits in queue but still jumps ahead. No priority = value `0`; `globalDefault: true` sets a cluster default (one only).
- **Multiple schedulers / profiles:** run a custom scheduler (pod sets `schedulerName`), or since v1.18 run **multiple profiles in one binary** (each with its own `schedulerName`, enabling/disabling plugins) — avoids the race conditions of separate processes.
- **Admission controllers** run *after* authn/authz but *before* the object is persisted — they **mutate** then **validate**, and can reject the request (no `latest` tag, no root, auto-add default StorageClass). Configured on the API server (`--enable-admission-plugins`).

```
Request → [Authentication] → [Authorization/RBAC] → [Admission: mutate → validate] → etcd →
(scheduler)
```

> ✅ **Check yourself before Rung 4:** A pod is **Pending**. Walk the filter phase: name the *three* different reasons a node could get filtered out (one about the node, one about the pod's node requirement, one about capacity). And: which field does the scheduler read to check capacity — `requests` or `limits`?

RUNG 4 — The Vocabulary Map 🏷️

Pin every scary term to the matchmaker machinery

Term	What it actually is	Which lever / phase
nodeName	The field the scheduler sets	The output of Binding
Taint	A "keep out" mark on a **node**	Filter (repel)
Toleration	A pod's permission to ignore a taint	Filter (pass the repel)
NoSchedule / PreferNoSchedule / NoExecute	Taint strengths (block / soft / block+evict)	Filter
nodeSelector	Simple "pod requires node label"	Filter (attract)
nodeAffinity	Expressive version (operators, soft/hard)	Filter + Score
required… / preferred…	Hard rule (Pending if unmet) / soft preference	Filter / Score
requests	Minimum guaranteed; used to place	Filter (fit)
limits	Runtime ceiling (CPU throttle / mem OOMKill)	not scheduling
LimitRange / ResourceQuota	Per-ns defaults / per-ns total cap	admission-time
DaemonSet	One pod per node, cluster-wide	Bypass (per-node)
Static pod	Pod the kubelet runs directly from a dir	Bypass (bootstraps control plane)
PriorityClass	Ranks pods; enables preemption	Queue order
preemption	Evicting lower-priority pods to fit a higher one	Queue/Filter
schedulerName / profile	Which scheduler (or profile) handles a pod	Whole loop
Admission controller	Validate/mutate/reject a request	Before scheduling

The big unlock: group the levers by what they do

```
REPEL (node pushes pods away):      Taints + Tolerations
ATTRACT/REQUIRE (pod pulls to node): nodeSelector + nodeAffinity
FIT (capacity):                      requests (+ LimitRange/ResourceQuota)
BYPASS (skip the scheduler):         static pods · manual nodeName · DaemonSets
QUEUE (who goes first, who gets evicted): PriorityClass + preemption
GATE (before scheduling at all):     Admission controllers
```

Six groups, one machine. Note **taint↔toleration** and **label↔nodeSelector/affinity** are each two-sided pairs (one on the node, one on the pod).

> ✅ **Check yourself before Rung 5:** Sort these into "repel / attract / fit / bypass": a `NoSchedule` taint, a `requiredDuringScheduling` nodeAffinity, a `memory: 256Mi` request, a static pod. Which two must be used *together* to truly dedicate a node?

RUNG 5 — The Trace 🎬

Follow ONE pod through the scheduler, lever by lever

Trace a **GPU training pod** onto a **dedicated GPU node**, in a cluster that also has a high-priority preemption rule. This one path touches every lever.

Step 1 — Pod is created, hits the API server. It passes authn/authz, then **admission controllers** run: a policy rejects `image: …:latest`, so the pod uses a pinned tag. Object is written to etcd with `nodeName: <empty>`.

Step 2 — Scheduling Queue (PrioritySort). The pod has `priorityClassName: high-priority` (value 1,000,000), so it's ordered ahead of default (value 0) pods waiting in the queue.

Step 3 — Filtering (feasible nodes only). The scheduler eliminates nodes:

- CPU-only nodes → **filtered out** by the pod's `nodeAffinity: gpu=true` (attract lever).
- Nodes without enough free memory for the pod's `requests` → **filtered out** (fit lever).
- The GPU node is tainted `gpu=true:NoSchedule`; the pod has a matching **toleration**, so it **passes** that repel. Other pods without the toleration were filtered off this node.

Step 4 — Preemption (if no node fits). Suppose the GPU node is full of low-priority pods. Because our pod is high-priority with `PreemptLowerPriority`, the scheduler **evicts** a low-priority pod to free room, then proceeds.

Step 5 — Scoring. Among surviving nodes (here, just the GPU node), soft signals (image already cached? `preferred` affinity?) score them; the best wins.

Step 6 — Binding. The scheduler sets `spec.nodeName: gpu-node01` (a Binding write through the API server → etcd).

Step 7 — kubelet runs it. The kubelet on `gpu-node01` (watching the API server) sees its pod and starts the container via the runtime. The scheduler's job ended at Step 6 — it *decided*, it never *placed*.

```
create → [admission: reject :latest] → etcd(nodeName:empty)
      → QUEUE(priority high, goes first)
      → FILTER(affinity gpu=true ✓ · requests fit ✓ · toleration passes taint ✓)
      → [PREEMPT a low-prio pod if full]
      → SCORE → BIND(nodeName=gpu-node01) → kubelet starts container
```

> ✅ **Check yourself before Rung 6:** In this trace, three separate levers all "chose" the GPU node — the taint/toleration, the nodeAffinity, and the resource request. State what each one contributed. And: at which single step does the scheduler's involvement *end*?

RUNG 6 — The Contrast ⚖️

The boundary of each lever defines it

This…	vs that…	The real difference
Taint/Toleration	**nodeAffinity**	node *repels* pods vs pod *requires* a node (repel vs attract)
nodeSelector	**nodeAffinity**	simple equality vs operators + soft/hard rules
requests	**limits**	schedule-time fit vs runtime ceiling
Static pod	**DaemonSet**	kubelet runs it directly (control-plane bootstrap) vs API-managed one-per-node
Manual `nodeName`	**the scheduler**	you bypass matchmaking vs it filters+scores for you
PriorityClass preemption	**plain scheduling**	can *evict* others to fit vs only uses free space

Why taints alone can't dedicate a node: a taint stops *other* pods landing, but your target pod could still schedule on any *other* untainted node — so you add **affinity** to pull it back. Two-sided problem, two levers.

When NOT to use these: don't reach for affinity/taints on a small uniform cluster (adds complexity for no benefit); don't set CPU **limits** reflexively (they can waste idle capacity via throttling — set **requests** always, limits carefully); don't hand-pin `nodeName` in production (defeats self-healing across node failures).

One-sentence "why this over that":

> Use taints to keep the wrong pods *off* a node and affinity to keep the right pods *on* it; use requests so the scheduler can place by real capacity, and reserve static pods for the control plane and DaemonSets for true one-per-node agents.

> ✅ **Check yourself before Rung 7:** A teammate tainted the GPU node and is surprised their GPU pod landed on a *different* node anyway. Explain *why* — and what single addition fixes it.

RUNG 7 — The Prediction Test 🧪

Predict BEFORE you run. A wrong prediction repairs your model.

Prediction 1 — Dedicate a node needs BOTH taint and affinity

> **My prediction:** "If I only taint `node01` for blue, then blue pods might still land elsewhere; only after I *also* add nodeAffinity requiring `app=blue` will blue pods run **exclusively** on node01 and nothing else run there — *because* taint = repel others, affinity = require this node."

```
kubectl taint nodes node01 app=blue:NoSchedule     # repel everyone else
kubectl label nodes node01 app=blue                # so affinity has something to match
```

```
spec:
  tolerations: [{ key: app, operator: Equal, value: blue, effect: NoSchedule }]
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions: [{ key: app, operator: In, values: [blue] }]
```

Verify: `kubectl get pods -o wide` → blue pods only on node01, no other pods there. Remove *one* lever and watch leakage — that's the proof you need both.

Prediction 2 — A missing toleration keeps a pod Pending

> **My prediction:** "If I taint every candidate node `NoSchedule` and my pod has no matching toleration, then it stays **Pending** with a `FailedScheduling` event mentioning taints — *because* filtering removed every node."

```
kubectl taint nodes node01 key=val:NoSchedule
kubectl run test --image=nginx
kubectl get pod test                # Pending
kubectl describe pod test | grep -A3 Events   # "node(s) had untolerated taint"
```

Verify: the event names the taint. Add a toleration (or `kubectl taint nodes node01 key=val:NoSchedule-`) and it schedules. If it scheduled anyway, another untainted node existed.

Prediction 3 — OOMKilled = memory limit too low, and it's immutable live

> **My prediction:** "If a pod's memory `limit` is below what it uses, then `describe` shows `Last State: Terminated, Reason: OOMKilled`, and `kubectl edit pod` will *refuse* to raise the limit live — *because* memory can't be throttled (hard kill) and resource limits are immutable on a running pod."

```
kubectl describe pod elephant | grep -A3 "Last State"   # OOMKilled
kubectl edit pod elephant           # raise memory limit → DENIED (immutable)
kubectl replace --force -f /tmp/kubectl-edit-XXXXX.yaml  # kubectl saved your edit here
kubectl get pod elephant -w         # Running
```

Verify: the pod runs after recreate; OOMKilled is gone. Lesson: **memory = hard kill**, and immutable pod fields need `replace --force` (or edit the Deployment, which auto-rolls).

Prediction 4 — A static pod ignores `kubectl delete`

> **My prediction:** "If I drop a manifest in `/etc/kubernetes/manifests/` the kubelet creates the pod (name gets the node suffix), and `kubectl delete pod` won't remove it — it comes right back — until I delete the **file** — *because* the kubelet owns it directly; the API object is only a mirror."

```
grep staticPodPath /var/lib/kubelet/config.yaml     # find the dir
kubectl run static-web --image=nginx --dry-run=client -o yaml \
  > /etc/kubernetes/manifests/static-web.yaml
kubectl get pods                    # static-web-<node> appears
kubectl delete pod static-web-<node>   # ...it comes back
rm /etc/kubernetes/manifests/static-web.yaml   # THIS deletes it
```

Verify: gone only after the file is removed. If the name ends in a different node, ssh there — the file lives on that node.

Prediction: "If I do X, then Y, because [mechanism]"	Ran?	Right?	If wrong, what did I miss?
1.			

🎁 CAPSTONE — Compress It

One sentence, no notes:

> Scheduling is a matchmaker that filters nodes a pod can't run on, scores the rest, and binds the best — and taints, affinity, requests, priority, static pods and DaemonSets are just levers that bias, override, or bypass that filter/score.

Explain it to a beginner in 3 sentences:

> 1. When you create a pod, a component called the scheduler picks which node it runs on by first ruling out unsuitable nodes, then ranking what's left.
> 2. You steer that decision with levers — taints push pods away from a node, affinity pulls the right pods onto it, resource requests make sure the node has room, and priority lets critical pods jump the queue (even evicting others).
> 3. A few workloads skip the scheduler entirely: static pods are run straight by the node's kubelet (that's how the control plane starts), and DaemonSets deliberately run one copy on every node.

Which rung to revisit hands-on?

- **Rung 3B — taints vs affinity.** People conflate repel and attract. Fix: do Prediction 1 and delete one lever.
- **Rung 3C — static pods.** The mirror/file distinction is slippery. Fix: Prediction 4, on a worker node.

🎯 CKA exam tips & quick notes

- **Generate, then edit:** `k run/create ... $do > f.yaml`, add the taint/affinity/resources block, `k apply -f`.
- **Taint syntax:** `key=value:Effect` to add, **append** `-` to remove. Toleration values are **quoted strings**.
- **Pending-pod triage order:** (1) is the scheduler running? (2) taints vs tolerations, (3) affinity/selector match, (4) resource fit — all via `describe pod` → Events.

- **Immutable pod fields** → `kubectl replace --force -f`. Deployment-managed pods → `kubectl edit deployment` (auto-rolls).
- **No `create daemonset`** → Deployment YAML, flip `kind`, drop `replicas` + `strategy`.
- **Static pods** in `/etc/kubernetes/manifests`; delete via the **file**. Find dir: `grep staticPodPath /var/lib/kubelet/config.yaml`.
- Editing `kube-apiserver.yaml` (admission plugins) **restarts the apiserver** — brief `kubectl` outage, then verify.
- **CPU = throttle, Memory = OOMKill; request = schedule, limit = cap.**
- `kubectl get events -o wide` → which **scheduler** placed a pod.

📌 Command cheat sheet

```
# TAINTS / LABELS / AFFINITY
k taint nodes node01 key=val:NoSchedule          # add   (…:NoSchedule- to remove)
k label nodes node01 size=large                  # label (for nodeSelector/affinity)
k describe node node01 | grep -iE 'taint|label'
# RESOURCES
k describe pod x | grep -A5 -iE 'limits|requests|Last State'   # values / OOMKilled
k replace --force -f /tmp/kubectl-edit-xxxx.yaml               # recreate after immutable edit
# DAEMONSET (no create cmd)
k create deploy ds --image=nginx $do > ds.yaml   # kind->DaemonSet, drop replicas/strategy
# STATIC PODS
grep staticPodPath /var/lib/kubelet/config.yaml
k run web --image=nginx $do > /etc/kubernetes/manifests/web.yaml
# PRIORITY / SCHEDULERS / ADMISSION
k get priorityclass
k get events -o wide                             # which scheduler placed the pod
grep -- "admission-plugins" /etc/kubernetes/manifests/kube-apiserver.yaml
```

Related sections

- Section 1 — Core Concepts — the scheduler's place in the pod-creation flow; labels/selectors.
- Section 4 — Application Lifecycle Management — configmaps/secrets/rollouts on the pods you schedule.
- Section 6 — Security — RBAC/authorization that admission controllers run *after*.
- Section 7 — Storage — the `DefaultStorageClass` mutating admission controller in action.
- Section 13 — Troubleshooting — diagnosing Pending / OOMKilled pods.
- ../../Linux/14-cgroups.md — the cgroups that enforce CPU throttle / memory OOMKill.

✅ Answers — "Check yourself before Rung N"

Before Rung 2

Q: Give two concrete failures that happen if the scheduler ignores *what kind* of node a pod needs.

A: (1) **Special hardware:** a GPU training pod lands on a cheap CPU-only node and either crashes or runs ~100x slower — the hardware guarantee is gone. (2) **Noisy neighbors:** a noisy batch job lands right next to your production API, eats all the RAM, and the API gets OOM-killed — there's no isolation. (Licensed workloads landing on non-licensed nodes, and critical pods stuck Pending behind low-value pods, are further failures from the same list.)

Before Rung 3

Q: Say the one-sentence idea. Then: a taint and a nodeAffinity rule both keep a pod off the "wrong" node — which is a *node repelling pods* and which is a *pod requiring a node*? Why do you often need both?

A: The sentence: **the scheduler is a matchmaker — for every pod with an empty `nodeName` it FILTERS out nodes that can't run it, SCORES the survivors, and BINDS the pod to the best one, and every scheduling feature is a lever that biases that filter or score (or bypasses the matchmaker entirely).** A **taint** is the *node repelling pods*: the node says "keep out unless you tolerate me." **nodeAffinity** is the *pod requiring a node*: the pod says "only nodes labeled like this." You often need both because each only solves half of dedicating a node: the taint keeps *other* pods off your node but does nothing to stop your pod landing on some other untainted node, while affinity pulls your pod to the node but doesn't keep strangers away. Taint + affinity together give "only these pods here, and these pods only here."

Before Rung 4

Q: A pod is Pending. Name the three different reasons a node could get filtered out (one about the node, one about the pod's node requirement, one about capacity). Which field does the scheduler read for capacity — `requests` or `limits`?

A: (1) **About the node — taints:** the node carries a taint (e.g. `NoSchedule`) that the pod has no toleration for, so the node repels it (TaintToleration filter). (2) **About the pod's requirement — affinity:** the pod's `nodeSelector` or `requiredDuringSchedulingIgnoredDuringExecution` nodeAffinity doesn't match the node's labels, so the node fails the pod's requirement (NodeAffinity filter). (3) **About capacity — fit:** the node doesn't have enough free CPU/memory to satisfy the pod's resource requests (NodeResourcesFit filter). The scheduler reads `requests`, never `limits` — requests are for schedule-time fit; limits are only a runtime ceiling. If no node survives all three filters, the pod stays Pending.

Before Rung 5

Q: Sort into repel / attract / fit / bypass: a `NoSchedule` taint, a `requiredDuringScheduling` nodeAffinity, a `memory: 256Mi` request, a static pod. Which two must be used together to truly dedicate a node?

A: `NoSchedule` taint = **repel** (the node pushes pods away); `requiredDuringScheduling` nodeAffinity = **attract/require** (the pod pulls itself to matching nodes); `memory: 256Mi` request = **fit** (capacity check); static pod = **bypass** (the kubelet runs it directly, skipping the scheduler). The two that must be combined to truly dedicate a node are the **taint** and the **affinity**: the taint keeps everyone else off the node, and the affinity keeps your pods on it — neither alone closes both sides.

Before Rung 6

Q: In the GPU-pod trace, three levers all "chose" the GPU node — taint/toleration, nodeAffinity, and the resource request. What did each contribute? At which step does the scheduler's involvement end?

A: The **nodeAffinity** (`gpu=true`) was the attract lever: it filtered out every CPU-only node so only GPU nodes remained candidates. The **resource request** was the fit lever: it filtered out any node without enough free memory to satisfy the pod's requests. The **taint/toleration** was the repel lever working in reverse: the GPU node's `gpu=true:NoSchedule` taint had already kept ordinary pods off it, and our pod's matching toleration let it pass that repel filter. The scheduler's involvement

ends at **Step 6 — Binding**, when it writes `spec.nodeName: gpu-node01` through the API server; it only *decides*, it never *places* — the kubelet on that node actually starts the container.

Before Rung 7

Q: A teammate tainted the GPU node and is surprised their GPU pod landed on a different node anyway. Why — and what single addition fixes it?

A: Because **taints only repel; they don't attract.** The taint keeps intolerant pods *off* the GPU node, and the teammate's pod (with its toleration) is merely *allowed* onto it — but the scheduler is still free to bind that pod to any other untainted node that passes filtering, and it did. The single fix is to add a **nodeAffinity** (or nodeSelector) on the pod requiring the GPU node's label (e.g. `gpu=true`, after labeling the node), so the pod is *required* to land there. That's the classic taint + affinity combo: repel others off, pull yours on.

Logging & Monitoring, Climbed the Ladder 🪜

Section 3 of the CKA — deriving how you *see* into a running cluster

> A compact section, but `kubectl top` and `kubectl logs` are guaranteed exam muscle memory. We climb from **the pain of a black-box cluster → the two-windows idea → the data paths** → then the commands as predictions. Each rung ends with a ✅ **Check yourself**.

RUNG 0 — The Setup 🎯

What am I learning? The two built-in ways to observe a cluster: live resource usage (`kubectl top`, via the Metrics Server) and application output (`kubectl logs`).

Why did it land on my desk? Every troubleshooting task (30% of the CKA) starts with "look at the metrics and the logs." These two commands are reflexes you need under a 2-hour clock.

What do I already know? Probably `kubectl logs`. What's fuzzy: *where* the numbers in `kubectl top` come from, why it needs a separate install, and why a crashed pod's logs seem empty (the `--previous` trick).

RUNG 1 — The Pain 🔥

Why does cluster observability exist at all?

A running cluster is a black box. Without a window in:

```
THE BLACK-BOX PAIN
  "which pod is eating all the memory?"  → no idea; you SSH to nodes and run top by hand
  "why did the app just crash-loop?"      → the container's gone; its output vanished with it
  "is this node about to fall over?"      → you find out when pods start getting OOMKilled
```

Before / without it: SSH to each node, `docker logs` per container, eyeball `top` — none of which correlates to *pods*, and all of which disappears when a container restarts. And **Kubernetes ships no full monitoring stack** — you must add one.

What breaks without it: you can't size requests/limits (Section 2), can't drive autoscaling, and can't debug a `CrashLoopBackOff` because the failing container's output is already gone.

Who feels it most? Whoever is on call — you get paged for a symptom and need the metrics and the logs to find the cause fast.

> ✅ **Check yourself before Rung 2:** Why is "just run `docker logs` on the node" a poor answer in Kubernetes? (Hint: what happens to a crashed container, and does the node view map to *pods*?)

RUNG 2 — The One Idea 💡

The single sentence everything hangs off

> **Kubernetes gives you two windows into a live cluster — the Metrics Server aggregates real-time CPU/memory from every kubelet's built-in cAdvisor for `kubectl top`, and the container runtime captures each container's stdout/stderr for `kubectl logs` — and by default neither keeps any history.**

Derivations:

- *"Metrics Server aggregates… for `kubectl top`"* → `top` fails with "Metrics API not available" until you **install** the Metrics Server; it's not there by default.
- *"from every kubelet's cAdvisor"* → the metric source already lives *inside* the kubelet; the Metrics Server just scrapes and sums.
- *"runtime captures stdout/stderr"* → `kubectl logs` = your app's stdout. If the app logs to a *file* inside the container, `kubectl logs` won't see it.
- *"neither keeps history"* → Metrics Server is **in-memory** (no graphs/past data), and a restarted container's live logs start fresh — hence `--previous` to read the dead one.

> ✅ **Check yourself before Rung 3:** If `kubectl top pod` returns numbers but `kubectl top node` was empty a minute ago, what changed — and what does that tell you about *where* the data comes from and *when* it's ready?

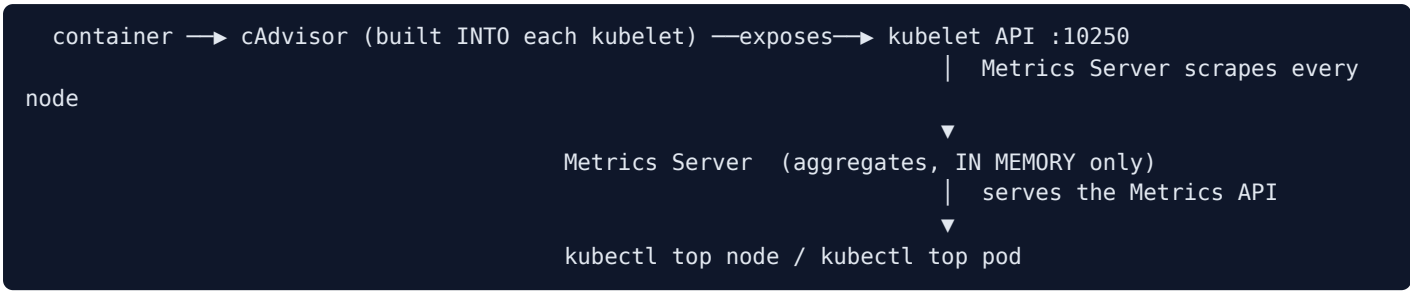
RUNG 3 — The Machinery ⚙️

The two data paths — go slow

> ### 🧸 Plain-English first (read this before the technical version)
>
> Imagine you manage a large apartment building and want to know two different things: **how much electricity and water each apartment is using right now**, and **what the tenants have been saying**. Kubernetes gives you one separate system for each — and neither keeps any history.
>
> **(A) The usage-meter path (metrics).** Every floor of the building has a caretaker (the "kubelet" — the worker program on each computer), and each caretaker has a built-in meter reader (called "cAdvisor") that constantly measures how much power and space every apartment (each running app) is using. A single clerk for the whole building (the "Metrics Server") walks around, reads every floor's meters, and adds it all up on a whiteboard in his head. When you ask "who's using the most memory right now?" (the `kubectl top` command), you're asking that clerk. Three things follow:
>
> - The clerk isn't hired by default — you must bring him in yourself (install the Metrics Server), and give him a minute or two to do his first rounds before he can answer.
> - He keeps everything **in his head only** — no notebook. So there are no charts of yesterday, no trends. If you want recorded history and graphs, you hire a full bookkeeping firm (add-on tools like Prometheus or EFK). An older clerk named Heapster was retired.
> - In practice labs, the clerk sometimes needs permission to skip a strict ID check at each floor's door (a certificate setting) — fine for practice, never for a real building.
>
> **(B) The message path (logs).** Whatever an app "says" — its printed output — is caught by the machine running it (the container runtime) like an answering machine taping every word. Asking for a pod's logs replays that tape. Key wrinkles:
>
> - If an apartment houses **two tenants** (a pod with multiple containers), you must say *which* tenant's tape you want.
> - If a tenant **stormed out and was replaced** (the container crashed and restarted), the new tenant's tape is nearly blank — the useful complaint is on the **previous** tenant's tape, which you must ask for specifically (the `--previous` flag). This is the number-one trick for debugging crash loops.
> - And note: the tape only records what's spoken aloud (standard output) — if the app writes its diary in a private file instead, the tape has nothing.
>
> The two systems share no plumbing: the meters need a special install; the tapes work from day one.

Now the original technical deep-dive — the same ideas, in precise form:

(A) The metrics path



Every node's **kubelet** embeds **cAdvisor (Container Advisor)**, which measures per-container CPU/memory and exposes it on the kubelet API. The **Metrics Server** (one per cluster, lightweight, **in-memory**) scrapes all nodes and serves the aggregated **Metrics API** that `kubectl top` reads. No history — for that you'd add **Prometheus** or **EFK** (the deprecated **Heapster** was folded into Metrics Server).

```
# deploy it (not present by default):
kubectl apply -f https://github.com/kubernetes-sigs/metrics-
server/releases/latest/download/components.yaml
# minikube: minikube addons enable metrics-server
# give it ~1–2 min to scrape before `top` works
```

> ⚠️ In kubeadm/lab clusters the kubelet serving cert is self-signed, so the Metrics Server Deployment often needs `--kubelet-insecure-tls` in its args. **Never** use that in production.

```
kubectl top node                # CPU (millicores) + memory per node
kubectl top pod -A              # all namespaces
kubectl top pod --sort-by=cpu   # busiest first
kubectl top pod --containers    # break down by container
```

(B) The logs path

Apps write to **stdout/stderr**; the **container runtime** captures that stream; the kubelet exposes it; `kubectl logs` reads it (same idea as `docker logs -f`).

```
kubectl logs -f my-pod                  # live tail
kubectl logs my-pod --previous          # the CRASHED previous container (CrashLoopBackOff!)
kubectl logs my-pod --since=1h --tail=50
# multi-container pod → you MUST name the container:
kubectl logs my-pod -c event-simulator  # (else: "a container name must be specified")
kubectl logs my-pod --all-containers
```

> 🎯 **CKA tip:** `--previous` (`-p`) is the single most useful log flag. A `CrashLoopBackOff` pod's *current* container may have no logs yet — the *previous, crashed* one holds the actual error. Always pair `kubectl logs` (app errors) with `kubectl describe pod` (Events + Last State = scheduling/runtime errors).

> ✅ **Check yourself before Rung 4:** Name the component that (1) *measures* CPU/memory, (2) *aggregates* it cluster-wide, and (3) *captures* your app's log lines. Which one is in-memory-only, and what's the consequence?

RUNG 4 — The Vocabulary Map 🏷️

Term	What it actually is	Which path
cAdvisor	Metric collector built into every kubelet	Metrics — the source
Metrics Server	Cluster-wide aggregator, in-memory, serves Metrics API	Metrics — the hub
Metrics API	The API `kubectl top` reads	Metrics — the interface
`kubectl top`	CLI for live node/pod CPU+memory	Metrics — the window
Heapster	Old aggregator, **deprecated** → Metrics Server	Metrics (history)
Prometheus / EFK	Full historical monitoring stacks (add-on)	Metrics (history)
stdout/stderr	The stream `kubectl logs` shows	Logs — the source
`kubectl logs -c`	Pick a container in a multi-container pod	Logs
`--previous` / `-p`	Logs from the crashed prior container	Logs (debugging)

The unlock: two independent pipelines — **metrics** (cAdvisor → Metrics Server → `top`) and **logs** (stdout → runtime → `logs`). They don't share plumbing; `top` needs an install, `logs` works out of the box.

> ✅ **Check yourself before Rung 5:** `kubectl top` errors but `kubectl logs` works fine. Which pipeline is broken, and what's the fix?

RUNG 5 — The Trace 🎬

Trace A — "Which pod uses the most memory?"

1. You run `kubectl top pod --sort-by=memory`.
2. kubectl calls the **Metrics API** on the API server.
3. That's served by the **Metrics Server**, which returns its latest in-memory snapshot.
4. That snapshot was built by **scraping each node's kubelet/cAdvisor**.
5. kubectl sorts and prints — top row = biggest memory user. (If step 3 has no data yet, you get "Metrics API not available" — it hasn't scraped, or isn't installed.)

Trace B — "Why did webapp crash-loop?"

1. `kubectl get pod webapp` → `CrashLoopBackOff`, `RESTARTS: 5`.
2. `kubectl logs webapp` → empty/partial (the *current* attempt just started).
3. `kubectl logs webapp --previous` → the **runtime's captured stderr of the dead container** shows the real panic ("config key missing").
4. `kubectl describe pod webapp` confirms `Last State: Terminated, Reason: Error`. Cause found — fix the config (Section 4).

> ✅ **Check yourself before Rung 6:** In Trace B, why did step 2 look empty while step 3 had the error? What does that say about where a restarted container's logs go?

RUNG 6 — The Contrast ⚖️

This…	vs that…	Difference
Metrics Server	**Prometheus / EFK**	live + in-memory, no history vs full historical graphs/alerting
`kubectl top`	`kubectl describe` **(resources)**	*actual* current usage vs *requested/limit* configuration
`kubectl logs`	`kubectl describe pod`	*app* errors (stdout) vs *scheduling/runtime* errors (Events, Last State)
`logs` **(current)**	`logs --previous`	this container's run vs the crashed prior run

When NOT to rely on Metrics Server: anything needing history, dashboards, or alerting — that's Prometheus/EFK territory. Metrics Server exists to feed `kubectl top` and the autoscalers, nothing more.

One-sentence "why this over that":

> Use `kubectl top` + Metrics Server for a fast live snapshot and to size/autoscale; use `kubectl logs --previous` + `describe` to find *why* a pod broke; reach for Prometheus/EFK only when you need history.

> ✅ **Check yourself before Rung 7:** A pod's `kubectl top` shows low CPU but it's still slow. Name one thing `top` won't tell you that `logs` or `describe` might.

RUNG 7 — The Prediction Test 🧪

Prediction 1 — `top` fails until the Metrics Server has scraped

> **My prediction:** "If I run `kubectl top pod` on a fresh cluster, it errors with *Metrics API not available*; after I deploy the Metrics Server and wait ~1 min, it returns numbers — *because* the Metrics API has no server behind it until then."

```
kubectl top pod                          # error
kubectl apply -f https://github.com/kubernetes-sigs/metrics-
server/releases/latest/download/components.yaml
kubectl -n kube-system rollout status deploy/metrics-server
kubectl top node ; kubectl top pod --sort-by=memory
```

Verify: numbers appear after the rollout + a short wait. Still failing? `kubectl -n kube-system logs deploy/metrics-server` — usually the `--kubelet-insecure-tls` cert issue.

Prediction 2 — A multi-container pod demands `-c`

> **My prediction:** "If I `kubectl logs` a two-container pod without `-c`, it errors and lists the container names; adding `-c <name>` returns that container's logs — *because* logs are per-container and Kubernetes won't guess which one."

```
kubectl logs webapp-2                  # error: "a container name must be specified"
kubectl get pod webapp-2 -o jsonpath='{.spec.containers[*].name}{"\n"}'
kubectl logs webapp-2 -c webapp | grep -i warn
```

Verify: the error lists the names; `-c` returns logs. `--all-containers` merges them.

Prediction 3 — `--previous` reveals the crash cause

> **My prediction:** "If a pod is `CrashLoopBackOff`, plain `logs` is thin but `logs --previous` shows the real error — *because* the current container just (re)started, while the previous one is the one that actually failed."

```
kubectl get pod webapp                 # CrashLoopBackOff, RESTARTS climbing
kubectl logs webapp                    # little/nothing
kubectl logs webapp --previous         # the actual panic/stacktrace
kubectl describe pod webapp | grep -A3 "Last State"
```

Verify: `--previous` shows the failure; `describe` confirms `Reason`. That pair localizes almost any crash.

Prediction: "If I do X, then Y, because [mechanism]"	Ran?	Right?	Miss?
1.			

🎁 CAPSTONE — Compress It

One sentence, no notes:

> `kubectl top` reads live CPU/memory that the Metrics Server aggregates from every kubelet's cAdvisor (in-memory, no history), while `kubectl logs` streams a container's stdout/stderr — use `--previous` to read a crashed container and `describe` for scheduling/runtime errors.

Explain it to a beginner in 3 sentences:

> 1. Kubernetes doesn't ship monitoring, so you install a small Metrics Server that gathers CPU/memory from every node and powers `kubectl top`.
> 2. To see what an app printed, `kubectl logs` shows its output — and `--previous` recovers the output of a container that already crashed.
> 3. Metrics and logs are separate pipelines with no memory of the past, so for history you add Prometheus or EFK.

Which rung to revisit hands-on? Rung 7 Prediction 3 — the `--previous` reflex is the one that saves you on a `CrashLoopBackOff` exam task.

🎯 CKA exam tips & quick notes

- `kubectl top` needs the **Metrics Server running** + ~1 min. Sort `--sort-by=cpu|memory`; scope `-A` / `-n`.
- `kubectl logs -c <container>` is mandatory for multi-container pods.
- `kubectl logs --previous` = crashed container — your `CrashLoopBackOff` go-to.
- Metrics Server is **in-memory** → no history, no graphs.
- Pair `logs` (app errors) with `describe pod` (Events, Last State/Reason = scheduling/runtime).

📌 Command cheat sheet

```
# MONITORING
kubectl top node                        kubectl top pod -A
kubectl top pod --sort-by=cpu           kubectl top pod --containers
# LOGS
kubectl logs -f my-pod                   # live tail
kubectl logs my-pod --previous           # crashed container
kubectl logs my-pod -c app               # multi-container
kubectl logs my-pod --tail=50 --since=1h
```

Related sections

- Section 13 — Troubleshooting — logs + describe + events as the core debugging loop.
- Section 2 — Scheduling — requests/limits that `kubectl top` helps you size.
- Section 4 — Application Lifecycle Management — CrashLoopBackOff from bad configs, read via `logs`.
- Section 1 — Core Concepts — where the kubelet/cAdvisor sit in the architecture.
- ../../Linux/21-performance-monitoring.md — `top` / `vmstat` and the Linux resource metrics underneath cAdvisor.

✅ Answers — "Check yourself before Rung N"

Before Rung 2

Q: Why is "just run `docker logs` on the node" a poor answer in Kubernetes?

A: Two reasons. First, when a container crashes and restarts, its output vanishes with it — the node-level view has no memory, so the very failure you're debugging is the one whose logs are gone (Kubernetes solves this with `kubectl logs --previous`). Second, the node view doesn't map to *pods*: you'd have to SSH to each node and correlate raw container IDs to Kubernetes pods by hand, which doesn't scale and breaks the moment the scheduler moves a pod to another node. Kubernetes gives you a pod-centric window (`kubectl logs <pod>`) that works from anywhere via the API.

Before Rung 3

Q: If `kubectl top pod` returns numbers but `kubectl top node` was empty a minute ago, what changed — and what does that tell you about *where* the data comes from and *when* it's ready?

A: What changed is that the **Metrics Server finished its first scrape**: it needs ~1–2 minutes after deployment to scrape every node's kubelet before it has anything to serve. That tells you the data does not live in the API server or etcd — it comes from each kubelet's built-in **cAdvisor**, which the Metrics Server scrapes, aggregates **in memory**, and serves via the Metrics API that `kubectl top` reads. So `top` is only ready once the Metrics Server is installed, rolled out, and has completed a scrape cycle — and because it's in-memory only, there is never any history.

Before Rung 4

Q: Name the component that (1) *measures* CPU/memory, (2) *aggregates* it cluster-wide, and (3) *captures* your app's log lines. Which one is in-memory-only, and what's the consequence?

A: (1) **cAdvisor**, built into every kubelet, measures per-container CPU/memory and exposes it on the kubelet API (:10250). (2) The **Metrics Server** — one per cluster — scrapes every node's kubelet/cAdvisor and aggregates the results, serving them via the Metrics API. (3) The **container runtime** captures each container's stdout/stderr stream, which `kubectl logs` reads. The in-memory-only one is the **Metrics Server**: the consequence is no history — no graphs, no past data, just the latest snapshot. For historical monitoring you must add Prometheus or EFK.

Before Rung 5

Q: `kubectl top` errors but `kubectl logs` works fine. Which pipeline is broken, and what's the fix?

A: The **metrics pipeline** (cAdvisor → Metrics Server → Metrics API → `top`) is broken — logs are a completely independent pipeline (stdout → runtime → `logs`) that works out of the box, which is why it's unaffected. The fix: install the Metrics Server (`kubectl apply -f .../metrics-server/.../components.yaml`), wait ~1–2 minutes for it to scrape, and if it still fails check `kubectl -n kube-system logs deploy/metrics-server` — in kubeadm/lab clusters the usual culprit is the self-signed kubelet cert, fixed by adding `--kubelet-insecure-tls` to the Metrics Server's args (lab only, never production).

Before Rung 6

Q: In Trace B, why did step 2 (`kubectl logs webapp`) look empty while step 3 (`--previous`) had the error? What does that say about where a restarted container's logs go?

A: Because in a CrashLoopBackOff the *current* container has only just been (re)started — it hasn't produced (or reproduced) the failure output yet — while the container that actually crashed is the *previous* one. Plain `kubectl logs` reads only the current container's stream, so it looks empty; `--previous` reads the runtime's captured stderr of the dead prior container, which holds the real panic ("config key missing"). The lesson: a restarted container's logs start fresh — the old run's output isn't merged in, it's kept separately as the "previous" container's logs and is only reachable via `--previous` (`-p`).

Before Rung 7

Q: A pod's `kubectl top` shows low CPU but it's still slow. Name one thing `top` won't tell you that `logs` or `describe` might.

A: `top` only shows current CPU/memory usage — it says nothing about *why* the app is slow. `kubectl logs` might show application-level errors: timeouts to a downstream dependency, retries, slow-query warnings — app problems that consume no CPU. `kubectl describe pod` might show scheduling/runtime causes in Events and Last State: e.g. the container was recently OOMKilled and restarted, readiness failures, or (per the contrast table) that the pod's *requested/limit* configuration

is throttling-relevant — `top` shows actual usage, `describe` shows the configured requests/limits. Low CPU with slowness usually means the bottleneck is waiting, not computing — and only logs/describe reveal waiting.

Application Lifecycle, Climbed the Ladder 🪜

Section 4 of the CKA — deriving how apps are configured, updated, and scaled

> Running and evolving apps: rollouts/rollbacks, commands/args, env + ConfigMaps + Secrets, and multi-container/init/sidecar patterns. We climb from **the pain of updating apps by hand** → **the one "it's all the pod template" idea** → **the machinery** → then the commands as predictions. Each rung ends with a ✅ **Check yourself**.

RUNG 0 — The Setup 🎯

What am I learning? Everything about an app's life *after* you can create a pod: updating it with zero downtime, feeding it config and secrets, controlling its process, running helper containers, and scaling it.

Why did it land on my desk? *Workloads & Scheduling* is 15% of the CKA, and ConfigMaps/Secrets/rollouts show up in tasks and in troubleshooting (bad config = `CrashLoopBackOff`). This is high-YAML, high-imperative-shortcut territory.

What do I already know? Probably `kubectl set image`. What's fuzzy: why a rollout is "safe," the `command` ↔ `args` Docker trap, and that Secrets are *not* encrypted.

RUNG 1 — The Pain 🔥

Why does lifecycle management exist at all?

You can create a Deployment. Now real life happens: you need to ship v2, change a setting, rotate a password, and handle a spike — without an outage.

```
LIFE WITHOUT LIFECYCLE MANAGEMENT (the pain)

  new version    -> stop ALL v1, start ALL v2          -> downtime window, and no undo
  change a config value -> rebuild the image, redeploy -> 20-min loop for a one-line change
  a password     -> baked into the image / in Git      -> leaked forever
  bad deploy     -> "how do we get back to yesterday?"  -> panic, manual rollback
  traffic spike -> SSH in and start more copies        -> too slow, done by hand
```

Before / without it: config and secrets were baked into images (rebuild to change anything), updates were stop-the-world, and there was no revision history to roll back to.

What breaks without it: availability (no zero-downtime updates), agility (config changes need image rebuilds), security (secrets in images), and recoverability (no rollback).

Who feels it most? Everyone shipping software — but the platform team owns making updates *safe and reversible* for every team.

> ✅ **Check yourself before Rung 2:** Name two things that force an image rebuild in the "before" world that Kubernetes lets you change *without* touching the image. (Hint: a setting, and a password.)

RUNG 2 — The One Idea 💡

The single sentence everything hangs off

> **An app's entire life — the process it runs, the config and secrets it reads, how many copies exist, and how it's upgraded — is described in the Deployment's pod template; change the template and the Deployment performs a rollout: a new ReplicaSet scaled up as the old one scales down, recorded as a revision you can reverse.**

Derivations:

- *"the process it runs"* → `command` / `args` in the template override the image's ENTRYPOINT/CMD.
- *"config and secrets it reads"* → `env` / ConfigMaps / Secrets injected into the template; changing them is a template change → a new rollout.
- *"how many copies"* → `replicas` (scaling) — or an HPA driving it from metrics.
- *"how it's upgraded... new RS up as old scales down"* → this is why updates are **zero-downtime** and **contained**: a bad new version only affects the *new* ReplicaSet while the old keeps serving.
- *"recorded as a revision you can reverse"* → `kubectl rollout undo` = reconcile back to a previous template.

Once you see **every change is "edit the pod template, Kubernetes reconciles,"** rollouts, config, and scaling stop being separate topics.

> ✅ **Check yourself before Rung 3:** If you `kubectl set image` a Deployment to a broken image, why do your *existing* users usually keep getting served during the failed rollout? (Hint: which ReplicaSet are the broken pods in?)

RUNG 3 — The Machinery ⚙️

How it ACTUALLY works — go slow

🧸 Plain-English first (read this before the technical version)

Think of your app as a restaurant kitchen staffed by several identical cooks, all trained from one **master recipe card** (the "pod template"). This section is about five things: how the kitchen switches to a new recipe without closing, how you tell a cook exactly what to do, how you hand cooks their settings and passwords, how helper staff work, and how you add more cooks.

(A) Switching recipes without closing (rollouts). When you update the recipe, the kitchen doesn't fire everyone at once. It hires one new-recipe cook, waits until that cook proves they can actually cook (passes a health check), then lets one old-recipe cook go — and repeats, batch by batch, so the restaurant never stops serving. Every recipe version is kept in a filing drawer (a "revision"), so if the new dish is a disaster you say "go back to yesterday's recipe" (rollback) and the swap runs in reverse. There's also a blunt alternative — fire everyone, then hire the new crew ("Recreate") — which closes the restaurant briefly and is only for when old and new can't work side by side. Crucially, if the new cooks turn out to be broken, the old crew keeps serving: a bad update stalls instead of taking you down.

(B) Telling the cook what to do (command and args). Each cook runs exactly one job, and quits when it ends. The training manual (the app's packaged image) has a default job and default instructions; the recipe card can override either. The classic trap: the two override fields don't line up with the names people expect from the packaging tool (Docker) — one replaces the *program*, the other its *arguments*.

(C) Settings and passwords (ConfigMaps and Secrets). Instead of laminating settings into the training manual (rebuilding the image for every change), you keep them on separate slips of paper: a pinboard note for ordinary settings (a "ConfigMap") and a sealed envelope for sensitive ones (a "Secret"). Either can be handed to a cook three ways: pin the whole sheet up, copy out one line, or leave it as a file in their station. One big warning: the "sealed" envelope is only *scrambled in a reversible way* (base64 encoding), not locked — anyone with access to the record room can read it. Real protection means locking the record room itself ("encryption at rest") and controlling who may enter.

(D) Helper staff (multi-container patterns). A cooking station (pod) can hold more than one worker, sharing the same counter and phone line: a **prep worker** ("init container") who must finish setup — e.g. wait until the pantry is stocked — before the cook may start; and a **standing assistant** ("sidecar") who works alongside the cook the whole shift, e.g. carrying notes to the office.

(E) More cooks (scaling). You can simply order five cooks instead of three, or install a thermostat (an "autoscaler") that hires and releases cooks automatically based on how busy the kitchen gets.

Now the original technical deep-dive — the same ideas, in precise form:

(A) The rollout engine

A Deployment version = **rollout → new ReplicaSet → revision**. Kubernetes scales the new RS up while scaling the old down, one batch at a time.

```
RollingUpdate (default):  [v1 v1 v1 v1] → [v1 v1 v1 v2] → … → [v2 v2 v2 v2]   (seamless)
Recreate:                 [v1 v1 v1 v1] → [           ] → [v2 v2 v2 v2]        (downtime!)
```

Strategy	Behavior	Downtime
RollingUpdate (default)	new RS up / old RS down in batches (`maxSurge` , `maxUnavailable` , default 25% each)	none
Recreate	kill all old, then create all new	brief outage

```
kubectl rollout status deployment/web            # watch it
kubectl rollout history deployment/web           # revisions
kubectl set image deployment/web nginx=nginx:1.26   # imperative update (drifts from your file)
kubectl apply -f deploy.yaml                        # declarative (file = source of truth)
kubectl rollout undo deployment/web                 # roll back to previous
kubectl rollout undo deployment/web --to-revision=2
```

> 🎯 `set image` is fastest but your YAML no longer matches the live object. Switch strategy by editing `spec.strategy.type` (remove the `rollingUpdate:` block for `Recreate`). A bad new version = `CrashLoopBackOff` / `ImagePullBackOff` on the *new* RS → `kubectl rollout undo` .

(B) The process it runs — command & args

A container runs **one process**; when it exits, the container exits. The image's `ENTRYPOINT` / `CMD` set that process; the pod overrides them:

Docker	Pod field	Overrides
`ENTRYPOINT ["sleep"]`	`command:`	the executable
`CMD ["5"]`	`args:`	its default arguments

```
# Image: ENTRYPOINT ["sleep"]  CMD ["5"]  → default "sleep 5"
spec:
  containers:
  - name: ubuntu-sleeper
    image: ubuntu-sleeper
    command: ["sleep"]     # overrides ENTRYPOINT
    args: ["10"]           # overrides CMD → "sleep 10"
```

⚠️ **The trap:** `command` = ENTRYPOINT, **not** Docker's `CMD` . Array elements must be **strings** (`["sleep","5000"]`). Imperatively: everything after `--` is **args**; add `--command` to override the entrypoint too:

```
kubectl run web --image=webapp-color -- --color green     # args
kubectl run app --image=busybox --command -- sleep 1000   # command
```

(C) Config & secrets — externalize what varies

ConfigMap (non-secret) and **Secret** (sensitive) are the same shape; you **create** then **inject**.

```
kubectl create configmap app-config --from-literal=APP_COLOR=blue --from-file=app.properties
kubectl create secret generic db-secret --from-literal=DB_PASSWORD='p@ss'
echo -n 'mysql' | base64        # encode for declarative YAML   |   base64 -d to decode
```

Inject three ways (identical for ConfigMap/Secret — swap `configMapRef` ↔ `secretRef`):

```
envFrom:                                    # (a) whole object as env vars
- configMapRef: { name: app-config }
- secretRef:    { name: db-secret }
env:                                         # (b) one key → one env var
- name: APP_COLOR
  valueFrom: { configMapKeyRef: { name: app-config, key: APP_COLOR } }
volumes:                                     # (c) as files in a volume
- name: cfg
  configMap: { name: app-config }
```

> ⚠️ **Secrets are base64-*encoded*, not encrypted** — anyone with etcd/API access can decode them. Safety comes from *practice*: don't commit secret YAML, enable **encryption at rest**, use RBAC. Kubernetes helps: a Secret is only sent to nodes that need it and the kubelet keeps it in **tmpfs (RAM)**.

Encryption at rest (make etcd store secrets encrypted):

```
# 1) EncryptionConfiguration with an aescbc/secretbox provider + a base64 32-byte key
# 2) add to kube-apiserver.yaml:  --encryption-provider-config=/etc/kubernetes/enc/enc.yaml (+ mount)
# 3) verify it's no longer plaintext in etcd:
ETCDCTL_API=3 etcdctl --cacert=... --cert=... --key=... get /registry/secrets/default/my-secret |
hexdump -C
kubectl get secrets -A -o json | kubectl replace -f -   # re-encrypt EXISTING secrets
```

Provider **order matters** — the *first* encrypts; put `identity` (no encryption) last. Only new/updated secrets get encrypted until you re-`replace`.

(D) Multi-container patterns

Helpers that share the pod's **network (localhost), storage, and lifecycle** without merging code:

```
┌──────────────────── Pod ────────────────────┐
│ initContainers (run to completion,          │  ← wait for DB, run migrations (in ORDER, exit 0)
│    in ORDER)                                │
│           ↓ then                            │
│ containers: [ main-app, sidecar ]           │  ← run together for the pod's life
│    share localhost + volumes                │
└─────────────────────────────────────────────┘
```

Pattern	Where	Lifecycle
Co-located	extra entries in `containers:`	start together, run for pod life
Init	`initContainers:`	sequential, **to completion**, before main; each exits 0
Sidecar (native v1.33+)	`initContainers:` + `restartPolicy: Always`	starts first, **stays running** alongside

```
spec:
  initContainers:
  - name: init-db
    image: busybox:1.31
    command: ["sh","-c","until nslookup mydb; do sleep 2; done"]
  - name: log-sidecar               # native sidecar
    image: busybox:1.31
    restartPolicy: Always
    command: ["sh","-c","while true; do echo shipping logs; sleep 10; done"]
  containers:
  - { name: app, image: busybox:1.28, command: ["sh","-c","echo run && sleep 3600"] }
```

> 🎯 A pod stuck `Init:0/2` is waiting on an **init container** → `kubectl logs <pod> -c <init-name>`. `restartPolicy` is **container-level** (a crashed container restarts in place).

(E) Scaling

```
kubectl scale deployment web --replicas=5
kubectl autoscale deployment web --min=2 --max=10 --cpu-percent=70   # HPA (needs Metrics Server)
```

> ✅ **Check yourself before Rung 4:** Three questions, one each: (1) which pod field overrides the image's ENTRYPOINT? (2) which injection method makes a ConfigMap show up as *files*? (3) does base64-encoding a Secret make it secure — why/why not?

RUNG 4 — The Vocabulary Map 🏷️

Term	What it actually is	Machinery
Rollout	The process of moving to a new template version	Rollout engine
Revision	A saved snapshot of a rollout (for rollback)	`rollout history/undo`
RollingUpdate / Recreate	Strategies: batched swap / stop-all-start-all	`spec.strategy`
maxSurge / maxUnavailable	How many extra/absent pods during a roll	RollingUpdate knobs
command / args	ENTRYPOINT / CMD overrides	The container process
env / envFrom / valueFrom	Inject one var / whole object / one key	Config injection
ConfigMap	Externalized non-secret config	Config
Secret	Base64 (not encrypted) sensitive config	Config
encryption at rest	Make etcd store secrets encrypted	apiserver provider
initContainer	Runs to completion before the app	Multi-container
sidecar	Long-running helper (init + `restartPolicy: Always`)	Multi-container
restartPolicy	Always/OnFailure/Never, per **container**	Lifecycle
HPA	Autoscaler driving `replicas` from metrics	Scaling

The unlock: everything is a **pod-template field** (`command`, `env`, `initContainers`, `replicas`). Change any → new template → the Deployment reconciles via a rollout. ConfigMap and Secret are *the same object* with different sensitivity + injection keyword.

> ✅ **Check yourself before Rung 5:** Which of these trigger a new rollout when changed on a Deployment: `image`, `env`, `replicas`? (Two do something different from the third — which, and why?)

RUNG 5 — The Trace 🎬

Trace — a rolling update, then a rollback:

1. `kubectl set image deployment/web nginx=nginx:1.26` edits the **pod template**.
2. The Deployment controller creates a **new ReplicaSet** (revision 2) at 0 replicas.
3. It scales the **new RS +1** (`maxSurge`) and, as those pods pass **readiness**, scales the **old RS −1** (`maxUnavailable`). Repeat batch by batch.
4. At each step total capacity stays ~100%, so **users never see downtime**; the old RS serves until the new pods are Ready.
5. You push a bad `nginx:doesnotexist`: the new RS's pods go `ImagePullBackOff` and **never become Ready**, so the rollout **stalls with the old RS still serving** — the failure is contained.
6. `kubectl rollout undo deployment/web` re-selects the previous template; the good old RS scales back to full, the bad RS to 0.

```
set image ─▶ template v2 ─▶ new RS(0) ──surge+1──▶ new pods Ready? ──yes──▶ old RS −1 … ─▶ all v2
                                                  └── no (bad image) → stall, old RS keeps serving
undo ─▶ template v1 ─▶ old RS back to full, bad RS → 0
```

Trace — config reaches the app: at container start, the kubelet reads the referenced ConfigMap/Secret and sets the env vars (or mounts the volume) *before* running the process, so `printenv` inside shows `APP_COLOR`. Change the ConfigMap later → env vars **don't** update live (you must roll the pods); a mounted volume *does* eventually update.

> ✅ **Check yourself before Rung 6:** In step 5, what specific pod condition stops the rollout from proceeding — and why is that the safety mechanism, not a bug?

RUNG 6 — The Contrast ⚖️

This…	vs that…	Difference
RollingUpdate	**Recreate**	zero-downtime batched swap vs stop-all (needed when versions can't coexist)
command	**args**	overrides ENTRYPOINT (the program) vs CMD (its arguments)
ConfigMap	**Secret**	non-sensitive plaintext vs base64 sensitive (+ tmpfs, need-to-know delivery)
env injection	**volume mount**	fixed at start, no live update vs file that can update live
init container	**sidecar**	runs once to completion, blocks app vs runs alongside for pod life
`set image`	`apply -f`	fast but drifts from file vs file stays source of truth

When NOT to: don't use `Recreate` for a stateless web app (needless downtime); don't put real secrets in a ConfigMap; don't rely on env-injected config updating live (it won't — roll the pods).

One-sentence "why this over that":

> Edit the pod template and let a RollingUpdate reconcile for zero-downtime, reversible changes; externalize settings to ConfigMaps and sensitive values to Secrets (with encryption at rest); use init containers to *gate* startup and sidecars to *accompany* the app.

> ✅ **Check yourself before Rung 7:** A colleague changed a ConfigMap value but the running pods still show the old value. Explain *why*, and what makes it take effect.

RUNG 7 — The Prediction Test 🧪

Prediction 1 — A bad image is contained to the new ReplicaSet

> **My prediction:** "If I `set image` to a non-existent tag, then new pods go `ImagePullBackOff` and the rollout stalls, but the app stays up — *because* the old RS keeps serving until the new pods are Ready, which they never become; `rollout undo` restores it."

```
kubectl create deployment web --image=nginx:1.25 --replicas=4
kubectl set image deployment/web nginx=nginx:1.26 && kubectl rollout status deployment/web
kubectl set image deployment/web nginx=nginx:doesnotexist
kubectl get pods                # new pods ImagePullBackOff; old still Running
kubectl rollout undo deployment/web
```

Verify: during the bad roll, old pods keep serving; after `undo`, all run 1.26. If the whole app went down, check you weren't on `Recreate`.

Prediction 2 — Injected config is immutable on a live pod

> **My prediction:** "If I `kubectl edit pod` to add `envFrom`, it's rejected (immutable), and I must `replace --force`; afterward `printenv` shows the ConfigMap + Secret values — *because* env is set at container start and can't change on a live pod."

```
kubectl create configmap web-config --from-literal=APP_COLOR=dark-blue
kubectl create secret generic db-secret --from-literal=DB_PASSWORD='p@ss123'
kubectl edit pod webapp-color        # add envFrom → save fails → kubectl saves to /tmp
kubectl replace --force -f /tmp/kubectl-edit-XXXXX.yaml
kubectl exec webapp-color -- printenv | grep -E 'APP_COLOR|DB_PASSWORD'
```

Verify: both values print after recreate. On a Deployment you'd just `kubectl edit deployment` (it rolls new pods automatically).

Prediction 3 — An init container gates the app until its dependency exists

> **My prediction:** "If a pod has an init container that waits for service `mydb`, then the pod sits at `Init:0/1` until I create `mydb`, then flips to `Running` — *because* init containers must exit 0 before the main container starts."

```
spec:
  initContainers:
  - name: wait-db
    image: busybox:1.31
    command: ["sh","-c","until nslookup mydb; do echo waiting; sleep 2; done"]
  containers: [{ name: app, image: nginx }]
```

```
kubectl apply -f app.yaml
kubectl get pod app            # Init:0/1
kubectl logs app -c wait-db    # "waiting..."
kubectl expose deployment mydb --port=80    # nslookup succeeds → app starts
```

Verify: `Init:0/1` → `Running` only after `mydb` exists. The *init* container's logs (via `-c`) explain the block.

Prediction: "If I do X, then Y, because [mechanism]"	Ran?	Right?	Miss?
1.			

🎁 CAPSTONE — Compress It

One sentence, no notes:

> An app's process, config, secrets, replica count and update path all live in the Deployment's pod template — edit it and Kubernetes rolls a new ReplicaSet up as the old scales down (zero-downtime, reversible), while ConfigMaps/Secrets externalize settings and init/sidecar containers add setup and companions.

Explain it to a beginner in 3 sentences:

> 1. You never rebuild an image to change a setting or a password — you put those in ConfigMaps and Secrets and inject them into the pod.
> 2. To ship a new version you change the Deployment's template, and Kubernetes swaps pods a few at a time so users never see downtime — and keeps old versions so you can roll back instantly.
> 3. Extra containers can prepare the pod before the app starts (init) or run alongside it (sidecar), all sharing the pod's network and storage.

Which rung to revisit hands-on? Rung 3C — the ConfigMap/Secret injection shapes (`envFrom` vs `valueFrom` vs volume) and the "base64 ≠ encrypted" fact are the most-tested and most-confused.

🎯 CKA exam tips & quick notes

- **Rollout trio:** `kubectl set image deploy/x c=img:tag`, `kubectl rollout status|undo deploy/x`. Know cold.
- **`command` = ENTRYPOINT, `args` = CMD.** Container args after `--`; `--command` overrides the entrypoint.
- **Injection:** `envFrom` (whole), `env.valueFrom.*KeyRef` (one key), volume (files). `configMapRef` ↔ `secretRef`.
- **Secrets = base64, not encryption.** `echo -n x | base64` / `base64 -d`; `stringData:` skips manual encoding.
- **Immutable pod edit** → `kubectl replace --force -f /tmp/kubectl-edit-...yaml`; Deployment pods → `kubectl edit deployment`.
- **Init/sidecar debug:** `kubectl logs <pod> -c <container>`; `Init:x/y` = init phase.
- `kubectl create cm/secret --dry-run=client -o yaml > f.yaml` to template fast.

📌 Command cheat sheet

```
# ROLLOUTS
k set image deploy/web nginx=nginx:1.26
k rollout status|history|undo deploy/web
k rollout undo deploy/web --to-revision=2
k scale deploy web --replicas=5
# COMMANDS / ARGS
k run app --image=busybox --command -- sleep 1000     # command
k run web --image=webapp -- --color green             # args
# CONFIGMAP / SECRET
k create cm app-config --from-literal=K=V --from-file=app.props
k create secret generic db --from-literal=PASSWORD=p@ss
echo -n 'val' | base64      /      base64 -d
# INJECT (YAML):  envFrom:[{configMapRef|secretRef:{name:x}}]
```

Related sections

- Section 1 — Core Concepts — Deployments/ReplicaSets that rollouts manage.
- Section 7 — Storage — mounting ConfigMaps/Secrets as volumes.
- Section 6 — Security — encryption at rest, RBAC protecting secrets.
- Section 3 — Logging & Monitoring — `logs -c` for init/sidecar debugging.
- Section 13 — Troubleshooting — CrashLoopBackOff/ImagePullBackOff during rollouts.
- ../../Linux/26-tls-pki-openssl.md — base64/OpenSSL underpinning Secrets and encryption at rest.

✅ Answers — "Check yourself before Rung N"

Before Rung 2

Q: Name two things that force an image rebuild in the "before" world that Kubernetes lets you change *without* touching the image.

A: (1) **A setting (config value):** before, config was baked into the image, so a one-line change meant a rebuild-and-redeploy loop; Kubernetes externalizes it into a **ConfigMap** injected as env vars or files. (2) **A password (sensitive value):** before, secrets were baked into images or committed to Git — leaked forever; Kubernetes puts them in a **Secret** object injected at container start. In both cases you change the object and roll the pods — the image is never touched.

Before Rung 3

Q: If you `kubectl set image` a Deployment to a broken image, why do your *existing* users usually keep getting served during the failed rollout?

A: Because a rollout is "a **new ReplicaSet** scaled up as the old one scales down" — the broken pods live entirely in the *new* ReplicaSet. Those pods go `ImagePullBackOff` / `CrashLoopBackOff` and never become Ready, so the Deployment never proceeds to scale the *old* ReplicaSet down; the old RS's healthy pods keep serving traffic the whole time. The failure is contained to the new RS, the rollout stalls, and `kubectl rollout undo` reverses it — that containment is exactly why rolling updates are "safe."

Before Rung 4

Q: (1) Which pod field overrides the image's ENTRYPOINT? (2) Which injection method makes a ConfigMap show up as *files*? (3) Does base64-encoding a Secret make it secure — why/why not?

A: (1) `command:` overrides ENTRYPOINT (and `args:` overrides CMD — the classic trap is thinking `command` maps to Docker's CMD; it doesn't). (2) Mounting the ConfigMap as a **volume** (`volumes: - configMap: {name: ...}`) — each key becomes a file in the mount path; `envFrom` and `env.valueFrom` produce env vars, not files. (3) **No** — base64 is *encoding*, not encryption; anyone with etcd or API access can trivially `base64 -d` it back. Real safety comes from practice: don't commit secret YAML, enable **encryption at rest** on the API server, and restrict access with RBAC. Kubernetes helps a bit by sending a Secret only to nodes that need it and keeping it in tmpfs (RAM) on the kubelet.

Before Rung 5

Q: Which of these trigger a new rollout when changed on a Deployment: `image`, `env`, `replicas`? Two do something different from the third — which, and why?

A: **`image` and `env` trigger a rollout; `replicas` does not.** `image` and `env` are fields *inside the pod template*, and the rule is: change the template → the Deployment creates a new ReplicaSet and performs a rollout, recorded as a new revision. `replicas` lives on the Deployment spec *outside* the pod template — changing it is pure scaling: the *existing* ReplicaSet is resized up or down, no new RS, no new revision, nothing to roll back.

Before Rung 6

Q: In step 5 of the rollout trace, what specific pod condition stops the rollout from proceeding — and why is that the safety mechanism, not a bug?

A: The new pods never become **Ready** — with `nginx:doesnotexist` they sit in `ImagePullBackOff`, so the readiness condition that gates each batch is never met. The RollingUpdate engine only scales the old RS down (−1 per `maxUnavailable`) *after* the surged new pods pass readiness; since they never do, the rollout **stalls with the old ReplicaSet still at full strength and still serving**. That's the safety mechanism by design: readiness is the proof-of-health checkpoint between batches, so a bad version can never replace a good one — capacity stays ~100%, the blast radius is one stalled RS, and `rollout undo` cleanly reverses it.

Before Rung 7

Q: A colleague changed a ConfigMap value but the running pods still show the old value. Why, and what makes it take effect?

A: Because the pods consume the ConfigMap as **env vars**, and env vars are set by the kubelet once, at container start, *before* the process runs — they are never updated on a live container. Changing the ConfigMap object afterward changes nothing inside already-running pods. To make it take effect you must **roll the pods** so new containers start and re-read the ConfigMap — e.g. `kubectl rollout restart deploy/web` or any pod-template change that triggers a rollout. (The exception: a ConfigMap mounted as a **volume** does eventually update live in the files — only env-injected config is frozen at start.)

Cluster Maintenance, Climbed the Ladder 🪜

Section 5 of the CKA — deriving how to change a live cluster without breaking it

> Keeping a running cluster healthy: draining nodes, the version-skew rule, upgrading with kubeadm, and **etcd backup & restore**. The **upgrade** and **etcd restore** tasks are near-guaranteed on the exam. We climb from **the pain of touching a live cluster → the "control the blast radius, keep an undo" idea → the machinery** → then the commands as predictions. Each rung ends with a ✅ **Check yourself**.

RUNG 0 — The Setup 🎯

What am I learning? How to safely take a node out for maintenance, upgrade the cluster one version at a time, and back up / restore the one thing that *is* the cluster: etcd.

Why did it land on my desk? *Cluster Architecture, Installation & Configuration* is 25% of the CKA, and a **cluster upgrade** and an **etcd restore** are among the most reliably-appearing tasks. These are timed, procedural, and unforgiving — reflex matters.

What do I already know? Maybe `kubectl drain`. What's fuzzy: why `kubectl get nodes` shows the *old* version right after an upgrade, and the exact etcd-restore dance (which file to edit, which flags are mandatory).

RUNG 1 — The Pain 🔥

Why does cluster maintenance exist at all?

A cluster is a living system you must change *while it's serving traffic*. Every change is a chance to cause an outage:

```
THE MAINTENANCE PAIN
  patch a node's kernel  ─▶ just reboot it? → every pod on it dies without warning
  upgrade Kubernetes     ─▶ jump 1.27→1.30? → version skew → components refuse to talk
  etcd disk dies         ─▶ no backup?       → the ENTIRE cluster state is gone, no undo
  bad change             ─▶ "roll back?"      → to what? there's no snapshot
```

Before / without these tools: you rebooted nodes and hoped controllers noticed; you upgraded blindly and hit skew failures; and if etcd was lost, the cluster was *gone* — every object, every secret, unrecoverable.

What breaks without it: availability during maintenance (pods yanked out from under users), upgrade safety (skew breakage), and — the big one — **disaster recovery** (etcd is the single source of truth; lose it, lose everything).

Who feels it most? The platform/ops team — you own uptime *through* change, and you're the one restoring etcd at 3 AM.

> ✅ **Check yourself before Rung 2:** If a worker node's etcd... wait — where does etcd actually live, and why is losing *it* categorically worse than losing a worker node? (Hint: what is stored where?)

RUNG 2 — The One Idea 💡

The single sentence everything hangs off

> **Maintenance is about controlling *where disruption lands* and keeping an *undo button*: `drain` moves pods off a node before you touch it, the version-skew rule forces a safe upgrade order (control plane first, one minor at a time), and an etcd snapshot is the only true restore point for all cluster state.**

Derivations:

- *"controlling where disruption lands"* → **cordon/drain** evacuate a node so maintenance hits no live pods; controllers reschedule them elsewhere.
- *"safe upgrade order… control plane first"* → because **nothing may be newer than the API server**, you *must* upgrade it before the kubelets — the skew rule dictates the sequence.
- *"one minor at a time"* → Kubernetes only supports N-2 minors and tested one-step upgrades; skipping breaks things.
- *"an etcd snapshot is the only true restore point"* → your YAML in Git rebuilds *objects*, but only an **etcd snapshot** captures the exact live state (including things created imperatively).

> ✅ **Check yourself before Rung 3:** Why do you upgrade the **control plane before** the worker kubelets, never the reverse? State the rule it follows.

RUNG 3 — The Machinery ⚙️

How it ACTUALLY works — go slow

🧸 Plain-English first (read this before the technical version)

Think of the cluster as a hotel that must stay open while you renovate it. This section covers four things: emptying one wing safely, the rule for renovating in the right order, the actual renovation procedure, and the fireproof safe holding the hotel's only master ledger.

(A) Emptying a wing (cordon, drain, uncordon). If a wing (a "node" — one computer) suddenly goes dark, head office waits a few minutes before declaring its guests lost; guests who belong to a group booking get rebooked into other wings automatically, but walk-ins with no booking record are simply gone. For *planned* work you don't wait for a surprise: you can put up a "no new check-ins" sign (**cordon**), or politely move everyone out AND put up the sign (**drain**), then take the sign down when the work is done (**uncordon**). Wrinkles: some staff are stationed one-per-wing by design and can't move (you must tell the drain to skip them); walk-in guests need a forceful eviction; and taking the sign down does NOT bring the old guests back — the wing just becomes available for new arrivals.

(B) The renovation-order rule (version skew). Hotel systems come in numbered editions, and the strict rule is: **no department may run a newer edition than head office's front desk** (the API server). Assistants can be one edition behind, wing staff up to two behind — but never ahead. That's why you always modernize head office FIRST, then the wings, and always **one edition at a time** (no jumping from edition 27 to 30). While head office is being switched over, the phones are briefly down — but guests already in rooms are completely unaffected.

(C) The renovation procedure (the kubeadm upgrade flow). A foreman tool called kubeadm swaps out head office's own machinery, but it deliberately **never touches each wing's caretaker (the kubelet)** — you must update every caretaker yourself, by hand, wing by wing: empty the wing, install the caretaker's new edition, restart them, reopen the wing. This explains a famous surprise: right after upgrading head office, the status board still shows the OLD edition next to each wing — because that board reports the *caretakers'* editions, which only change after you restart each one.

(D) The master ledger (etcd backup and restore). One book records everything the hotel is — every booking, key, and rule. Lose it with no copy, and the hotel effectively ceases to exist even while guests still sit in their rooms. So: periodically **photocopy the ledger** (take a snapshot — this works while it's in use, though you must present four security credentials to do it). To recover from disaster, you rebuild a fresh ledger **in a new cabinet** from the photocopy, then change one line in the record-keeper's standing instructions telling it which cabinet to read from. The record-keeper's minder notices the changed instructions and restarts it automatically. Everything as of photocopy time comes back; anything written after the photocopy is gone. Two copier models exist — one for ledgers in use, one for ledgers taken offline — and note that your blueprints-in-a-filing-cabinet (YAML files in Git) are NOT a substitute: they miss everything that was ever arranged verbally.

Now the original technical deep-dive — the same ideas, in precise form:

(A) Node maintenance — cordon, drain, uncordon

When a node goes `NotReady`, the control plane waits the **pod eviction timeout** (~5 min, on the controller-manager; modern K8s applies a `NoExecute` taint and evicts after a grace period). Then: **ReplicaSet-backed pods are recreated elsewhere; bare pods are gone forever.** To do maintenance *deliberately* rather than by surprise:

Command	Effect
`kubectl cordon <node>`	mark **unschedulable**; existing pods stay
`kubectl drain <node>`	evict all pods gracefully **+ cordon**
`kubectl uncordon <node>`	make schedulable again

```
kubectl drain node01 --ignore-daemonsets                    # DS pods can't move
kubectl drain node01 --ignore-daemonsets --delete-emptydir-data   # if pods use emptyDir
kubectl drain node01 --ignore-daemonsets --force            # also evict bare pods
# ...maintenance / reboot...
kubectl uncordon node01
```

> 🎯 `drain` **fails** on DaemonSet pods (need `--ignore-daemonsets`), bare pods (`--force`), or `emptyDir` (`--delete-emptydir-data`). Drained pods **do not** come back on `uncordon` — the node just becomes eligible for *new* pods.

(B) The version-skew rule — why order matters

Version = `major.minor.patch`. Kubernetes supports the **3 latest minors**; **upgrade one minor at a time** (1.27→1.28→1.29). Nothing may be **newer than the API server**:

```
                kube-apiserver        (X)     ← reference, highest
        ┌──────────────┬──────────────┐
 controller-manager  scheduler        kubectl
      (X-1)            (X-1)       (X-1 … X+1  ← kubectl may be +1)
        │
   kubelet / kube-proxy   (X-2)      ← up to two minors behind
```

This is *why* the control plane upgrades first, then kubelets. During a control-plane upgrade, `kubectl`/API is briefly down but **running workloads keep serving** (kubelets and pods don't stop).

(C) The kubeadm upgrade flow

kubeadm upgrades the control-plane **static pods**, but **never the kubelet — you do that manually** on every node.

```
per node: bump the pkgs.k8s.io repo to the target minor  →  apt-cache madison kubeadm (find patch)
CONTROL PLANE: apt install kubeadm=<v> → kubeadm upgrade plan → kubeadm upgrade apply <v>
               → drain CP → apt install kubelet kubectl=<v> → systemctl restart kubelet → uncordon
EACH WORKER:   apt install kubeadm=<v> → kubeadm upgrade node   (NOT "apply")
               → drain → apt install kubelet kubectl=<v> → restart kubelet → uncordon
```

> 🎯 After `kubeadm upgrade apply`, `kubectl get nodes` **still shows the old version** — that column is the **kubelet** version, which changes only after you `restart kubelet` (step 4). Extra control-plane nodes use `kubeadm upgrade **node**`, not `apply`.

(D) etcd backup & restore — the only real undo

etcd holds **all** cluster state. Back it up (snapshot of a *live* etcd), and restore into a *new* data dir.

```
# BACKUP — talks to the live etcd (all 4 flags mandatory):
ETCDCTL_API=3 etcdctl snapshot save /opt/snapshot.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
etcdctl snapshot status /opt/snapshot.db --write-out=table   # inspect

# RESTORE — offline, into a NEW dir, then repoint the manifest:
ETCDCTL_API=3 etcdctl snapshot restore /opt/snapshot.db --data-dir=/var/lib/etcd-from-backup
#   (modern: etcdutl snapshot restore /opt/snapshot.db --data-dir=/var/lib/etcd-from-backup)
sudo vi /etc/kubernetes/manifests/etcd.yaml   # set the etcd-data volume's hostPath → /var/lib/etcd-from-backup
kubectl get pods -A                           # after etcd + apiserver static pods restart
```

🎯 In restore you **only edit the etcd manifest's** `volumes[].hostPath.path` (the `etcd-data` volume). The kubelet sees the change and recreates the etcd (and dependent apiserver) static pods. Restore **initializes a brand-new cluster** (new member IDs) on purpose. Get the cert paths/endpoint from `cat /etc/kubernetes/manifests/etcd.yaml`. **`etcdctl`** = live/network (save/status); **`etcdutl`** = offline files (the modern restore) — either works for the exam.

✅ **Check yourself before Rung 4:** In an etcd restore, which single field in `etcd.yaml` do you change, and why does editing that file cause etcd to restart on its own?

RUNG 4 — The Vocabulary Map 🏷️

Term	What it actually is	Machinery
cordon	Mark node unschedulable	Node maintenance
drain	Evict pods + cordon	Node maintenance
uncordon	Re-enable scheduling	Node maintenance
pod eviction timeout	Wait before declaring a NotReady node's pods dead	Controller-manager
version skew	Allowed version gaps between components	Upgrade order
minor / patch	`1.29` (feature) / `.3` (bugfix)	Versioning
kubeadm upgrade plan/apply/node	Preview / do (control plane) / do (worker)	Upgrade
static pod	Control-plane pod kubeadm upgrades	Upgrade target
snapshot save/status/restore	Backup / inspect / recover etcd	etcd
data-dir	Where etcd stores its DB	etcd restore
hostPath	The volume path you repoint on restore	etcd manifest
etcdctl / etcdutl	Live-cluster CLI / offline-file CLI	etcd

The unlock: three verbs of maintenance — **evacuate** (drain), **advance** (upgrade, skew-ordered), **preserve/recover** (etcd snapshot/restore). All three are about *change with a safety net*.

> ✅ **Check yourself before Rung 5:** Which command previews an upgrade without doing it, and which one do you run on a *worker* (hint: not `apply`)?

RUNG 5 — The Trace 🎬

Trace — upgrade the control plane 1.28 → 1.29:

1. Bump the `pkgs.k8s.io` repo to `v1.29`, `apt-get update`, `apt-cache madison kubeadm` → pick `1.29.3-1.1`.
2. `apt install kubeadm=1.29.3-1.1`; `kubeadm version` confirms.
3. `kubeadm upgrade plan` shows the path; `kubeadm upgrade apply v1.29.3` upgrades the apiserver/scheduler/controller-manager/etcd **static pods** (kubeadm rewrites their manifests; the kubelet restarts them). API blips; **workloads keep serving**.
4. `kubectl get nodes` still shows **1.28** — because that's the *kubelet* version, unchanged.
5. `kubectl drain controlplane --ignore-daemonsets` (it runs CoreDNS etc.).
6. `apt install kubelet kubectl=1.29.3-1.1`; `systemctl daemon-reload && systemctl restart kubelet`. **Now** `get nodes` flips to 1.29.
7. `kubectl uncordon controlplane`. Repeat the worker flow with `kubeadm upgrade node` per node.

Trace — etcd disaster recovery: snapshot save (live) → something wipes state → `snapshot restore --data-dir=<new>` (offline) → edit `etcd.yaml` hostPath → kubelet recreates etcd + apiserver → the objects that existed *at snapshot time* reappear (anything created after the snapshot is gone).

> ✅ **Check yourself before Rung 6:** At which exact step does `kubectl get nodes` finally show the new version — and why not earlier?

RUNG 6 — The Contrast ⚖️

This…	vs that…	Difference
cordon	**drain**	block *new* pods only vs *also evict* existing ones
drain	**delete node**	temporary, reversible with uncordon vs removes it from the cluster
etcd snapshot	**config backup (Velero / `get -o yaml`)**	exact full state incl. imperative objects vs declarative objects (+ PVs with Velero)
`kubeadm upgrade apply`	**`kubeadm upgrade node`**	control-plane primary vs workers / extra control-plane nodes
etcdctl	**etcdutl**	live etcd over network (save/status) vs offline files (restore/backup)

When NOT to: don't `drain` without `--ignore-daemonsets` (it just errors); don't skip minors on upgrade; don't restore etcd onto the *same* data dir the live etcd is using (repoint to a new dir); don't forget the four etcd TLS flags (the #1 mistake).

One-sentence "why this over that":

> Drain to evacuate a node before maintenance, upgrade the control plane first and one minor at a time to respect skew, and rely on an etcd snapshot — not just your YAML — when you need to truly rewind cluster state.

> ✅ **Check yourself before Rung 7:** Your teammate says "we keep all manifests in Git, so we don't need etcd backups." Give one thing a Git of manifests would *not* recover that an etcd snapshot would.

RUNG 7 — The Prediction Test 🧪

Prediction 1 — Drain keeps controller-backed apps up

> **My prediction:** "If I drain node01, then its ReplicaSet-backed pods reappear on other nodes and the app stays up, while node01 shows `SchedulingDisabled` — *because* drain evicts + cordons, and controllers reconcile the evicted pods elsewhere."

```
kubectl get pods -o wide | grep node01
kubectl drain node01 --ignore-daemonsets --delete-emptydir-data
kubectl get nodes                    # node01 = SchedulingDisabled
# ...patch/reboot...
kubectl uncordon node01
```

Verify: RS apps stay served; node01 accepts new pods only after uncordon. A **bare** pod blocks the drain until `--force` (and won't come back — it had no controller).

Prediction 2 — `get nodes` shows the old version until the kubelet restarts

> **My prediction:** "If I run `kubeadm upgrade apply v1.29.0` but haven't touched the kubelet, then `kubectl get nodes` still says 1.28 — *because* that column reports the kubelet version, and I upgrade+restart the kubelet in a later step."

```
sudo apt-get install -y kubeadm=1.29.0-1.1
sudo kubeadm upgrade apply v1.29.0
kubectl get nodes                    # STILL 1.28
sudo apt-get install -y kubelet=1.29.0-1.1 && sudo systemctl daemon-reload && sudo systemctl restart
kubelet
kubectl get nodes                    # NOW 1.29.0
```

Verify: the version flips only after the kubelet restart. Surprised? That's the point — the node column ≠ control-plane component versions.

Prediction 3 — etcd restore rewinds state to the snapshot

> **My prediction:** "If I snapshot etcd, delete some objects, then restore that snapshot and repoint `etcd.yaml`, then the deleted objects reappear — *because* restore rebuilds etcd from the snapshot's point in time."

```
ETCDCTL_API=3 etcdctl snapshot save /opt/snap.db \
  --endpoints=https://127.0.0.1:2379 --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/kubernetes/pki/etcd/server.key
# ...delete a deployment to simulate loss...
ETCDCTL_API=3 etcdctl snapshot restore /opt/snap.db --data-dir=/var/lib/etcd-from-backup
sudo vi /etc/kubernetes/manifests/etcd.yaml    # etcd-data hostPath → /var/lib/etcd-from-backup
watch kubectl get pods -A
```

Verify: snapshot-time objects return after etcd+apiserver restart. If `kubectl` hangs, the apiserver is still coming up behind etcd — wait a minute. Post-snapshot changes are (correctly) gone.

Prediction: "If I do X, then Y, because [mechanism]"	Ran?	Right?	Miss?
1.			

🎁 CAPSTONE — Compress It

One sentence, no notes:

> Maintain a live cluster by draining nodes before you touch them, upgrading the control plane first and one minor at a time (skew rule), and keeping etcd snapshots as your only true restore point for cluster state.

Explain it to a beginner in 3 sentences:

> 1. Before patching a node you `drain` it, which safely moves its pods elsewhere so users aren't affected.
> 2. Upgrades go control-plane-first, one version step at a time, because Kubernetes forbids any component being newer than the API server.
> 3. etcd is the cluster's memory, so you take snapshots of it — restoring one is the only way to truly rewind the whole cluster after a disaster.

Which rung to revisit hands-on? Rung 3D + Prediction 3 — the **etcd restore**. It's the single highest-value skill here and the easiest to fumble (wrong flags, wrong field). Do it end-to-end until it's boring.

🎯 CKA exam tips & quick notes

- **`kubectl drain <node> --ignore-daemonsets`** (+ `--delete-emptydir-data` / `--force` if it complains), then **`uncordon`**. Pods don't auto-return.
- **Upgrade order:** control plane first, one minor at a time — repo bump → `kubeadm=<v>` → `upgrade plan` → `upgrade apply <v>` (CP) / `upgrade node` (workers) → drain → `kubelet kubectl=<v>` → `daemon-reload && restart kubelet` → uncordon.

- `kubectl get nodes` version = **kubelet** → doesn't change until you restart kubelet.
- **etcd:** every command needs `--endpoints --cacert --cert --key` (find them in `etcd.yaml`). save → status → restore `--data-dir=<new>` → repoint the manifest `hostPath`.
- **Skew:** nothing newer than apiserver; kubelet/kube-proxy X-2; kubectl X+1.
- **Drill the etcd restore** end-to-end — highest-value single skill in this section.

📌 Command cheat sheet

```
# NODE MAINTENANCE
k drain node01 --ignore-daemonsets --delete-emptydir-data
k cordon node01 ; k uncordon node01
# UPGRADE (kubeadm)
sudo kubeadm upgrade plan
sudo kubeadm upgrade apply v1.29.3       # control plane
sudo kubeadm upgrade node                # workers / extra control-plane
sudo apt-get install -y kubelet=1.29.3-1.1 kubectl=1.29.3-1.1
sudo systemctl daemon-reload && sudo systemctl restart kubelet
# ETCD BACKUP / RESTORE
ETCDCTL_API=3 etcdctl snapshot save /opt/snap.db \
  --endpoints=https://127.0.0.1:2379 --cacert=... --cert=... --key=...
etcdctl snapshot status /opt/snap.db --write-out=table
etcdctl snapshot restore /opt/snap.db --data-dir=/var/lib/etcd-from-backup
# then edit /etc/kubernetes/manifests/etcd.yaml hostPath → new dir
```

Related sections

- Section 9 — Design & Install a Kubernetes Cluster — where etcd/control-plane layout comes from.
- Section 10 — Install Kubernetes the Kubeadm Way — the tool you upgrade with.
- Section 6 — Security — the PKI certs etcd/kubeadm use.
- Section 1 — Core Concepts — etcd's role and static pods.
- Section 13 — Troubleshooting — recovering when the control plane won't start.
- ../../Linux/16-systemd-services.md — the systemd/kubelet service you restart during upgrades.

✅ Answers — "Check yourself before Rung N"

Before Rung 2

Q: Where does etcd actually live, and why is losing *it* categorically worse than losing a worker node?

A: etcd lives on the **control plane** (in kubeadm clusters, as a static pod on the control-plane node, its database in `/var/lib/etcd`) — it does not live on worker nodes at all. A worker node holds only *running copies* of pods; lose one and the control plane notices, waits out the eviction timeout, and controllers recreate the ReplicaSet-backed pods on other nodes — nothing definitional is lost. etcd, by contrast, holds **all cluster state** — every object, deployment, secret, role — it *is* the cluster's single source of truth. Lose etcd with no backup and the entire cluster state is gone with no undo: workers can keep running what they have, but the cluster's memory and ability to reconcile anything is unrecoverable. That's why an etcd snapshot is the only true restore point.

Before Rung 3

Q: Why do you upgrade the control plane before the worker kubelets, never the reverse? State the rule it follows.

A: The rule is the **version-skew rule: nothing may be newer than the kube-apiserver** — the API server (X) is the reference and highest version; controller-manager/scheduler may be X-1, and kubelet/kube-proxy may be up to X-2 behind (only kubectl may be X+1). If you upgraded the worker kubelets first, they would be *newer* than the API server, violating the rule — components could refuse to talk. Upgrading the control plane first keeps every kubelet at or below the API server's version at every step, and the "one minor at a time" companion rule (1.27→1.28→1.29) keeps each step within the supported, tested skew window.

Before Rung 4

Q: In an etcd restore, which single field in `etcd.yaml` do you change, and why does editing that file cause etcd to restart on its own?

A: You change only the **`etcd-data` volume's `volumes[].hostPath.path`** in `/etc/kubernetes/manifests/etcd.yaml`, repointing it from the old data dir to the new one you restored into (e.g. `/var/lib/etcd-from-backup`). It restarts on its own because etcd is a **static pod**: the kubelet continuously watches the `/etc/kubernetes/manifests/` directory, and when a manifest file changes it automatically recreates that pod — no kubectl, no API server needed. The kubelet recreates etcd (and the dependent kube-apiserver static pod follows), and the restored snapshot's state comes up as a brand-new etcd cluster (new member IDs, by design).

Before Rung 5

Q: Which command previews an upgrade without doing it, and which one do you run on a *worker* (not `apply`)?

A: **`kubeadm upgrade plan`** previews the upgrade — it shows the available target versions and the path without changing anything. On a worker (and on any *extra* control-plane node) you run **`kubeadm upgrade node`** — `kubeadm upgrade apply <version>` is only for the primary control-plane node. And on every node, kubeadm never touches the kubelet: you upgrade it yourself (`apt install kubelet=<v>`, then `systemctl daemon-reload && systemctl restart kubelet`).

Before Rung 6

Q: At which exact step does `kubectl get nodes` finally show the new version — and why not earlier?

A: At **Step 6** — after you install the new kubelet package (`apt install kubelet kubectl=1.29.3-1.1`) and run `systemctl daemon-reload && systemctl restart kubelet`. Not earlier because the VERSION column in `kubectl get nodes` reports the **kubelet's** version, not the control plane's: `kubeadm upgrade apply` in Step 3 only rewrote the static-pod manifests for the apiserver/scheduler/controller-manager/etcd, leaving the kubelet — a systemd service kubeadm never upgrades — still at 1.28. Only when the new kubelet binary restarts and re-registers does the column flip to 1.29.

Before Rung 7

Q: "We keep all manifests in Git, so we don't need etcd backups." Give one thing a Git of manifests would *not* recover that an etcd snapshot would.

A: Anything created **imperatively** and never written to a file — e.g. a Secret made with `kubectl create secret generic ... --from-literal=...`, a `kubectl run` pod, a live `kubectl edit` / `scale` / `set image` change, or an `expose`d Service. Git only rebuilds the *declarative objects you remembered to*

commit; an etcd snapshot captures the **exact live state of everything** — every object as it actually existed at snapshot time, including all the imperative drift. That's why the ladder calls the etcd snapshot "the only true restore point for all cluster state."

Security, Climbed the Ladder 🪜

Section 6 of the CKA — deriving how the cluster proves identity and limits power

> The largest CKA section — TLS/PKI, the CSR API, kubeconfig, RBAC, service accounts, image secrets, security contexts, and network policies. It looks like ten unrelated topics; it's really **one layered idea**. We climb from **the pain of an open cluster** → **the gatekeeping idea** → **the machinery** → then the commands as predictions. Each rung ends with a ✅ **Check yourself**.

RUNG 0 — The Setup 🎯

What am I learning? How Kubernetes answers two questions on *every* action — *who are you?* and *what may you do?* — plus how it confines what a running workload can do and reach.

Why did it land on my desk? Security spans two big CKA domains (Cluster Architecture 25% for RBAC/certs; Services & Networking 20% for NetworkPolicy). **Cert troubleshooting and RBAC tasks are exam staples**, and a broken cert can take the whole control plane down.

What do I already know? Maybe `kubectl config use-context`. What's fuzzy: why there are *two* CAs, the difference between "unauthorized" and "forbidden," and why a NetworkPolicy sometimes does nothing.

RUNG 1 — The Pain 🔥

Why does cluster security exist at all?

The API server is the keys to the kingdom — it can create, read, and delete everything, including Secrets. Without security layers:

```
THE OPEN-CLUSTER PAIN
  anyone who reaches :6443 ─▶ full control of every workload + every Secret
  a compromised pod        ─▶ runs as root with all Linux capabilities → owns the node
  any pod                  ─▶ can reach any other pod (flat network) → lateral movement
  a new teammate           ─▶ "just share the admin kubeconfig" → nobody has least privilege
  a private image          ─▶ can't be pulled at all (no registry creds)
```

Before / without it: no cryptographic identity (anyone claiming to be admin *is* admin), no least privilege (every credential is all-powerful), and a flat network where one popped pod can scan the whole cluster.

What breaks without it: confidentiality (Secrets readable by all), integrity (anyone mutates anything), and blast-radius control (one compromise = total compromise).

Who feels it most? The platform/security team — you must let many humans and apps use one cluster *without* handing each the master key.

> ✅ **Check yourself before Rung 2:** Name the two distinct questions any secure system must answer for a request, and give a Kubernetes example of each. (Hint: identity, then permission.)

RUNG 2 — The One Idea 💡

The single sentence everything hangs off

> **Kubernetes security is layered gatekeeping: the API server first authenticates *who you are* (TLS client certs for humans/components, tokens for service accounts) then authorizes *what you may do* (RBAC) — and separately, each workload is confined by the identity it runs as (security context) and which pods it may reach (network policy).**

Derivations:

- *"authenticates via TLS certs / tokens"* → all the PKI, the `/etc/kubernetes/pki` certs, the **CSR API** (how a new user gets a cert), and **kubeconfig** (where your cert lives) are just *authentication plumbing*.
- *"authorizes via RBAC"* → **Role/RoleBinding** (namespaced) and **ClusterRole/ClusterRoleBinding** (cluster-wide) are *authorization*. A `403 Forbidden` is an authz failure; a `401 Unauthorized` is an authn failure.
- *"tokens for service accounts"* → pods get a **ServiceAccount** identity, then RBAC decides what that SA may do — same two-step, just for apps.
- *"confined by identity it runs as"* → **security context** (runAsUser, dropped Linux capabilities).
- *"which pods it may reach"* → **NetworkPolicy** (a pod-level firewall).

Every topic in this section is one of those four boxes: **who / what-may-you-do / what-power / who-can-it-talk-to.**

> ✅ **Check yourself before Rung 3:** A user's `kubectl get pods` returns `Forbidden`. Which of the two gates did they pass, and which did they fail? What would `Unauthorized` have meant instead?

RUNG 3 — The Machinery ⚙️

How it ACTUALLY works — the most important rung. Go slow.

🧸 Plain-English first (read this before the technical version)

Think of the cluster as a high-security office building. This section explains, piece by piece, how the building checks who you are, what you're allowed to do, and what each worker inside can touch.

(A) Proving identity with certificates. Everyone in the building carries a tamper-proof ID badge. A "certificate" is that badge: it names you, and it carries the signature of a trusted badge office (the "CA," or Certificate Authority — the office everyone agrees to trust). Each badge comes in two parts: a public half anyone can look at, and a private half (a secret key) you must never share — like the raised seal only the real owner can produce. The building actually has **two separate badge offices**: one for the whole building, and a private one used only by the records vault (etcd — the cluster's filing cabinet where everything is stored). Mixing up which office stamped which badge is a classic way the front desk stops recognizing people.

(B) Getting a badge. A newcomer fills out a badge application (a "CSR" — Certificate Signing Request), a manager approves it, and the badge office stamps it. Your wallet (the "kubeconfig" file) then holds your badge plus the building's address, so you can walk up and be recognized.

(C) Permissions. A valid badge only proves *who* you are. A separate permissions list ("RBAC" — role-based access control) says *what* you may do: "this person may read the mailroom, but not open the safe." Some permission lists apply to one floor only (a Role, for one namespace — a walled-off department); others apply building-wide (a ClusterRole). There's even a way to test "could this person do X?" without them actually trying.

(D) Badges for machines. Software running inside the building gets its own kind of badge too — a "service account" with a short-lived pass slipped into its room — so apps are checked the same way people are.

(E) Limiting workers once inside. Three final controls: a job description limiting what powers a program runs with (the "security context" — e.g., never run as the all-powerful root user); a stored courier password for fetching packages from private warehouses ("image pull secrets" for private software downloads); and internal door locks ("NetworkPolicy") deciding which rooms may talk to which — by default every room can call every other room, until you lock one down and list its allowed visitors. One catch: the locks only work if the building's wiring contractor (the network plugin) actually installs them — some contractors silently ignore the order.

Now the original technical deep-dive — the same ideas, in precise form:

(A) The TLS/PKI foundation — how identity is proven

- **Asymmetric crypto:** a key **pair** — a **public key** (share freely) and a **private key** (kept secret). A **certificate** = public key + identity (CN/SANs) **signed by a CA**. You trust a cert if a **CA you trust** signed it. A **CSR** asks a CA to sign your public key + identity. **PKI** = the whole system.
- Naming: `.crt` / `.pem` = public cert; `.key` = **private key** (keep secret).

- **Three cert roles: server** certs (apiserver/etcd/kubelet prove *they* are legit), **client** certs (admin/scheduler/controller-manager/kubelet authenticate *to* the apiserver), **CA** cert (everyone verifies the others with `ca.crt`).

Where the certs live (kubeadm) — note the TWO CAs:

```
/etc/kubernetes/pki/
├── ca.crt / ca.key                    # CLUSTER CA (Issuer: "kubernetes")
├── apiserver.crt / .key               # apiserver SERVING cert
├── apiserver-etcd-client.crt/.key     # apiserver → etcd (client)
├── apiserver-kubelet-client.crt/.key  # apiserver → kubelet (client)
└── etcd/
    ├── ca.crt / ca.key                # SEPARATE etcd CA (Issuer: "etcd-ca")
    └── server.crt / server.key        # etcd serving cert
```

```
openssl x509 -in /etc/kubernetes/pki/apiserver.crt -text -noout
# read:  Subject: CN=kube-apiserver   Issuer: CN=kubernetes
#        X509v3 Subject Alternative Name: DNS:kubernetes, ...svc.cluster.local, IP:10.96.0.1
#        Validity: Not Before / Not After   ← expiry
```

> 🎯 When `kubectl` is dead, the control-plane containers still exist: **`crictl ps -a | grep apiserver`** + **`crictl logs <id>`**. The classic break is a wrong cert **path** or the **wrong CA** in `etcd.yaml` / `kube-apiserver.yaml` → "certificate signed by unknown authority" (you mixed the cluster CA with etcd's own CA).

(B) Authentication — getting and using a client cert

The CSR API (how a new human gets access): they make a key + CSR; the admin creates a **CertificateSigningRequest**, approves it; the **controller-manager** signs it (with `--cluster-signing-cert-file/-key-file`).

```
openssl genrsa -out akshay.key 2048
openssl req -new -key akshay.key -out akshay.csr -subj "/CN=akshay"   # CN = username, O = group
```

```
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata: { name: akshay }
spec:
  signerName: kubernetes.io/kube-apiserver-client
  usages: ["client auth"]
  request: <cat akshay.csr | base64 -w0>        # SINGLE line (no wraps!)
```

```
kubectl apply -f csr.yaml
kubectl certificate approve akshay             # or: certificate deny <name>
kubectl get csr akshay -o jsonpath='{.status.certificate}' | base64 -d > akshay.crt
```

> 🎯 `base64 -w0` prevents wrapping ("illegal base64 data" otherwise). **`CN` = username, `O` = group** — a CSR requesting `O=system:masters` = full cluster-admin, so **`deny`** suspicious ones.

kubeconfig (`~/.kube/config`) bundles **clusters** (endpoint + CA), **users** (client cert/token), **contexts** (user × cluster × namespace):

```
kubectl config get-contexts
kubectl config use-context prod-admin@prod
kubectl config set-context --current --namespace=dev
```

Certs go as file paths *or* inline base64 (`client-certificate-data`). A wrong path = "unable to read client cert."

(C) Authorization — RBAC

Modes on the apiserver (evaluated in order; deny → next, allow → stop): `--authorization-mode=Node,RBAC,Webhook` . **Node** authorizes kubelets; **RBAC** is what you configure; ABAC/Webhook/AlwaysAllow-Deny exist.

Role + RoleBinding = namespaced. A Role is `(apiGroups, resources, verbs)` rules; a RoleBinding grants it to a subject *in one namespace*.

```
kind: Role                            # apiVersion: rbac.authorization.k8s.io/v1
metadata: { name: developer, namespace: default }
rules:
- apiGroups: [""]                     # "" = core (pods, services, configmaps)
  resources: ["pods"]
  verbs: ["get","list","create","delete"]
  # resourceNames: ["blue"]           # optional: restrict to NAMED objects
---
kind: RoleBinding
metadata: { name: dev-binding, namespace: default }
subjects: [{ kind: User, name: dev-user, apiGroup: rbac.authorization.k8s.io }]
roleRef: { kind: Role, name: developer, apiGroup: rbac.authorization.k8s.io }
```

ClusterRole + ClusterRoleBinding = cluster-wide (for cluster-scoped resources — nodes, PVs, namespaces, CSRs — or a namespaced resource across *all* namespaces).

```
kubectl create role developer --verb=get,list,create,delete --resource=pods
kubectl create rolebinding dev-binding --role=developer --user=dev-user
kubectl create clusterrole node-reader --verb=get,list,watch --resource=nodes
kubectl create clusterrolebinding michelle-binding --clusterrole=node-reader --user=michelle
kubectl auth can-i create deployments --as dev-user -n blue     # TEST without logging in
kubectl api-resources --namespaced=false                        # what's cluster-scoped
```

> 🎯 Roles/bindings are **namespaced** — always `-n <ns>` . "Forbidden" ⇒ check the Role's `apiGroups` / `resources` / `verbs` / `resourceNames` and the binding's `subjects` . `apiGroups: [""]` =core, `["apps"]` =deployments/replicasets.

(D) Service Accounts — identity for pods

Users are for humans; **ServiceAccounts** for apps. Every namespace has a `default` SA; its **bound token** (short-lived, projected) mounts at `/var/run/secrets/kubernetes.io/serviceaccount/token` .

```
kubectl create serviceaccount dashboard-sa
kubectl create token dashboard-sa --duration=48h      # short-lived token
```

```
spec:
  serviceAccountName: dashboard-sa
  automountServiceAccountToken: false     # opt out
```

Grant permissions via RBAC with `subjects: [{kind: ServiceAccount, name: dashboard-sa, namespace: default}]`.

(E) Workload confinement — security context, image secrets, network policy

Security context — the UID and Linux **capabilities** a container runs with. Pod-level applies to all; container-level overrides; **capabilities are container-only.**

```
spec:
  securityContext: { runAsUser: 1001, fsGroup: 2000 }   # POD level
  containers:
  - name: app
    securityContext:                                     # CONTAINER (wins)
      runAsUser: 1002
      runAsNonRoot: true
      readOnlyRootFilesystem: true
      capabilities: { add: ["NET_ADMIN"], drop: ["ALL"] }
```

Image pull secrets — private registries:

```
kubectl create secret docker-registry regcred \
  --docker-server=my-registry.io:5000 --docker-username=u --docker-password='p'
# then: spec.imagePullSecrets: [{ name: regcred }]
```

NetworkPolicy — **default is every pod can reach every pod.** Selecting a pod flips it to **default-deny** for the chosen direction; you add explicit allows. **Replies are auto-allowed** (stateful) — only reason about the direction traffic *originates*. **Needs an enforcing CNI** (Calico/Cilium/Weave) — **plain Flannel ignores it silently.**

```
kind: NetworkPolicy                    # networking.k8s.io/v1
metadata: { name: db-policy }
spec:
  podSelector: { matchLabels: { role: db } }    # protect db
  policyTypes: [Ingress]
  ingress:
  - from:
    - podSelector: { matchLabels: { name: api } }   # allow only api
    ports: [{ protocol: TCP, port: 3306 }]
```

AND vs OR gotcha: `podSelector` + `namespaceSelector` in the **same** `from` **item** = AND; split into **two** `-` **items** = OR. `ipBlock` allows external CIDRs.

> 🎯 An **Egress** policy that selects a pod **blocks its DNS too** — allow egress to CoreDNS (UDP/TCP **53**) or name resolution breaks.

> ✅ **Check yourself before Rung 4:** Four quick ones: (1) which CA signs a *user's* client cert? (2) does `403` mean authn or authz failed? (3) where does a pod find its service-account token? (4) why might a NetworkPolicy you applied do nothing?

RUNG 4 — The Vocabulary Map 🏷️

Term	What it actually is	Which box
CA / PKI	Trust anchor / whole cert system	Authn (identity)
client cert (CN/O)	A human/component's identity (CN=user, O=group)	Authn
CSR / signerName	Request to sign a cert / which signer	Authn (issue)
kubeconfig (cluster/user/context)	Where your cert + endpoint live	Authn (use)
server cert	apiserver/etcd/kubelet proving themselves	Authn (TLS)
authorization-mode	Node,RBAC,Webhook order	Authz
Role / RoleBinding	Namespaced permissions + grant	Authz
ClusterRole / ClusterRoleBinding	Cluster-wide permissions + grant	Authz
verbs / apiGroups / resources	The permission triple	Authz
can-i	Test a permission as a user	Authz (debug)
ServiceAccount / bound token	App identity + its credential	Authn (pods)
imagePullSecret	Registry creds for private images	Workload
securityContext / capabilities	UID + Linux powers a container has	Confinement (power)
NetworkPolicy / podSelector	Pod firewall + who it targets	Confinement (reach)
CRD / Operator	New API kind / its controller	(awareness)

The big unlock: four boxes, not ten topics

```
WHO ARE YOU (authn):        certs · CSR API · kubeconfig · service-account tokens
WHAT MAY YOU DO (authz):    Role/RoleBinding · ClusterRole/ClusterRoleBinding · can-i
WHAT POWER (confine):       securityContext (runAsUser, capabilities) · imagePullSecrets
WHO CAN IT TALK TO (confine): NetworkPolicy (podSelector/namespaceSelector/ipBlock)
```

Human and ServiceAccount are the **same two-step** (identity → RBAC), just different identity sources.

> ✅ **Check yourself before Rung 5:** Sort into the four boxes: a `ClusterRoleBinding`, a `runAsNonRoot: true`, a `CertificateSigningRequest`, a `podSelector` ingress rule.

RUNG 5 — The Trace 🎬

Follow ONE request through every gate

Trace — `kubectl get pods -n blue` as user `akshay`:

1. **kubeconfig** selects akshay's context → his **client cert** + the cluster CA + endpoint `:6443`.
2. **TLS handshake:** the apiserver presents its **server cert** (akshay verifies it against the cluster CA); akshay presents his **client cert**; the apiserver verifies it was **signed by the cluster CA**.

✅ **Authentication** passes → identity = `CN=akshay`, groups from `O=…`. (A cert signed by an untrusted CA → **401 Unauthorized**, and the trace ends here.)

3. **Authorization:** modes run in order. **Node** doesn't apply (akshay isn't a kubelet). **RBAC** searches namespace `blue` for a (Cluster)RoleBinding whose subject is akshay, whose Role allows verb `list` on resource `pods`. If found → allow; if not → **403 Forbidden** (authenticated but not permitted).
4. **Admission** controllers run (Section 2) — none reject a read.
5. The apiserver reads pods from **etcd** and returns them.

```
kubeconfig → TLS (server cert ✓ + client cert signed by cluster CA ✓)  ── authn ──▶ CN=akshay
      → Node? no · RBAC: binding+role allow (list,pods,blue)? ── authz ──▶ allow/403
      → admission → etcd → pod list
```

Trace — a blocked pod-to-pod call: `web` pod → `db:3306`. The `db-policy` NetworkPolicy selected `db` for Ingress, allowing only `podSelector name=api`. The CNI (Calico) has programmed the node's dataplane to **drop** `web`'s SYN to `db:3306` → `web`'s `nc` times out, while `api`'s connects (its reply is auto-allowed).

> ✅ **Check yourself before Rung 6:** In the first trace, at which step would a cert signed by the *wrong* CA fail, and what HTTP error results? At which step does a missing RoleBinding fail, and what error?

RUNG 6 — The Contrast ⚖️

This…	vs that…	Difference
Authentication (401)	**Authorization (403)**	*unknown identity* vs *known but not permitted*
Cluster CA	**etcd CA**	signs apiserver/user/kubelet certs vs signs only etcd's certs (separate trust)
Role / RoleBinding	**ClusterRole / ClusterRoleBinding**	one namespace vs cluster-wide (or all namespaces)
User	**ServiceAccount**	human identity (cert) vs app identity (token), same RBAC after
pod-level	**container-level** securityContext	default for all containers vs override (container wins; caps are container-only)
default network	**NetworkPolicy**	allow-all vs default-deny once a pod is selected
same `from` item	**two `from` items**	AND (pod *and* namespace) vs OR

When NOT to: don't grant `ClusterRole` when a namespaced `Role` suffices (least privilege); don't run containers as root or with `capabilities.add` beyond need (`drop: ["ALL"]` first); don't apply a NetworkPolicy on Flannel expecting enforcement; don't forget DNS egress when locking a pod's egress.

One-sentence "why this over that":

> Authenticate with certs/tokens and authorize with the *narrowest* RBAC (namespaced Role over ClusterRole), then confine workloads to a non-root identity with dropped capabilities and a default-deny NetworkPolicy — trust nothing more than it needs.

> ✅ **Check yourself before Rung 7:** A pod runs fine but `etcd` throws "certificate signed by unknown authority" after someone edited `etcd.yaml`. Which CA did they probably point it at, and which should it be?

RUNG 7 — The Prediction Test 🧪

Prediction 1 — CSR + RBAC gives exactly the granted access, nothing more

> **My prediction:** "If I sign akshay a cert and bind a pods-only Role, then `can-i delete pods --as akshay` is **yes** but `can-i delete nodes` is **no** — *because* authentication succeeds (valid cert) but authorization only matches the pods Role."

```
openssl genrsa -out akshay.key 2048
openssl req -new -key akshay.key -out akshay.csr -subj "/CN=akshay"
cat <<EOF | kubectl apply -f -
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata: { name: akshay }
spec:
  signerName: kubernetes.io/kube-apiserver-client
  usages: ["client auth"]
  request: $(cat akshay.csr | base64 -w0)
EOF
kubectl certificate approve akshay
kubectl create role pod-mgr --verb=get,list,create,delete --resource=pods
kubectl create rolebinding akshay-binding --role=pod-mgr --user=akshay
kubectl auth can-i delete pods  --as akshay      # yes
kubectl auth can-i delete nodes --as akshay      # no
```

Verify: `yes` for pods, `no` for nodes. A wrong `apiGroups` / `resourceNames` is the usual "forbidden."

Prediction 2 — A wrong cert path kills the control plane; crictl reveals it

> **My prediction:** "If a control-plane manifest points at a non-existent cert path, `kubectl` returns *connection refused* and the fix is only visible via `crictl logs` — *because* the apiserver/etcd container fails to start and there's no API to query."

```
kubectl get pods                         # connection refused
crictl ps -a | grep -E 'apiserver|etcd'  # exited container
crictl logs <etcd-id>                    # "server.crt: no such file"
ls /etc/kubernetes/pki/etcd/             # real file name
sudo vi /etc/kubernetes/manifests/etcd.yaml   # fix --cert-file path
watch crictl ps                          # etcd + apiserver restart
```

Verify: `kubectl` responds after the path fix. Check both the *path* and that etcd uses **its own** CA.

Prediction 3 — A NetworkPolicy default-denies everyone except the allowed selector

> **My prediction:** "If I select `db` for Ingress allowing only `name=api`, then `api` reaches `db:3306` but `web` times out — *because* selecting the pod flips it to default-deny and only the api rule is permitted (assuming an enforcing CNI)."

```
kubectl label pod db role=db ; kubectl label pod api name=api
kubectl apply -f - <<'EOF'
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: db-policy }
spec:
  podSelector: { matchLabels: { role: db } }
  policyTypes: [Ingress]
  ingress:
  - from: [{ podSelector: { matchLabels: { name: api } } }]
    ports: [{ protocol: TCP, port: 3306 }]
EOF
kubectl exec api -- nc -zv -w3 db 3306      # allowed
kubectl exec web -- nc -zv -w3 db 3306      # blocked (timeout)
```

Verify: `api` connects, `web` times out. If `web` *also* connects, your CNI isn't enforcing policy (Flannel) — that's the silent-no-op trap.

Prediction: "If I do X, then Y, because [mechanism]"	Ran?	Right?	Miss?
1.			

🎁 CAPSTONE — Compress It

One sentence, no notes:

> Every API request is authenticated (TLS client cert or SA token, verified against the cluster CA) then authorized (RBAC Role/ClusterRole bindings), while workloads are confined by the identity they run as (security context) and the pods they may reach (NetworkPolicy).

Explain it to a beginner in 3 sentences:

> 1. To use the cluster you present a certificate the cluster's CA signed (that's *who you are*), and RBAC rules then decide *what you're allowed to do* — a `403` means you're known but not permitted.
> 2. Apps get the same treatment through service accounts and tokens instead of certificates.
> 3. On top of that, you make containers run as non-root with minimal Linux powers, and use NetworkPolicies to stop pods from talking to pods they shouldn't.

Which rung to revisit hands-on?

- **Rung 3A (the two CAs)** — cluster CA vs etcd CA trips people on control-plane cert breaks. Fix: Prediction 2.

- **Rung 3E (NetworkPolicy AND/OR + DNS egress)** — the same-item-vs-two-items rule and the DNS-egress gotcha. Fix: Prediction 3, then add an egress policy and watch DNS break.

🎯 CKA exam tips & quick notes

- **Certs:** `openssl x509 -in <f> -text -noout` → **Subject(CN)/Issuer/SAN/validity**. All under `/etc/kubernetes/pki` (etcd has its own CA). kubectl down → **`crictl ps -a`** + **`crictl logs`**.
- **CSR:** `request` must be **`base64 -w0`**; `certificate approve/deny`; `CN`=user, `O`=group; deny `system:masters`.
- **RBAC:** `create role/rolebinding` (namespaced, `-n`) vs `clusterrole/clusterrolebinding`; **test with `kubectl auth can-i <verb> <res> --as <user> -n <ns>`**; `apiGroups: [""]`=core.
- **Service accounts:** `create serviceaccount`, `serviceAccountName` in spec, `create token`; token at `/var/run/secrets/kubernetes.io/serviceaccount`.
- **imagePullSecrets** + `create secret docker-registry` for private images.
- **securityContext:** container overrides pod; **caps container-only**; `runAsUser` immutable live → `replace --force`.
- **NetworkPolicy:** selecting a pod = default-deny that direction; replies auto-allowed; **egress must allow DNS(53)**; needs Calico/Cilium (not Flannel); same-`from`-item = AND.

📌 Command cheat sheet

```
# CERTS
openssl x509 -in /etc/kubernetes/pki/apiserver.crt -text -noout
crictl ps -a | grep apiserver ; crictl logs <id>
# CSR
kubectl get csr ; kubectl certificate approve <name> ; kubectl certificate deny <name>
# RBAC (test!)
k create role dev --verb=get,list,create --resource=pods
k create rolebinding devb --role=dev --user=dev-user
k create clusterrole nr --verb=get,list --resource=nodes
k create clusterrolebinding nrb --clusterrole=nr --user=michelle
k auth can-i create pods --as dev-user -n blue
# SERVICE ACCOUNTS / IMAGES
k create serviceaccount app-sa ; k create token app-sa
k create secret docker-registry regcred --docker-server=r --docker-username=u --docker-password=p
# KUBECONFIG
k config use-context <ctx> ; k config set-context --current --namespace=dev
```

Related sections

- Section 2 — Scheduling — admission controllers that run *after* RBAC.
- Section 4 — Application Lifecycle Management — Secrets + encryption at rest.
- Section 8 — Networking — the CNI that enforces NetworkPolicies; DNS for egress rules.
- Section 5 — Cluster Maintenance — the PKI you back up with etcd.
- Section 13 — Troubleshooting — cert/control-plane recovery with `crictl`.
- ../../Linux/26-tls-pki-openssl.md · ../../Linux/17-capabilities.md — the OpenSSL/PKI and Linux capabilities under certs and security contexts.
- ../../Networking/28-kubernetes-network-policies.md — NetworkPolicy from the networking side.

✅ Answers — "Check yourself before Rung N"

Before Rung 2

Q: Name the two distinct questions any secure system must answer for a request, and give a Kubernetes example of each.

A: The two questions are **authentication — "who are you?"** — and **authorization — "what may you do?"**. In Kubernetes, authentication is answered by presenting a TLS client certificate signed by the cluster CA (for humans/components) or a service-account token (for pods); failing it yields `401 Unauthorized`. Authorization is answered by RBAC: the apiserver looks for a Role/ClusterRole bound to your identity that allows the verb on the resource; failing it yields `403 Forbidden`.

Before Rung 3

Q: A user's `kubectl get pods` returns `Forbidden`. Which of the two gates did they pass, and which did they fail? What would `Unauthorized` have meant instead?

A: `403 Forbidden` means they **passed authentication** (the apiserver knows who they are — their cert or token was valid) but **failed authorization** — no RoleBinding/ClusterRoleBinding grants their identity the `get` / `list` verb on `pods` in that namespace. `401 Unauthorized` would have meant the first gate failed: the cluster couldn't establish their identity at all, e.g. a cert signed by an untrusted CA or an invalid token — the request never even reaches RBAC.

Before Rung 4

Q: Four quick ones: (1) which CA signs a *user's* client cert? (2) does `403` mean authn or authz failed? (3) where does a pod find its service-account token? (4) why might a NetworkPolicy you applied do nothing?

A: (1) The **cluster CA** (`/etc/kubernetes/pki/ca.crt` / `ca.key`, Issuer `CN=kubernetes`) — the controller-manager signs approved CSRs with it; the separate etcd CA signs only etcd's certs. (2) `403 Forbidden` is an **authorization (RBAC)** failure — authentication already succeeded. (3) At `/var/run/secrets/kubernetes.io/serviceaccount/token` — a short-lived bound token mounted into the pod. (4) Because the cluster's **CNI doesn't enforce NetworkPolicy** — plain Flannel ignores policies silently; you need an enforcing CNI like Calico, Cilium, or Weave.

Before Rung 5

Q: Sort into the four boxes: a `ClusterRoleBinding`, a `runAsNonRoot: true`, a `CertificateSigningRequest`, a `podSelector` ingress rule.

A: `ClusterRoleBinding` → **WHAT MAY YOU DO (authz)** — it grants a ClusterRole's permissions cluster-wide. `runAsNonRoot: true` → **WHAT POWER (confinement)** — a securityContext field limiting the identity a container runs as. `CertificateSigningRequest` → **WHO ARE YOU (authn)** — it's how a new user gets a signed client cert, i.e. authentication plumbing. A `podSelector` ingress rule → **WHO CAN IT TALK TO (confinement)** — part of a NetworkPolicy, the pod-level firewall.

Before Rung 6

Q: In the first trace, at which step would a cert signed by the *wrong* CA fail, and what HTTP error results? At which step does a missing RoleBinding fail, and what error?

A: A cert signed by the wrong CA fails at **step 2, the TLS handshake** — the apiserver cannot verify the client cert against the cluster CA, so authentication fails with `401 Unauthorized` and the

trace ends there. A missing RoleBinding fails at **step 3, authorization** — RBAC searches namespace `blue` for a (Cluster)RoleBinding whose subject is akshay with a Role allowing `list` on `pods`, finds none, and returns `403 Forbidden` (authenticated but not permitted).

Before Rung 7

Q: etcd throws "certificate signed by unknown authority" after someone edited `etcd.yaml`. Which CA did they probably point it at, and which should it be?

A: They probably pointed etcd at the **cluster CA** (`/etc/kubernetes/pki/ca.crt`), but etcd has its **own separate CA** — it should be `/etc/kubernetes/pki/etcd/ca.crt` (Issuer `etcd-ca`). The two CAs are separate trust domains: the cluster CA signs apiserver/user/kubelet certs, while the etcd CA signs only etcd's certs, so mixing them makes cert verification fail with "signed by unknown authority." Diagnose with `crictl ps -a` and `crictl logs` since `kubectl` is down, then fix the CA path in `/etc/kubernetes/manifests/etcd.yaml`.

Storage, Climbed the Ladder 🪜

Section 7 of the CKA — deriving how data outlives a pod

> Persisting data: volumes, **PV/PVC** binding, access modes, reclaim policies, and **StorageClasses**. We climb from **the pain of ephemeral data → the one "claim, don't name the disk" idea → the binding machinery** → then the commands as predictions. Each rung ends with a ✅ **Check yourself**.

RUNG 0 — The Setup 🎯

What am I learning? How a pod gets storage that survives its death, and how admins/StorageClasses supply that storage without every pod hard-coding a disk.

Why did it land on my desk? *Storage* is 10% of the CKA, and "**why is my PVC Pending?**" plus PV/PVC binding are classic tasks.

What do I already know? Maybe that pods lose data when they die. What's fuzzy: the PV↔PVC binding rules, why a claim can bind a *bigger* volume, and why some PVCs sit Pending *on purpose*.

RUNG 1 — The Pain 🔥

Why does persistent storage exist at all?

A pod's container filesystem is scratch — it dies with the pod:

```
THE EPHEMERAL-DATA PAIN
  db pod restarts     → its writable layer is gone → the database is empty
  hostPath /data      → works on 1 node; on a 3-node cluster each node's /data differs
  bake disk config into every pod → users must know the storage backend (AWS? NFS?) → tight coupling
  need a disk per app → admin manually makes a disk, then a PV, every single time → doesn't scale
```

Before / without it: data lived in the container and vanished on restart; the only "persistence" was a node-local `hostPath` that broke the moment a pod moved to another node.

What breaks without it: stateful workloads (databases, queues), portability (a pod that moves nodes must keep its data), and separation of concerns (app authors shouldn't need to know the storage vendor).

Who feels it most? App teams (their data) *and* the platform team (who must offer storage safely and at scale).

> ✅ **Check yourself before Rung 2:** Why is a `hostPath` volume a trap on a multi-node cluster? (Hint: what does `/data` point to when the pod reschedules onto a different node?)

RUNG 2 — The One Idea 💡

The single sentence everything hangs off

> **Kubernetes storage decouples the app from the disk through a *claim*: an admin (or a StorageClass) offers PersistentVolumes, a pod's PersistentVolumeClaim requests a size + access mode + class, Kubernetes binds a matching PV to the claim 1:1, and the pod mounts the *claim* — so the pod never names a physical disk.**

Derivations:

- *"the pod mounts the claim, never a disk"* → app authors write `claimName: my-pvc`; the storage backend (EBS, NFS, local) is the admin's/StorageClass's concern.
- *"binds a matching PV... 1:1"* → binding needs three matches (capacity ≥, access mode, class); a claim can bind a *larger* PV (rest wasted); no match → **Pending**.
- *"an admin **or a StorageClass** offers PVs"* → two worlds: **static** (admin pre-creates PVs) and **dynamic** (a StorageClass provisions a PV on demand when the PVC appears).
- *"requests... access mode"* → RWO/RWX etc. decide how many nodes/pods can mount it.

Once you see **PVC = a request, PV = a supply, binding = matchmaking**, "why is it Pending" becomes "which of the three matches failed."

> ✅ **Check yourself before Rung 3:** A 50Mi PVC bound to a 100Mi PV. Why is that allowed, and what happens to the other 50Mi? What does "1:1" mean for a second claim wanting that PV?

RUNG 3 — The Machinery ⚙️

How it ACTUALLY works — go slow

> ### 🧸 Plain-English first (read this before the technical version)
>
> Imagine each running app (a "pod") as a hotel guest. Anything the guest leaves in the room is thrown out at checkout — apps normally lose all their files when they stop. This section explains the storage system that lets a guest keep belongings safely between stays.
>
> **(A) Volumes — the basic idea.** A "volume" is simply a storage box attached to the room instead of keeping things in the wastebasket-by-default room itself. There are three kinds:
>
> - A scratch box (`emptyDir` — everyday words: a temporary bin) that's tossed when the guest leaves.
> - A box bolted to one specific hotel building (`hostPath` — a folder on one particular computer). Trap: if the guest is moved to a different building, their box stays behind — it only works when there's just one building.
> - Professional off-site storage from any vendor, connected through a universal adapter socket (called "CSI") so the hotel can work with any storage company.
>
> **(B) Supply, request, and matchmaking.** The clever part: guests never name a specific storage unit. Instead, the facility manager stocks a supply of storage units (each is a "PersistentVolume," or PV), and a guest files a claim ticket (a "PersistentVolumeClaim," or PVC) saying "I need at least this much space, with this kind of access." The system then plays matchmaker: a claim gets a unit only if three things line up — the unit is big enough, the access style matches (one user at a time vs. shared, like a private locker vs. a shared garage), and they're in the same service tier. A small claim can win a bigger unit — the extra space is just wasted — and each unit serves exactly one claim. No match? The ticket sits in the "Pending" tray. There's also a policy for what happens to a unit when its claim is cancelled: keep the contents for manual cleanup ("Retain") or shred everything ("Delete").
>
> **(C) StorageClasses — storage on demand.** Rather than the manager pre-building every unit by hand, a "StorageClass" is like a standing contract with a storage company: when a claim ticket names that tier, a brand-new unit is built automatically. One smart option ("WaitForFirstConsumer") delays building the unit until we know which building the guest will actually stay in — so the unit is built nearby. That's why some tickets sit in Pending on purpose: the system is just waiting to see where the guest lands. It's normal, not a fault.

Now the original technical deep-dive — the same ideas, in precise form:

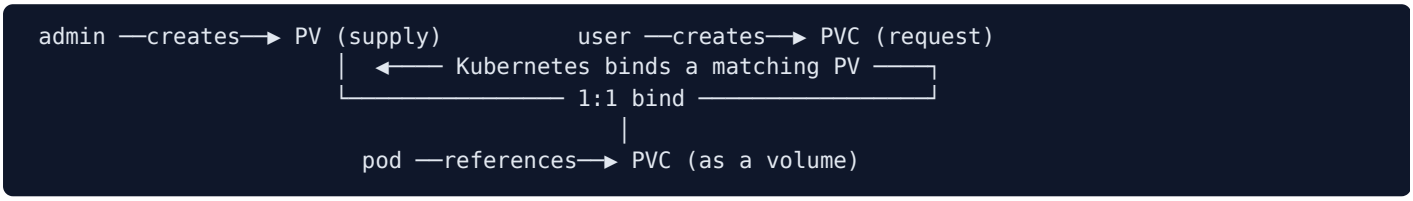
(A) Volumes — the raw form

A **volume** stores data outside the container's writable layer. Kubernetes generalizes Docker's mounts and adds cloud/network backends via the **CSI (Container Storage Interface)** so any vendor plugs in.

```
spec:
  containers:
  - name: app
    volumeMounts: [{ name: data, mountPath: /opt }]   # where the container sees it
  volumes:
  - name: data
    hostPath: { path: /data, type: DirectoryOrCreate } # a directory ON THE NODE
```

- `emptyDir` — scratch, lives with the pod (shared between its containers).
- `hostPath` — a node directory. ⚠️ **single-node only** — each node's `/data` differs.
- **cloud/CSI** (`awsElasticBlockStore` , `nfs` , `ebs.csi.aws.com` …) — real cross-node persistence.

(B) PV/PVC — supply, request, bind




```
kind: PersistentVolume                # apiVersion: v1
metadata: { name: pv-log }
spec:
  capacity: { storage: 100Mi }
  accessModes: ["ReadWriteMany"]
  persistentVolumeReclaimPolicy: Retain
  hostPath: { path: /pv/log }         # demo backend
---
kind: PersistentVolumeClaim
metadata: { name: claim-log }
spec:
  accessModes: ["ReadWriteMany"]      # MUST be satisfiable by the PV
  resources: { requests: { storage: 50Mi } }
```

Access modes: RWO (one node RW), ROX (many nodes RO), RWX (many nodes RW), RWOP (one *pod* RW).

Reclaim policy (on PVC delete): **Retain** (default for manual PVs — kept, → `Released`, manual cleanup), **Delete** (default for dynamic — removes PV + backend), **Recycle** (deprecated).

The binding triple — a PVC binds a PV when ALL hold: capacity **≥** request, **accessModes** match, **storageClassName** match. Otherwise the PVC is **Pending**.

```
# use it in a pod:
  volumes:
  - name: logs
    persistentVolumeClaim: { claimName: claim-log }
```

> 🎯 A **Pending PVC** with a seemingly-fitting PV is almost always an **access-mode** or **storageClassName** mismatch → `kubectl describe pvc` . Deleting a **bound** PVC that a pod still uses hangs in **Terminating** (finalizer) until the pod is gone.

(C) StorageClass — dynamic provisioning

Manually making a disk *then* a PV is **static**. A **StorageClass** automates it: the PVC names a class, and its **provisioner** creates the disk + PV on demand.

```
kind: StorageClass                    # storage.k8s.io/v1
metadata: { name: gold }
provisioner: ebs.csi.aws.com          # backend; "no-provisioner" = static/local
parameters: { type: gp3 }
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer   # vs Immediate
```

- `volumeBindingMode: Immediate` binds/provisions at once; `WaitForFirstConsumer` waits until a **pod** uses the PVC (so the volume lands in the pod's zone — topology-correct).
- The **default StorageClass** is applied to PVCs with no `storageClassName` by the `DefaultStorageClass` **admission controller** (Section 2).

> 🎯 A PVC on a `WaitForFirstConsumer` class stays **Pending** with *"waiting for first consumer"* — this is **normal**, not a bug; it binds when a pod mounts it. Local-storage (`no-provisioner`) classes work this way.

> ✅ **Check yourself before Rung 4:** Name the three things that must match for a PVC to bind a PV. And: which mechanism creates a PV *automatically* versus an admin creating it by hand?

RUNG 4 — The Vocabulary Map 🏷️

Term	What it actually is	Which part
volume	Storage attached to a pod	Raw form
emptyDir / hostPath	Pod-scratch / node-dir volume	Raw (node-local)
CSI	Vendor plug-in interface for storage	Backend
PersistentVolume (PV)	A piece of supplied storage	Supply
PersistentVolumeClaim (PVC)	A request for storage	Request
capacity / accessModes / storageClassName	The three binding matches	Binding
RWO/ROX/RWX/RWOP	How many nodes/pods can mount RW/RO	Access modes
reclaimPolicy (Retain/Delete)	Fate of the PV on PVC delete	Lifecycle
Released	PV state after PVC deleted (Retain)	Lifecycle
StorageClass	Template for dynamic PV creation	Dynamic
provisioner	The backend that makes disks	Dynamic
volumeBindingMode / WaitForFirstConsumer	Bind now vs when a pod mounts	Dynamic
DefaultStorageClass	Admission controller assigning the default class	Dynamic

The unlock — two worlds, one binding:

```
STATIC:   admin creates PV  ─┐
                             ├─▶ PVC binds a matching PV (capacity ≥, accessModes, class)
DYNAMIC:  StorageClass makes PV on demand ─┘   ─▶ pod mounts the PVC
```

> ✅ **Check yourself before Rung 5:** Put these in "static" or "dynamic": an admin hand-writes a PV; a PVC names class `gold` and a disk appears; `provisioner: no-provisioner`.

RUNG 5 — The Trace 🎬

Trace — a stateful pod gets durable storage (dynamic):

1. You create a **PVC** naming class `gold`, `RWO`, `10Gi`.
2. The `DefaultStorageClass`/named class routes it to the `ebs.csi.aws.com` **provisioner**.
3. With `WaitForFirstConsumer`, binding **waits** — the PVC is `Pending` ("waiting for first consumer"). *(With `Immediate`, a disk + PV would appear now.)*
4. You create a **pod** referencing the PVC. The scheduler places it; now the provisioner creates an **EBS disk in that node's zone**, wraps it in a **PV**, and **binds** the PV to the PVC 1:1.
5. The **kubelet** mounts the volume into the container at `mountPath`. The app writes data.
6. The pod dies and reschedules → the same PVC re-binds the same PV → **the data is still there**.
7. Later the PVC is deleted → **reclaimPolicy** decides: `Delete` removes the PV + EBS disk; `Retain` keeps the PV as `Released` for manual cleanup.

```
PVC(gold,RWO,10Gi) ─▶ provisioner ─(WaitForFirstConsumer)─▶ Pending
      pod mounts PVC ─▶ disk+PV created in pod's zone ─▶ bind 1:1 ─▶ kubelet mounts ─▶ data persists
      PVC deleted ─▶ reclaimPolicy: Delete→gone | Retain→Released
```

Trace — a Pending PVC (static, mismatch): PVC(RWO) vs the only PV(RWX) → binder finds **no access-mode match** → PVC stays `Pending`; `describe pvc` says so; fix the mode → it binds (to the whole PV, even if bigger than requested).

> ✅ **Check yourself before Rung 6:** In the dynamic trace, why did the disk only get created at step 4 (not step 1)? What real-world problem does that timing solve?

RUNG 6 — The Contrast ⚖️

This…	vs that…	Difference
hostPath / emptyDir	**PV/PVC**	node-local / pod-scratch vs portable, claimed, durable
static provisioning	**dynamic (StorageClass)**	admin pre-creates PVs vs auto-created on demand
Retain	**Delete**	keep data (manual cleanup) vs wipe PV + backend
Immediate	**WaitForFirstConsumer**	bind now vs bind when a pod mounts (topology-aware)
RWO	**RWX**	one node RW vs many nodes RW (needs a shared backend like NFS)

When NOT to: don't use `hostPath` for anything that must survive rescheduling; don't set `Delete` on a PV holding data you can't lose; don't expect `RWX` from a block device (EBS is RWO — you need NFS/EFS for RWX); don't "fix" a `WaitForFirstConsumer` Pending PVC — mount it with a pod.

One-sentence "why this over that":

> Claim storage with a PVC so the pod never hard-codes a disk; use a StorageClass for on-demand dynamic provisioning, `Retain` for data you must not lose, and match access modes to how many nodes need to write.

> ✅ **Check yourself before Rung 7:** A teammate set a database PV's reclaim policy to `Delete` and deleted the PVC to "clean up." What just happened to the data, and what policy should they have used?

RUNG 7 — The Prediction Test 🧪

Prediction 1 — hostPath survives the pod but is node-local

> **My prediction:** "If I add a `hostPath` volume, then the file appears on the node's path and survives pod deletion — but only on *that* node — *because* hostPath is just a directory on the node the pod runs on."

```
kubectl edit pod webapp        # add volume + volumeMount → save fails → /tmp
kubectl replace --force -f /tmp/kubectl-edit-XXXX.yaml
kubectl exec webapp -- cat /log/app.log
cat /var/log/webapp/app.log    # same content, on the NODE
```

```
  volumes: [{ name: log-vol, hostPath: { path: /var/log/webapp } }]
  # container: volumeMounts: [{ name: log-vol, mountPath: /log }]
```

Verify: the file persists on the node after pod recreate. Move the pod to another node and the file's *gone from view* — proving node-locality.

Prediction 2 — An access-mode mismatch keeps the PVC Pending

> **My prediction:** "If my PV is RWX but my PVC asks RWO (or the classes differ), the PVC stays **Pending**; matching the access mode binds it — to the *PV's* capacity, even if I requested less — *because* binding requires capacity≥, accessModes, and class to all match, 1:1."

```
kubectl apply -f pv.yaml -f pvc.yaml
kubectl get pv,pvc                 # PV Available, PVC Pending
kubectl describe pvc claim-log     # no matching PV (access mode)
# fix PVC accessModes → ReadWriteMany:
kubectl replace --force -f pvc.yaml
kubectl get pvc                    # Bound, shows 100Mi (the PV's size)
```

Verify: Pending → Bound after matching the mode; bound capacity = the PV's. This is the #1 "PVC Pending" cause.

Prediction 3 — WaitForFirstConsumer binds only when a pod mounts

> **My prediction:** "If a PVC uses a `WaitForFirstConsumer` class, it stays **Pending** ('waiting for first consumer') until I create a pod that mounts it, then it **Binds** — *because* the class delays binding until a consumer's placement is known."

```
kubectl get pvc local-pvc          # Pending
kubectl describe pvc local-pvc     # "waiting for first consumer"
kubectl run nginx --image=nginx:alpine --dry-run=client -o yaml > nginx.yaml
# add volumes.persistentVolumeClaim.claimName=local-pvc + a volumeMount
kubectl apply -f nginx.yaml
kubectl get pvc local-pvc          # Bound
```

Verify: Pending → Bound the instant a pod mounts it. That Pending was *expected*, not a fault.

Prediction: "If I do X, then Y, because [mechanism]"	Ran?	Right?	Miss?
1.			

🎁 CAPSTONE — Compress It

One sentence, no notes:

> A pod mounts a PersistentVolumeClaim (never a disk); Kubernetes binds that claim 1:1 to a PersistentVolume matching its capacity, access mode, and class — supplied either statically by an admin or dynamically by a StorageClass — and the reclaim policy decides the volume's fate when the claim is deleted.

Explain it to a beginner in 3 sentences:

> 1. Container storage disappears when a pod dies, so you attach a volume — and for durable storage, the pod uses a *claim* rather than naming a real disk.
> 2. Kubernetes matches that claim to an available volume by size, access mode, and class; if nothing matches, the claim sits Pending.
> 3. A StorageClass can create the disk automatically on demand, and a reclaim policy decides whether the disk is kept or wiped when you delete the claim.

Which rung to revisit hands-on? Rung 3B + Prediction 2 — the **three-match binding rule** and reading `describe pvc` are what turn "PVC Pending" from mystery into a 20-second fix.

🎯 CKA exam tips & quick notes

- **Binding needs three matches:** capacity (≥), **accessModes**, **storageClassName**. Debug Pending with `kubectl describe pvc`.
- **Access modes:** RWO / ROX / RWX / RWOP — RWO PV won't satisfy an RWX claim.
- **Reclaim:** `Retain` (keep → Released) vs `Delete` (auto-wipe) vs `Recycle` (deprecated).
- **Deleting a bound PVC** hangs `Terminating` until the consuming pod is gone.
- **StorageClass** = dynamic; `no-provisioner` = static/local; `WaitForFirstConsumer` PVCs stay Pending until a pod mounts — *expected*.
- Mount via `volumes[].persistentVolumeClaim.claimName` + `volumeMounts`. Short names `pv/pvc/sc`.

📌 Command cheat sheet

```
kubectl get pv,pvc,sc                          # overview
kubectl describe pvc <name>                     # why Pending?
kubectl get pvc <name> -o jsonpath='{.status.phase}{"\n"}'
# in a Pod/Deployment:
#   volumes: [{ name: v, persistentVolumeClaim: { claimName: my-pvc } }]
#   volumeMounts: [{ name: v, mountPath: /data }]
```

Related sections

- Section 2 — Scheduling — the `DefaultStorageClass` mutating admission controller.
- Section 4 — Application Lifecycle Management — mounting ConfigMaps/Secrets as volumes.
- Section 1 — Core Concepts — pods and how volumes attach.
- Section 13 — Troubleshooting — a pod stuck `ContainerCreating` on a volume.
- ../../Linux/15-storage-mounts.md — the Linux mount/filesystem layer underneath volumes.

✅ Answers — "Check yourself before Rung N"

Before Rung 2

Q: Why is a `hostPath` volume a trap on a multi-node cluster?

A: A `hostPath` volume is just a directory **on the node the pod happens to run on**. When the pod reschedules onto a different node, `/data` now points to *that* node's local directory — a different, likely empty, filesystem — so the data written on the first node is gone from the pod's view. Each

node's `/data` differs, which is why hostPath only "works" on a single-node cluster and breaks portability the moment a pod moves.

Before Rung 3

Q: A 50Mi PVC bound to a 100Mi PV — why is that allowed, what happens to the other 50Mi, and what does "1:1" mean for a second claim?

A: Binding only requires the PV's capacity to be **≥** the claim's request (plus matching accessModes and storageClassName), so a 50Mi claim can bind a 100Mi PV. The claim gets the **whole PV** — the extra 50Mi is simply **wasted**; it isn't carved off or shared. "1:1" means one PV binds to exactly one PVC, so a second claim can never bind that PV while it's bound — it must find another matching PV or sit Pending.

Before Rung 4

Q: Name the three things that must match for a PVC to bind a PV. Which mechanism creates a PV automatically versus an admin creating it by hand?

A: The binding triple: (1) **capacity** — the PV's storage must be ≥ the PVC's request; (2) **accessModes** — the PV must satisfy the claim's mode (RWO/ROX/RWX/RWOP); (3) **storageClassName** — the class names must match. If any fails, the PVC stays Pending. A **StorageClass with a provisioner** creates PVs automatically on demand (**dynamic** provisioning), whereas an admin hand-writing PV manifests is **static** provisioning.

Before Rung 5

Q: Put these in "static" or "dynamic": an admin hand-writes a PV; a PVC names class `gold` and a disk appears; `provisioner: no-provisioner`.

A: An admin hand-writing a PV is **static** — the supply is pre-created manually. A PVC naming class `gold` that makes a disk appear is **dynamic** — the class's provisioner (e.g. `ebs.csi.aws.com`) creates the disk and PV on demand. `provisioner: no-provisioner` is **static** — the class exists (typically for local storage with `WaitForFirstConsumer`) but nothing auto-creates PVs; an admin still supplies them.

Before Rung 6

Q: In the dynamic trace, why did the disk only get created at step 4 (not step 1)? What real-world problem does that timing solve?

A: The class used `volumeBindingMode: WaitForFirstConsumer`, so provisioning and binding are delayed until a **pod actually mounts the PVC** — until then the PVC sits Pending with "waiting for first consumer," which is normal. The timing solves the **topology problem**: only once the scheduler places the pod does the provisioner know which node (and therefore which availability zone) to create the disk in, so the EBS disk lands in the pod's zone. With `Immediate`, the disk could be provisioned in a zone the pod can never reach.

Before Rung 7

Q: A teammate set a database PV's reclaim policy to `Delete` and deleted the PVC to "clean up." What happened to the data, and what policy should they have used?

A: With `reclaimPolicy: Delete`, deleting the PVC removed **both the PV and the backend disk** — the database's data is gone. They should have used **`Retain`**: on PVC deletion the PV is kept and

moves to the `Released` state, preserving the data for manual cleanup or recovery. Rule of thumb from the ladder: never set `Delete` on a PV holding data you can't lose.

Networking, Climbed the Ladder 🪜

Section 8 of the CKA — deriving how traffic finds a pod

> How traffic moves: **CNI** (pod IPs), **kube-proxy** (service VIPs), **CoreDNS** (names), and **Ingress** (L7 edge). It looks like four unrelated tools; it's really **one four-layer stack answering "how do I reach that?"** We climb from **the pain of ephemeral pod IPs → the layered idea → the machinery** → then the commands as predictions. Each rung ends with a ✅ **Check yourself**.
>
> **Prereqs** live in the sibling guides: ../../Linux/13-namespaces.md (veth/netns), ../../Networking/09-dns.md (resolv.conf, search domains), ../../Networking/24-kubernetes-pod-networking-cni.md. This file is the **Kubernetes** layer.

RUNG 0 — The Setup 🎯

What am I learning? The four layers that let anything reach anything in a cluster: pod-to-pod (CNI), name-and-VIP-to-pod (Services + kube-proxy + CoreDNS), and outside-to-app (Ingress).

Why did it land on my desk? *Services & Networking* is 20% of the CKA. The CNI/kube-proxy/CoreDNS **inspection commands** and **Ingress creation** are reliable points, and "pod can't reach service" is a staple troubleshooting task.

What do I already know? Probably that Services exist (Section 1). What's fuzzy: what actually forwards a packet to a pod, why a Service IP won't ping, and why short DNS names fail across namespaces.

RUNG 1 — The Pain 🔥

Why does cluster networking exist at all?

Pods are ephemeral and spread across nodes. Getting a packet to the right one is genuinely hard:

```
THE REACHABILITY PAIN
  pod IPs change on restart      → hard-code one and it breaks constantly
  pods live on different nodes   → how does a pod on node A reach a pod on node B?
  which of 3 replicas do I hit?  → you need ONE stable address that load-balances
  address by name, not IP        → apps want "mysql", not 10.244.3.7
  expose 20 apps externally      → 20 cloud LoadBalancers = 20 bills, no host/path routing
```

Before / without it: you'd wire up routes and NAT by hand, chase changing IPs, and pay for a load balancer per service — with no name-based discovery.

What breaks without each layer: no **CNI** → pods have no IPs / can't cross nodes; no **kube-proxy** → a Service VIP forwards nowhere; no **CoreDNS** → you must use raw IPs; no **Ingress** → external HTTP routing and shared TLS are impossible.

Who feels it most? The platform team — you own a network that must be flat, stable, name-addressable, and externally reachable, across many nodes and constant pod churn.

> ✅ **Check yourself before Rung 2:** Give the single reason you can't just point your frontend at a backend pod's IP. What property of pods makes that fragile?

RUNG 2 — The One Idea 💡

The single sentence everything hangs off

> **Kubernetes networking is a four-layer stack, each answering "how do I reach that?": the CNI gives every pod a unique, flat, NAT-free IP; kube-proxy turns a Service's virtual IP into iptables/IPVS rules that DNAT to a live pod; CoreDNS maps names to those virtual IPs; and Ingress adds one L7 door that routes external traffic by host/path.**

Derivations:

- *"CNI gives every pod a flat NAT-free IP"* → the **network model's rules** (every pod reachable without NAT); Kubernetes doesn't implement it — you deploy a CNI addon, and a `NotReady` node usually means *no CNI*.
- *"kube-proxy… DNAT to a live pod"* → a Service is a **virtual IP nothing listens on**; kube-proxy programs the kernel to rewrite it to a real pod IP. That's why a ClusterIP **won't ping** — only its DNAT'd ports work.
- *"CoreDNS maps names to VIPs"* → `mysql` → `10.96.x.y`; the kubelet points every pod's `/etc/resolv.conf` at CoreDNS; **search domains** are why short names work *within* a namespace.
- *"Ingress = one L7 door"* → replaces N LoadBalancers with host/path routing + shared TLS, but needs a **controller** to do anything.

Read a failure by asking *which layer*: no IP → CNI; VIP forwards nowhere → kube-proxy; name won't resolve → CoreDNS; external 404 → Ingress.

> ✅ **Check yourself before Rung 2→3:** Why does a Service's ClusterIP not respond to `ping`, yet `curl <clusterIP>:80` works? Which layer explains it?

RUNG 3 — The Machinery ⚙️

How it ACTUALLY works — the most important rung. Go slow.

🧸 Plain-English first (read this before the technical version)

Picture the cluster as a city, and each running app (a "pod") as a shop that opens and closes at unpredictable addresses. This section explains the four layers that let anyone find and reach any shop — plus, first, a short list of official phone lines (fixed port numbers — think of them as well-known extensions the city's departments always answer on, which the security guards must not block).

Layer 1 — Streets and addresses (the "CNI," the city's road contractor). Kubernetes itself doesn't pave roads; you hire a contractor (a network plug-in) that gives every shop its own unique street address and builds roads so any shop can reach any other directly — even in another district (another computer) — with no detours or address-translation tricks. The contractor's tools and blueprints live in two known filing drawers on each computer. If a new district shows up as "NotReady," it usually means no road contractor was hired yet. One rule: the shops' address range and the phone-number range (next layer) must never overlap.

Layer 2 — One stable phone number per business ("Services" and "kube-proxy"). Shops move constantly, so each business gets a permanent phone number that isn't attached to any physical shop — it's a virtual number. A switchboard operator on every corner ("kube-proxy") keeps rewiring that number to whichever shop branch is currently open, using the building's wiring rules (kernel rules called iptables). That's why you can *call* the number but can't *knock on its door* — there's no building there (why the address won't answer a "ping").

Layer 3 — The phone book ("CoreDNS"). Nobody memorizes numbers; you ask directory assistance for "mysql" and get the virtual number back. Every shop is automatically given the directory's contact card. Handy shortcut: within your own neighborhood (a "namespace" — a walled-off area) first names work; to call another neighborhood you must use the full name, like "mysql.payroll."

Layer 4 — The city's front gate ("Ingress"). For visitors from outside the city, instead of building a separate private tollbooth per business (one paid load balancer each), you build one smart front gate that reads the visitor's destination — which website name and which page path — and directs them to the right business. Crucially, the gate needs a gatekeeper actually hired ("an Ingress controller" — software you must install); the rulebook alone does nothing. And sometimes the gate must trim the path off the address before forwarding, or the shop won't recognize it (the "rewrite" fix for 404 not-found errors).

Now the original technical deep-dive — the same ideas, in precise form:

Ports to know first (firewall/SG): apiserver **6443**, kubelet **10250**, scheduler **10259**, controller-mgr **10257**, etcd **2379/2380**, NodePort **30000–32767**.

(A) Layer 1 — pod IPs via the CNI

The **network model (4 rules):** every pod gets a unique IP; every pod reaches every other pod **without NAT** (same *and* cross node); nodes reach pods. A **CNI plugin** implements this:

- **Binaries:** `/opt/cni/bin/` (bridge, flannel, host-local…).

- **Active config:** `/etc/cni/net.d/*.conflist` (lowest filename wins) — the runtime reads it.
- On pod create, the **kubelet → runtime → CNI plugin** (`ADD`): make a **veth**, assign an IP (**IPAM**, usually `host-local`), add routes. The addon (Weave/Flannel/Calico, a **DaemonSet**) owns the **pod CIDR** (e.g. `10.244.0.0/16`).

```
ls /opt/cni/bin ; ls /etc/cni/net.d ; cat /etc/cni/net.d/*.conflist
ps -aux | grep kubelet | grep -o 'container-runtime-endpoint=[^ ]*'
```

> 🎯 **Pod CIDR and Service CIDR must NOT overlap.** `NotReady` right after install = **no CNI** → apply an addon.

(B) Layer 2 — service VIPs via kube-proxy

A **Service is a virtual IP** — no process listens on it. **kube-proxy** (a **DaemonSet**) watches Services/Endpoints and writes **iptables** (default) or **IPVS** rules that **DNAT** the service IP → a backend pod IP.

```
client pod → ClusterIP 10.96.0.50:80
   │  (kube-proxy wrote this iptables NAT rule)
   ▼ DNAT → pod 10.244.1.7:80   (routable via the CNI from Layer 1)
```

- **Service CIDR:** apiserver `--service-cluster-ip-range` (commonly `10.96.0.0/12`).

```
kubectl get pods -n kube-system -o wide | grep kube-proxy
kubectl logs -n kube-system <kube-proxy-pod> | grep -i proxier   # "Using iptables Proxier"
iptables -t nat -L KUBE-SERVICES -n | grep <clusterIP>           # the DNAT rules
```

(C) Layer 3 — names via CoreDNS

CoreDNS (pods in `kube-system`, fronted by the `kube-dns` **Service**, ClusterIP commonly `10.96.0.10`) resolves names. The kubelet injects that IP into every pod's `/etc/resolv.conf`.

- **Service name:** `<service>.<namespace>.svc.cluster.local` (short name works *within* a namespace via `search` domains).
- **Pod name:** `<ip-with-dashes>.<namespace>.pod.cluster.local`.

```
kubectl get svc -n kube-system kube-dns                  # 10.96.0.10
kubectl get configmap coredns -n kube-system -o yaml     # the Corefile (cluster.local zone)
kubectl exec test -- cat /etc/resolv.conf                # nameserver 10.96.0.10 + search domains
kubectl exec test -- nslookup web-service.payroll        # cross-namespace
```

> 🎯 Cross-namespace = FQDN `<svc>.<ns>` (short names only same-ns; **pods** need full FQDN). If DNS fails cluster-wide, check the CoreDNS pods are `Running`.

(D) Layer 4 — external L7 via Ingress

One `Service type: LoadBalancer` per app = one cloud LB each, no host/path routing. **Ingress** = one L7 entrypoint. **Two halves:** the **Ingress Controller** (nginx/Traefik/ALB — you must **deploy one**; not built in) and the **Ingress resource** (rules). No controller ⇒ nothing happens.

```
kind: Ingress                          # networking.k8s.io/v1
metadata:
  name: shop
  annotations: { nginx.ingress.kubernetes.io/rewrite-target: / }
spec:
  rules:
  - host: my-online-store.com
    http:
      paths:
      - { path: /wear,  pathType: Prefix, backend: { service: { name: wear-service,  port: { number:
80 } } } }
      - { path: /watch, pathType: Prefix, backend: { service: { name: video-service, port: { number:
80 } } } }
```

```
kubectl create ingress ingress-test --rule="wear.my-online-store.com/wear*=wear-service:80"
```

🎯 `rewrite-target: /` strips the matched path (backends often lack `/wear` → 404 without it). An Ingress can only reference Services in its **own namespace**.

✅ **Check yourself before Rung 4:** For each layer name the component and where it lives: (1) gives a pod its IP, (2) DNATs a service VIP, (3) resolves `mysql`, (4) routes `example.com/pay`. Which two run as DaemonSets?

RUNG 4 — The Vocabulary Map 🏷️

Term	What it actually is	Layer
CNI plugin / addon	Implements pod networking	1 pod IPs
/opt/cni/bin, /etc/cni/net.d	Plugin binaries / active config	1
IPAM / veth / pod CIDR	IP allocation / pod's cable / pod IP range	1
Service (VIP) / ClusterIP	Stable virtual IP for a pod set	2 service
kube-proxy	Programs iptables/IPVS DNAT (DaemonSet)	2
iptables / IPVS / DNAT	The kernel rules that rewrite VIP→pod	2
service CIDR	Range VIPs come from	2
CoreDNS / kube-dns	Name resolver / its Service	3 DNS
resolv.conf / search domain	Pod's DNS config / why short names work	3
FQDN	`<svc>.<ns>.svc.cluster.local`	3
Ingress controller	The proxy pods doing L7 routing	4 edge
Ingress resource	The host/path rules	4
rewrite-target / pathType	URL rewrite / match style	4

The unlock — the layers stack:

```
OUTSIDE ─► [4] Ingress (host/path, L7) ─► [2] Service VIP (kube-proxy DNAT)
                                              ▲ resolved by [3] CoreDNS (name → VIP)
                                              ▼
                                          [1] Pod IP (CNI, flat/NAT-free)
```

> ✅ **Check yourself before Rung 5:** Which layer would you suspect for each: `NotReady` node; ClusterIP reachable but `nslookup mysql` fails; external `/pay` returns 404; two pods on different nodes can't ping each other?

RUNG 5 — The Trace 🎬

Follow ONE request down all four layers

Trace — browser hits `my-store.com/wear`, which internally calls `mysql`:

1. **DNS (public):** `my-store.com` resolves to the **Ingress controller's** external IP/LB.
2. **[4] Ingress:** the controller matches `host=my-store.com, path=/wear` → backend `wear-service:80`; `rewrite-target: /` changes the path to `/`.
3. **[3] CoreDNS:** to reach `wear-service`, the name resolves (via `/etc/resolv.conf` → `10.96.0.10`) to the Service **ClusterIP**.
4. **[2] kube-proxy:** the packet to that ClusterIP hits the node's **iptables** `KUBE-SERVICES` chain → **DNAT** to one live `wear` **pod IP**.
5. **[1] CNI:** that pod IP is routable across nodes via the CNI's routes/veth; the packet arrives at the `wear` pod.
6. The `wear` app calls `mysql` → **[3]** CoreDNS resolves `mysql.<ns>.svc.cluster.local` → **[2]** kube-proxy DNATs the mysql Service VIP → **[1]** CNI routes to the mysql pod. Same three inner layers, one level deeper.
7. Responses walk back; kube-proxy's conntrack un-DNATs so the client sees replies from the VIP it sent to.

```
browser →(pubDNS) Ingress-LB →[4 host/path]→ Service VIP
                                  ▲[3 CoreDNS name→VIP]
                                  ▼[2 kube-proxy DNAT VIP→podIP]
                                wear pod [1 CNI routes] → calls mysql (repeat 3→2→1)
```

> ✅ **Check yourself before Rung 6:** In this trace, which layer made `wear-service` a *name* you could use, and which one actually forwarded the packet to a specific pod? Why are those two different components?

RUNG 6 — The Contrast ⚖️

This…	vs that…	Difference
CNI	**kube-proxy**	gives pods *real* IPs / routes vs turns *virtual* Service IPs into DNAT
Service (L4)	**Ingress (L7)**	stable IP + port load-balance vs host/path/TLS HTTP routing
NodePort / LoadBalancer	**Ingress**	one entry per service vs one shared entry routing many
iptables mode	**IPVS mode**	rule-chain (fine at small scale) vs hash table (better at large scale)
short name	**FQDN**	same-namespace only vs cross-namespace / pods

When NOT to: don't expect a ClusterIP to `ping` (no host owns it); don't use short DNS names across namespaces; don't deploy an Ingress *resource* without a *controller* (it does nothing); don't

overlap pod and service CIDRs.

One-sentence "why this over that":

> Let the CNI give pods flat IPs, kube-proxy give pod *sets* a stable VIP, CoreDNS give those VIPs *names*, and Ingress give the outside world one host/path door — four layers, each solving reachability at a different level.

> ✅ **Check yourself before Rung 7:** Explain why you'd choose one Ingress over ten `type: LoadBalancer` Services — name two concrete things Ingress gives you that per-service LBs don't.

RUNG 7 — The Prediction Test 🧪

Prediction 1 — Inspect the stack and name every layer

> **My prediction:** "If I read `/etc/cni/net.d`, kube-proxy's logs, and the apiserver manifest, I can name the CNI, proxy mode, pod CIDR, and service CIDR — and the two CIDRs won't overlap — *because* each layer records its config in a known place."

```
ls /etc/cni/net.d                                                  # CNI
kubectl logs -n kube-system -l app=flannel | grep -i ip-alloc      # pod CIDR
grep service-cluster-ip-range /etc/kubernetes/manifests/kube-apiserver.yaml   # service CIDR
kubectl logs -n kube-system <kube-proxy-pod> | grep -i proxier      # mode
```

Verify: you can state all four, non-overlapping. This *is* the standard "explore networking" lab.

Prediction 2 — Short DNS names fail across namespaces; FQDN fixes it

> **My prediction:** "If `web-app` in `default` looks up `mysql` (which lives in `payroll`), it's NXDOMAIN, but `mysql.payroll` resolves — *because* the pod's `search` domains only cover its own namespace, so crossing namespaces needs the qualified name."

```
kubectl exec web-app -- nslookup mysql            # NXDOMAIN
kubectl exec web-app -- nslookup mysql.payroll    # resolves
kubectl set env deployment/web-app DB_HOST=mysql.payroll
```

Verify: the qualified name resolves and the app connects. If *both* fail, CoreDNS itself is down (check its pods).

Prediction 3 — Ingress 404 until `rewrite-target`

> **My prediction:** "If I add a `/pay` path to a backend that only serves `/`, then `curl /pay` returns 404 until I add `rewrite-target: /` — *because* the controller forwards `/pay` verbatim, which the backend doesn't have."

```
kubectl create ingress ingress-pay -n critical-space --rule="/pay=pay-service:8282"
curl http://<ingress-ip>/pay                          # 404
kubectl logs -n critical-space <pay-pod>              # request arrived as /pay
kubectl annotate ingress ingress-pay -n critical-space \
  nginx.ingress.kubernetes.io/rewrite-target=/
curl http://<ingress-ip>/pay                          # 200
```

Verify: 404 → 200 after the annotation. The Ingress must be in the **same namespace** as `pay-service`.

Prediction: "If I do X, then Y, because [mechanism]"	Ran?	Right?	Miss?
1.			

🎁 CAPSTONE — Compress It

One sentence, no notes:

> The CNI gives every pod a flat NAT-free IP, kube-proxy turns each Service's virtual IP into iptables/IPVS DNAT to a live pod, CoreDNS maps names to those VIPs, and Ingress puts one L7 host/path door in front — four layers, each answering "how do I reach that?"

Explain it to a beginner in 3 sentences:

> 1. A network plugin (CNI) gives every pod its own IP and makes pods reachable across all nodes without NAT.
> 2. Because pod IPs change, a Service gives a set of pods one stable virtual IP that kube-proxy quietly rewrites to a real pod, and CoreDNS lets you use a *name* instead of that IP.
> 3. To let the outside world in, you deploy an Ingress controller and write rules that route by hostname and URL path to the right Service.

Which rung to revisit hands-on?

- **Rung 3B (kube-proxy DNAT)** — the "VIP that nothing listens on" is unintuitive. Fix: `iptables -t nat -L KUBE-SERVICES` and find your service.
- **Rung 3C (DNS search domains)** — the same-ns-vs-FQDN rule. Fix: Prediction 2, then `cat /etc/resolv.conf` and read the search list.

🎯 CKA exam tips & quick notes

- **Ports:** apiserver **6443**, kubelet **10250**, scheduler **10259**, controller-mgr **10257**, etcd **2379/2380**, NodePort **30000–32767**.
- **CNI:** binaries `/opt/cni/bin`, config `/etc/cni/net.d`; deploy with `kubectl apply -f`; `NotReady` → CNI missing; **pod CIDR ≠ service CIDR**.
- **kube-proxy** = DaemonSet; mode from its logs; DNATs VIP→pod via iptables/IPVS.
- **CoreDNS:** `kube-dns` ClusterIP `10.96.0.10`; Corefile = `coredns` ConfigMap; cross-ns = FQDN `<svc>.<ns>`.
- **Ingress:** deploy a **controller** first; `kubectl create ingress name --rule="host/path=svc:port"`; `pathType: Prefix`; `rewrite-target: /` fixes 404s; same-namespace as its Services.

- Debug from inside a pod with `nslookup` / `kubectl exec ... -- curl`.

📌 Command cheat sheet

```
# INSPECT
kubectl get nodes -o wide ; ip addr ; ip route
ls /opt/cni/bin ; ls /etc/cni/net.d
kubectl get pods -n kube-system -o wide          # coredns, kube-proxy, CNI DaemonSets
kubectl logs -n kube-system <kube-proxy-pod>     # proxy mode
# DNS
kubectl exec <pod> -- cat /etc/resolv.conf
kubectl exec <pod> -- nslookup <svc>.<ns>.svc.cluster.local
# INGRESS
kubectl create ingress web --rule="host.com/path*=svc:80"
kubectl describe ingress web
```

Related sections

- Section 1 — Core Concepts — Services (ClusterIP/NodePort/LoadBalancer) and kube-proxy's role.
- Section 6 — Security — NetworkPolicies enforced by the CNI; DNS for egress rules.
- Section 9 — Design & Install a Kubernetes Cluster — choosing pod/service CIDRs at install.
- Section 13 — Troubleshooting — DNS/CNI/service connectivity debugging.
- ../../Networking/24-kubernetes-pod-networking-cni.md · ../../Networking/25-kubernetes-services-kube-proxy.md · ../../Networking/26-kubernetes-dns-service-discovery.md · ../../Networking/27-kubernetes-ingress-gateway-api.md — each layer in depth.
- ../../Linux/12-iptables-netfilter.md — the iptables/netfilter kube-proxy programs.

✅ Answers — "Check yourself before Rung N"

Before Rung 2

Q: Give the single reason you can't just point your frontend at a backend pod's IP. What property of pods makes that fragile?

A: Pods are **ephemeral** — a pod's IP changes every time it restarts or reschedules, so any hard-coded pod IP breaks constantly. That churn is exactly why the stack exists: a Service gives the pod *set* one stable virtual IP, and CoreDNS gives that VIP a name, so the frontend addresses "backend" instead of a specific, short-lived 10.244.x.y address.

Before Rung 2→3

Q: Why does a Service's ClusterIP not respond to `ping`, yet `curl <clusterIP>:80` works? Which layer explains it?

A: Layer 2 — **kube-proxy** — explains it. A Service is a **virtual IP that no process listens on**; no host owns it, so ICMP `ping` gets no reply. kube-proxy programs iptables/IPVS **DNAT** rules for the Service's ports, so a TCP packet to `<clusterIP>:80` is rewritten in the kernel to a live pod IP and `curl` works — only the DNAT'd ports function, never the bare IP.

Before Rung 4

Q: For each layer name the component and where it lives: (1) gives a pod its IP, (2) DNATs a service VIP, (3) resolves `mysql`, (4) routes `example.com/pay`. Which two run as DaemonSets?

A: (1) The **CNI plugin** — binaries in `/opt/cni/bin/`, active config in `/etc/cni/net.d/`, with the addon (Flannel/Calico/Weave) running as a DaemonSet; the kubelet → runtime → CNI `ADD` call assigns the IP. (2) **kube-proxy**, a DaemonSet in `kube-system`, which writes iptables/IPVS DNAT rules on every node. (3) **CoreDNS**, pods in `kube-system` fronted by the `kube-dns` Service (ClusterIP `10.96.0.10`, injected into every pod's `/etc/resolv.conf`). (4) The **Ingress controller** (nginx/Traefik/ALB), proxy pods you deploy yourself, driven by Ingress resources. The two DaemonSets are the **CNI addon** and **kube-proxy**.

Before Rung 5

Q: Which layer would you suspect for each: `NotReady` node; ClusterIP reachable but `nslookup mysql` fails; external `/pay` returns 404; two pods on different nodes can't ping each other?

A: `NotReady` node → **Layer 1, CNI** — a node NotReady right after install usually means no CNI addon is applied. ClusterIP works but `nslookup mysql` fails → **Layer 3, CoreDNS** — the VIP and DNAT are fine, name resolution isn't (check CoreDNS pods, or use the FQDN across namespaces). External `/pay` 404 → **Layer 4, Ingress** — a missing/wrong rule or a missing `rewrite-target: /` annotation. Cross-node pod-to-pod failure → **Layer 1, CNI** — the flat NAT-free pod network isn't routing between nodes.

Before Rung 6

Q: In the trace, which layer made `wear-service` a *name* you could use, and which one actually forwarded the packet to a specific pod? Why are those two different components?

A: CoreDNS (Layer 3) made `wear-service` a usable name — it resolves the name to the Service's ClusterIP via the pod's `/etc/resolv.conf`. **kube-proxy (Layer 2)** actually forwarded the packet — its iptables `KUBE-SERVICES` DNAT rule rewrote the VIP to one live pod IP, which the CNI then routed. They're different components because they solve different problems: DNS is a one-time name-to-VIP lookup done in userspace, while forwarding must happen in the kernel on **every packet** and track which backends are currently alive — CoreDNS knows names, kube-proxy knows endpoints.

Before Rung 7

Q: Why choose one Ingress over ten `type: LoadBalancer` Services — name two concrete things Ingress gives you that per-service LBs don't.

A: Ten LoadBalancer Services mean ten cloud load balancers and ten bills, each a dumb L4 entry for one service. One Ingress gives you (1) **host/path L7 routing** — a single entrypoint that routes `example.com/wear` and `example.com/watch` to different backend Services — and (2) **one shared front door**, meaning a single external IP/LB (one bill) and shared TLS termination at that edge. The trade-off from the ladder: you must deploy an Ingress **controller** first, or the resource does nothing.

Design & HA, Climbed the Ladder 🪜

Section 9 of the CKA — deriving why a cluster needs an *odd* number of brains

> Designing a cluster and making it **highly available**: multiple control-plane nodes, leader election, and **etcd quorum**. Mostly conceptual, but the quorum math is testable. We climb from **the pain of a single point of failure** → **the "no SPOF, but consensus needs a majority" idea** → **the machinery** → then checks as predictions. Each rung ends with a ✅ **Check yourself**.

RUNG 0 — The Setup 🎯

What am I learning? How to choose a cluster build, and how to remove single points of failure from the control plane — including the counter-intuitive etcd quorum rule.

Why did it land on my desk? *Cluster Architecture* is 25% of the CKA. You won't be asked to memorize scale ceilings, but **HA reasoning and etcd quorum** show up, and you must know why "just add a second master" isn't enough.

What do I already know? Maybe that "prod needs more than one master." What's fuzzy: why apiservers all run at once but only one scheduler acts, and why a **2-node etcd is no safer than 1**.

RUNG 1 — The Pain 🔥

Why does HA design exist at all?

A single control-plane node is a single point of failure:

```
THE SINGLE-CONTROL-PLANE PAIN
  control-plane node dies ─▶ running pods keep serving, BUT:
                             • no new scheduling, no self-healing, no kubectl
                             • etcd is gone → the cluster's entire memory is unavailable
  "just add a 2nd etcd"   ─▶ surprise: a 2-node etcd tolerates ZERO failures (quorum trap)
  network partition       ─▶ an even cluster can split so NEITHER half has a majority → total stop
```

Before / without it: one master, one etcd — a single reboot or disk failure takes down cluster management, and a naive "add one more" for etcd doesn't actually buy fault tolerance.

What breaks without it: manageability (schedule/heal/kubectl all depend on the control plane) and — because etcd is a *consensus* store — availability the moment you can't reach a majority of etcd members.

Who feels it most? The platform team running production — a control-plane outage means nothing new deploys and nothing self-heals until it's back.

> ✅ **Check yourself before Rung 2:** If the single control-plane node dies, what *keeps* working and what *stops*? (Hint: think running pods vs. new decisions.)

RUNG 2 — The One Idea 💡

The single sentence everything hangs off

> **High availability means no single point of failure: replicate the control plane behind a load balancer — but because etcd is a consensus database that only commits a write when a *majority* (quorum) of members agree, you need an *odd* number of etcd members, at least 3, or you gain no real fault tolerance.**

Derivations:

- *"replicate the control plane"* → multiple control-plane nodes; but each component behaves differently: **apiserver** is stateless (**active-active** behind an LB) while **scheduler/controller-manager** would collide if doubled (**active-standby** via **leader election**).
- *"etcd… majority… quorum"* → **quorum = ⌊N/2⌋+1**, **fault tolerance = N − quorum**. This is *why* 2 nodes tolerate 0 failures (quorum is still 2) and why **odd is better** (an even split leaves neither half with a majority).
- *"at least 3"* → 3 tolerates 1 failure — the minimum real HA; 5 tolerates 2; beyond that rarely pays.

The whole section is: *replicate everything, but respect that a consensus store needs a majority alive.*

> ✅ **Check yourself before Rung 3:** Why is a 2-member etcd cluster no more fault-tolerant than a 1-member one? State the quorum for each.

RUNG 3 — The Machinery ⚙️

How it ACTUALLY works — go slow

> ### 🧸 Plain-English first (read this before the technical version)
>
> This section is about designing a cluster's "management office" so no single failure shuts the whole operation down — and one surprising piece of math about voting.
>
> **(A) Picking the right size of setup.** Like choosing office space: a home desk for learning (a tiny one-computer setup), a small rented office you furnish yourself for testing, and for real business either a professionally built multi-office headquarters or renting a fully serviced building from a big provider (a "managed" cluster, where a cloud company runs the management for you). One house rule: the management floor is kept clear of regular work — everyday workloads are deliberately kept off the managers' computers so the managers are never crowded out.
>
> **(B) Duplicating the managers — two different styles.** The management team has three kinds of roles, and you can't duplicate them all the same way:
>
> - The **front desk** (the "API server," which takes every request) keeps no personal notes, so you can simply staff several receptionists at once, with a greeter at the entrance (a "load balancer") sending each visitor to whichever desk is free. All work simultaneously.
> - The **decision-makers** (the "scheduler," which assigns work to computers, and the "controller manager," which fixes things that break) must NOT act in duplicate — two people assigning the same task twice causes chaos. So the copies play understudy: only the one holding a signed permission slip (a "lease" — a short-term lock that must be renewed every few seconds) acts; if the holder goes silent, an understudy picks up the slip within seconds and takes over.
>
> **(C) The record vault and the voting rule.** The cluster's entire memory lives in a record-keeping committee ("etcd"). You can house the record-keepers with the managers ("stacked" — cheaper, but losing one machine loses both a manager and a record-keeper) or in their own separate building ("external" — sturdier, more machines). The committee only accepts a change when a **majority** of members vote yes (that's "quorum"). The math: majority = half the members, rounded down, plus one. This creates the famous trap — a 2-person committee needs both votes, so losing one person halts everything: two is no safer than one! Three members can lose one and keep going; five can lose two. And **odd numbers beat even**: if a phone outage splits an even committee exactly in half, neither half has a majority and all record-keeping stops; an odd committee always leaves one side able to vote.

Now the original technical deep-dive — the same ideas, in precise form:

(A) Design choices

Goal	Recommended
Learning	**minikube** (single node, provisions the VM) or kubeadm single-node
Dev/test	**kubeadm** multi-node (you provision VMs)
Production	**HA multi-master** (kubeadm), or **managed** (EKS/GKE/AKS), or turnkey (kOps)

- **minikube** *creates* the VM (single node); **kubeadm** expects VMs but supports **multi-node**.
- Scale ceilings (in the docs, don't memorize): ~5000 nodes, ~150k pods, ~110 pods/node.
- **Dedicate control-plane nodes** — kubeadm taints them `node-role.kubernetes.io/control-plane:NoSchedule` so workloads stay off.

(B) HA control plane — two behaviors

```
                ┌──────── Load Balancer (:6443) ────────┐   ← kubectl / kubelets point HERE
                ▼                                       ▼
      ┌──── control-plane-1 ────┐           ┌──── control-plane-2 ────┐
      │ kube-apiserver  ACTIVE  │           │ kube-apiserver  ACTIVE  │  ← active-ACTIVE (stateless)
      │ scheduler       ACTIVE  │           │ scheduler       standby │  ← active-STANDBY (leader
elect)
      │ controller-mgr  ACTIVE  │           │ controller-mgr  standby │  ← active-STANDBY
      │ etcd (member)           │           │ etcd (member)           │  ← distributed (Raft)
      └─────────────────────────┘           └─────────────────────────┘
```

- **kube-apiserver:** stateless → **active-active**. Put an LB (nginx/HAProxy) on `:6443`; point kubectl/kubelets at it.
- **scheduler & controller-manager:** two acting at once would double-schedule → **active-standby** via **leader election**. Whoever holds the **Lease** lock is active; if it dies, the standby takes over within ~the lease duration.

```
--leader-elect=true                       # default
--leader-elect-lease-duration=15s         # hold the lock this long
--leader-elect-renew-deadline=10s         # active renews every 10s
--leader-elect-retry-period=2s            # standby retries every 2s
```

(C) etcd quorum — the consensus math

Two topologies: stacked (etcd on the control-plane nodes — fewer servers; losing a node loses a member *and* a control-plane) vs **external etcd** (separate servers — more resilient, ~2× the boxes). Either way, only the **apiserver** talks to etcd (`--etcd-servers=<list>`).

etcd replicates via **Raft**: writes go through an elected leader and **commit only when a majority confirm**.

Quorum = ⌊N/2⌋ + 1. Fault tolerance = N − quorum.

Nodes (N)	Quorum	Can lose
1	1	0
2	**2**	**0** ← no better than 1!
3	**2**	**1**
4	3	1
5	**3**	**2**
7	**4**	**3**

Odd numbers are preferred: a partition of an even cluster can leave *both* halves short of quorum (6-node split 3/3 → neither has 4 → whole cluster stops). An odd cluster always keeps one side ≥ quorum. **Minimum HA = 3.** etcd uses **2379** (client) / **2380** (peer).

> 🎯 Remember **quorum = N/2 + 1** and that **2 nodes ≠ HA**. Prefer odd counts.

> ✅ **Check yourself before Rung 4:** For a 5-node etcd cluster: what's the quorum, and how many members can fail before writes stop? Why would 6 nodes be a *worse* choice than 5 in some partition scenarios?

RUNG 4 — The Vocabulary Map 🏷️

Term	What it actually is	Which part
SPOF	Single point of failure	The problem
active-active	All instances serve at once	apiserver
active-standby	One acts, others wait	scheduler / controller-mgr
leader election / Lease	The lock deciding who's active	scheduler / controller-mgr
load balancer (:6443)	Front door spreading apiserver traffic	Control-plane HA
stacked etcd	etcd co-located on control-plane nodes	etcd topology
external etcd	etcd on dedicated servers	etcd topology
Raft	The consensus algorithm etcd uses	etcd
quorum / majority	⌊N/2⌋+1 members needed to commit	etcd availability
fault tolerance	N − quorum members you can lose	etcd availability
split-brain	Partition where neither side has quorum	Why odd wins

The unlock — three behaviors under replication:

```
STATELESS (just add copies + LB):   kube-apiserver → active-active
SINGLETON (must not double-run):     scheduler, controller-manager → active-standby (lease)
CONSENSUS (needs a majority alive):  etcd → odd count, quorum = N/2+1
```

> ✅ **Check yourself before Rung 5:** Sort into stateless / singleton / consensus: kube-apiserver, etcd, kube-controller-manager. Which one dictates you use an *odd* number of nodes?

RUNG 5 — The Trace 🎬

Follow ONE node failure through a 3-node HA cluster

Trace — control-plane-2 dies (of 3 HA nodes):

1. **apiserver:** the LB health-check fails for CP-2, so it stops routing there; CP-1 and CP-3 apiservers keep serving `:6443`. `kubectl` never notices. ✅ (active-active)
2. **scheduler / controller-manager:** if CP-2 held the **Lease**, it stops renewing; after ~lease-duration the standby on CP-1 or CP-3 grabs the lease and becomes active. Scheduling/reconciliation resume within seconds. ✅ (active-standby)
3. **etcd:** the cluster had 3 members; now 2 remain. Quorum = ⌊3/2⌋+1 = **2**, and 2 ≥ 2 → **writes still commit**. The cluster stays fully operational. ✅

4. You repair/replace CP-2; it rejoins etcd and re-registers. Back to 3, tolerance 1 again.

Contrast — the same failure in a 2-node etcd: quorum = 2; losing one leaves 1 < 2 → **writes stop cluster-wide**. That's why 2 nodes "isn't HA."

```
3-node HA, lose 1:  apiserver→LB reroutes ✓ · scheduler→standby takes lease ✓ · etcd 2/3 ≥ quorum(2) ✓  → cluster OK
2-node etcd, lose 1: etcd 1/2 < quorum(2) ✗  → writes halt
```

> ✅ **Check yourself before Rung 6:** In the 3-node trace, which component recovered by *rerouting*, which by *taking over a lease*, and which by *still having a majority*?

RUNG 6 — The Contrast ⚖️

This…	vs that…	Difference
Single control plane	**HA multi-master**	one SPOF vs redundant, LB-fronted
active-active (apiserver)	**active-standby (scheduler/ctrl-mgr)**	all serve vs one acts (avoid double-action)
Stacked etcd	**External etcd**	fewer servers, coupled failure vs more servers, isolated
Odd node count	**Even node count**	always a majority side vs risk of no-majority split
3 members	**5 members**	tolerate 1 vs tolerate 2 (more write latency)

When NOT to: don't build HA for a learning/dev cluster (needless cost/complexity — minikube is fine); don't run **2** control-plane/etcd nodes thinking it's safer than 1; don't scale etcd past 5 without reason (every write waits on a bigger majority → slower).

One-sentence "why this over that":

> Front stateless apiservers with a load balancer, let scheduler/controller-manager fail over via leader election, and size etcd to an odd count ≥ 3 so a majority survives a failure — 2 buys you nothing.

> ✅ **Check yourself before Rung 7:** A colleague proposes a 4-node etcd cluster "for extra safety." Explain why it tolerates the same number of failures as 3 while costing more.

RUNG 7 — The Prediction Test 🧪

Prediction 1 — Leader election means exactly one active holder

> **My prediction:** "If I check the scheduler/controller-manager manifests and the Leases, I'll see `--leader-elect=true` and a single current holder — *because* only one of each may act at a time."

```
grep -- '--leader-elect' /etc/kubernetes/manifests/kube-scheduler.yaml
grep -- '--leader-elect' /etc/kubernetes/manifests/kube-controller-manager.yaml
kubectl get lease -n kube-system         # holderIdentity = the active instance
```

Verify: `=true`, and each Lease names one holder. On multi-master, that holder is on one specific node.

Prediction 2 — etcd reports its members and one leader

> **My prediction:** "If I query etcd membership, a single-node lab shows one member that IS LEADER; an HA cluster shows N members with exactly one leader — *because* Raft elects a single leader for writes."

```
ETCDCTL_API=3 etcdctl member list \
  --endpoints=https://127.0.0.1:2379 --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/kubernetes/pki/etcd/server.key
ETCDCTL_API=3 etcdctl endpoint status --write-out=table --endpoints=... --cacert=... --cert=... --
key=...
```

Verify: the status table has exactly one `IS LEADER=true`.

Prediction 3 — Compute fault tolerance from N

> **My prediction:** "For N members, tolerance = N − (⌊N/2⌋+1). So 3→1, 4→1, 5→2, 7→3 — *because* you must keep a majority."

```
N=3: quorum 2, lose 1     N=4: quorum 3, lose 1 (same as 3!)
N=5: quorum 3, lose 2     N=7: quorum 4, lose 3
```

Verify: 4 tolerates the same as 3 — the concrete reason to pick odd. Say it back for 5 and 7 without the table.

Prediction: "If I do X, then Y, because [mechanism]"	Ran?	Right?	Miss?
1.			

🎁 CAPSTONE — Compress It

One sentence, no notes:

> HA removes single points of failure by running apiservers active-active behind a load balancer and scheduler/controller-manager active-standby via leader election, while etcd — a consensus store — needs an odd number of members (≥3) because it only commits when a majority agree.

Explain it to a beginner in 3 sentences:

> 1. One control-plane node is a single point of failure, so production runs several, with a load balancer in front of the API servers.
> 2. The API servers all work at once, but only one scheduler and one controller-manager act at a time (the others wait to take over).
> 3. etcd stores everything and only accepts writes when more than half its members agree, so you run an odd number — at least three — because two would stop the moment one fails.

Which rung to revisit hands-on? Rung 3C + Prediction 3 — the **quorum math** and the **2-node/even-node trap** are the memorable, testable core of this section.

🎯 CKA exam tips & quick notes

- **apiserver = active-active** behind an LB (`:6443`); **scheduler & controller-manager = active-standby** via `--leader-elect`.
- **etcd quorum = N/2 + 1**, **fault tolerance = N − quorum**, **odd preferred**, **min 3** (2 gives nothing).
- **Stacked** (etcd on control plane) vs **external etcd**; apiserver reaches etcd via `--etcd-servers`.
- kubeadm taints control-plane nodes to keep workloads off (removable).
- Don't memorize scale ceilings; **do** internalize the quorum math.

📌 Command cheat sheet

```
grep -- '--leader-elect' /etc/kubernetes/manifests/kube-scheduler.yaml
kubectl get lease -n kube-system                       # who holds the lease
ETCDCTL_API=3 etcdctl member list --endpoints=... --cacert=... --cert=... --key=...
ETCDCTL_API=3 etcdctl endpoint status --write-out=table --endpoints=... ...
# quorum = floor(N/2)+1 ;  tolerance = N - quorum ;  prefer ODD, min 3
```

Related sections

- Section 10 — Install Kubernetes the Kubeadm Way — bootstrapping the cluster you designed.
- Section 5 — Cluster Maintenance — etcd backup/restore + upgrades on this design.
- Section 1 — Core Concepts — the control-plane components you're making HA.
- Section 6 — Security — the PKI each etcd + apiserver instance needs.
- ../../Networking/18-load-balancing.md — the load balancer fronting the apiservers.

✅ Answers — "Check yourself before Rung N"

Before Rung 2

Q: If the single control-plane node dies, what *keeps* working and what *stops*?

A: Running pods **keep serving** — the workloads on worker nodes don't depend on the control plane moment to moment. What **stops** is everything requiring new decisions: no new scheduling, no self-healing (a crashed pod won't be replaced), and no `kubectl` — the API server is down. And because etcd lived on that node, the cluster's entire memory (its state store) is unavailable until the control plane is back.

Before Rung 3

Q: Why is a 2-member etcd cluster no more fault-tolerant than a 1-member one? State the quorum for each.

A: Quorum = ⌊N/2⌋+1. For 1 member, quorum = 1; for 2 members, quorum = ⌊2/2⌋+1 = **2** — both members must be alive to commit a write. So losing one member of a 2-node cluster leaves 1 < 2 and writes stop cluster-wide, exactly like losing the only member of a 1-node cluster: fault tolerance is **0** in both cases. Adding the second member bought no real fault tolerance — the minimum for real HA is 3 (quorum 2, tolerates 1).

Before Rung 4

Q: For a 5-node etcd cluster: what's the quorum, and how many members can fail before writes stop? Why would 6 nodes be a *worse* choice than 5 in some partition scenarios?

A: For N=5, quorum = ⌊5/2⌋+1 = **3**, so fault tolerance = 5 − 3 = **2** — writes continue until a third member fails. 6 nodes (quorum 4) still tolerates only 2 failures, and it adds a partition risk odd counts avoid: a 6-node cluster can split **3/3**, leaving *neither* half with the 4 needed for quorum, so the whole cluster stops. An odd cluster always leaves one side with a majority, which is why odd is preferred.

Before Rung 5

Q: Sort into stateless / singleton / consensus: kube-apiserver, etcd, kube-controller-manager. Which one dictates you use an *odd* number of nodes?

A: kube-apiserver → stateless: just add copies behind a load balancer, active-active. **kube-controller-manager → singleton**: it must not double-run, so it's active-standby via leader election on a Lease (the scheduler behaves the same way). **etcd → consensus**: it needs a majority of members alive to commit writes. It's **etcd** that dictates the odd node count — quorum = ⌊N/2⌋+1 means even counts add no tolerance and risk a no-majority split.

Before Rung 6

Q: In the 3-node trace, which component recovered by *rerouting*, which by *taking over a lease*, and which by *still having a majority*?

A: The **kube-apiserver** recovered by rerouting — the load balancer's health check dropped CP-2 and sent traffic to the remaining active-active apiservers, so kubectl never noticed. The **scheduler and controller-manager** recovered by lease takeover — the dead node stopped renewing the Lease, and after ~the lease duration a standby grabbed it and became active. **etcd** recovered by still having a majority — 2 of 3 members remained, and 2 ≥ quorum (⌊3/2⌋+1 = 2), so writes kept committing.

Before Rung 7

Q: A colleague proposes a 4-node etcd cluster "for extra safety." Explain why it tolerates the same number of failures as 3 while costing more.

A: For N=4, quorum = ⌊4/2⌋+1 = **3**, so fault tolerance = 4 − 3 = **1** — exactly the same one-failure tolerance as a 3-node cluster (quorum 2, lose 1). The fourth node therefore adds cost, an extra server every write must involve, and a new risk: a 2/2 partition leaves neither half with 3 members, so the whole cluster stops — a split an odd cluster can't suffer. For "extra safety," the right move is to go to **5** (quorum 3, tolerates 2), not 4.

Kubeadm Install, Climbed the Ladder 🪜

Section 10 of the CKA — deriving how prepared VMs become a cluster

> Bootstrapping a real multi-node cluster with **kubeadm**: prerequisites (runtime, cgroups, sysctls), `kubeadm init`, deploying a CNI, joining workers. We climb from **the pain of installing Kubernetes by hand** → **the "two commands over prepared nodes" idea** → **the machinery** → then the sequence as predictions. Each rung ends with a ✅ **Check yourself**.

RUNG 0 — The Setup 🎯

What am I learning? The exact, ordered procedure to turn bare Linux VMs into a working cluster: prepare every node, `kubeadm init` the control plane, deploy a pod network, and `kubeadm join` the workers.

Why did it land on my desk? *Installation & Configuration* is part of the 25% Cluster-Architecture domain, and building or **joining a node** to a cluster can be a task. It's also the foundation for the upgrade tasks (Section 5).

What do I already know? Maybe that `kubeadm init` "sets up the master." What's fuzzy: *why* you must fix the cgroup driver and sysctls first, and why a freshly-initialized node is `NotReady`.

RUNG 1 — The Pain 🔥

Why does kubeadm exist at all?

Installing Kubernetes "the hard way" means assembling a dozen moving parts by hand:

```
INSTALLING KUBERNETES BY HAND (the pain)
  generate a CA + ~10 certs (apiserver, etcd, kubelet, client…) → one wrong SAN = broken TLS
  write systemd units / static-pod manifests for apiserver, etcd, scheduler, controller-mgr
  wire every component's flags to the right cert paths and etcd endpoints
  bootstrap each worker's kubelet with a signed cert, by hand
  → days of work, a hundred ways to typo it
```

Before / without it: you did all of the above manually (Kelsey Hightower's "Kubernetes the Hard Way") — educational, but unthinkable for real setup.

What breaks without a bootstrapper: correctness (hand-made PKI and manifests are error-prone) and speed (a cluster should be minutes, not days). And nodes must be *prepared* (runtime, cgroups, kernel settings) or the kubelet won't run pods at all.

Who feels it most? Anyone standing up a non-managed cluster — the platform team on-prem or in raw VMs.

> ✅ **Check yourself before Rung 2:** Name two things `kubeadm init` generates for you that would be tedious and error-prone to build by hand. (Hint: think TLS, and think control-plane process definitions.)

RUNG 2 — The One Idea 💡

The single sentence everything hangs off

> **kubeadm turns prepared Linux VMs into a cluster with two commands — `kubeadm init` generates all the PKI and writes the control plane as static pods on the first node, then `kubeadm join` uses a token + CA-cert hash to securely enroll each worker — but only after you've fixed the container runtime, cgroup driver, sysctls, and swap on *every* node.**

Derivations:

- *"only after you've fixed runtime/cgroup/sysctls/swap"* → these are the **preflight** requirements; miss one and `init`/pods fail. The kubelet refuses swap; CNIs need bridge/forwarding sysctls; the runtime's cgroup driver **must match** the kubelet's.
- *"init… writes the control plane as static pods"* → the apiserver/etcd/scheduler/controller-manager land in `/etc/kubernetes/manifests/` — the same static pods you saw in Section 1.
- *"join… token + CA hash"* → a worker proves it's talking to the *real* control plane (CA hash) and is *authorized* to join (token); tokens expire in 24h.
- *(implicit)* the cluster is `NotReady` until you **deploy a CNI** — pods can't get IPs without one.

> ✅ **Check yourself before Rung 3:** Why must the container runtime's cgroup driver match the kubelet's? What kind of failure do you get if they disagree?

RUNG 3 — The Machinery ⚙️

How it ACTUALLY works — go slow. Order matters.

🧸 Plain-English first (read this before the technical version)

Imagine opening a chain of franchise restaurants from a startup kit. You can't just unlock the doors and start cooking — there's a strict order: get every building up to code, set up the headquarters, install the phone lines, then enroll the branch locations. That's exactly what this section walks through, in four steps.

- **Step A — Prepare every building first.** Before anything else, every computer that will be part of the cluster (the group of machines working together) needs some housekeeping:
  - Flip a few switches in the operating system so network traffic between apps can flow and be inspected properly (like making sure the building's plumbing and wiring meet code).
  - Turn off "swap" (a trick where the computer uses slow disk space as pretend memory) — Kubernetes' on-site manager program, the **kubelet**, refuses to work if it's on, because it makes performance unpredictable.
  - Install the "kitchen equipment": the **container runtime** (the software that actually runs your apps in their sealed boxes). One gotcha: the runtime and the kubelet must agree on the same accounting system for resources (the "cgroup driver") — like two managers who must use the same ledger, or the books never balance and everything crashes.
  - Install the toolkit and **pin the versions** so an automatic update doesn't quietly swap your tools mid-project.
- **Step B — Set up headquarters (`kubeadm init` , run on one machine only).** One command builds the entire head office: it prints all the security ID badges and certificates (so every part can prove who it is), writes the instruction sheets that start the management programs, and hands you two things — a login file so you can talk to the cluster, and an invitation command for the branches. Oddly, HQ reports itself as "NotReady" at first — that's expected, not broken.
- **Step C — Install the phone system (the CNI, or pod network).** The machines can't pass messages between apps until you install a networking add-on. One small agent runs on every machine (like putting one telephone technician in each building), and it hands out internal phone numbers (IP addresses) to the apps. Once installed, "NotReady" flips to "Ready."
- **Step D — Enroll the branches (`kubeadm join`).** Each worker machine joins using two secrets: a **token** (a temporary invitation code that expires in 24 hours — you can print a fresh one anytime) and a **certificate hash** (a fingerprint that lets the branch verify it's really talking to the genuine HQ, not an impostor). The branch then gets its own signed ID badge and is registered as part of the chain.

The one rule above all: **order matters.** Skip a preparation step and the later steps fail in confusing ways.

Now the original technical deep-dive — the same ideas, in precise form:

The workflow: provision VMs → **runtime** on all nodes → **kubeadm/kubelet/kubectl** on all nodes → `kubeadm init` on control plane → **deploy CNI** → `kubeadm join` workers.

(A) Prerequisites — on EVERY node (and *why* each)

```
# 1) Kernel modules + sysctls: CNIs/kube-proxy need bridged traffic to hit iptables + IP forwarding
sudo modprobe overlay && sudo modprobe br_netfilter
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF
sudo sysctl --system
# 2) Swap OFF — the kubelet refuses to run with swap by default
sudo swapoff -a
```

Container runtime (containerd) — the cgroup-driver gotcha:

```
sudo apt-get install -y containerd
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml >/dev/null
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml   # MATCH the kubelet
sudo systemctl restart containerd
```

> 🎯 The **kubelet and runtime must use the SAME cgroup driver** (systemd on a systemd host). kubeadm ≥1.22 defaults the *kubelet* to systemd, but you must set `SystemdCgroup = true` in containerd yourself.

Install the tools (pinned + held):

```
sudo apt-get install -y kubelet=1.31.1-1.1 kubeadm=1.31.1-1.1 kubectl=1.31.1-1.1
sudo apt-mark hold kubelet kubeadm kubectl    # pin so a stray upgrade doesn't drift versions
```

(B) `kubeadm init` — control-plane node only

```
sudo kubeadm init \
  --apiserver-advertise-address=192.168.56.11 \   # the RIGHT NIC's IP
  --pod-network-cidr=10.244.0.0/16                 # must match your CNI
  # --control-plane-endpoint=<LB>   ← add if you MIGHT go HA later
```

Under the hood it: generates the **CA + all component certs** (`/etc/kubernetes/pki`), writes the **static-pod manifests**, starts the control plane, and prints (1) the **kubeconfig setup** and (2) the `kubeadm join` command.

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
kubectl get nodes        # control plane = NotReady  (expected until CNI)
```

(C) Deploy a CNI

```
kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
kubectl get nodes        # flips to Ready
```

The addon runs as a **DaemonSet** (one agent per node) and owns the **pod CIDR** (match `--pod-network-cidr`).

(D) Join the workers

```
# control plane: (re)generate the join command anytime — tokens last 24h
kubeadm token create --print-join-command
# worker (as root):
sudo kubeadm join 192.168.56.11:6443 --token <t> --discovery-token-ca-cert-hash sha256:<hash>
```

The **token** authorizes the join; the **CA-cert hash** lets the worker verify it's the real control plane (not a MITM). The worker's kubelet does a TLS bootstrap, gets a signed cert, and registers the node.

> ✅ **Check yourself before Rung 4:** After `kubeadm init` succeeds, `kubectl get nodes` shows `NotReady`. Is that a bug? What single action fixes it, and why was the node not Ready before?

RUNG 4 — The Vocabulary Map 🏷️

Term	What it actually is	Which step
br_netfilter / ip_forward	Kernel module/sysctl so bridged pod traffic hits iptables + is forwarded	Prereqs
swapoff	Disable swap (kubelet requirement)	Prereqs
cgroup driver / SystemdCgroup	How the runtime + kubelet account resources; must match	Prereqs
kubeadm / kubelet / kubectl	Bootstrapper / node agent / CLI	Prereqs
apt-mark hold	Pin versions against upgrades	Prereqs
kubeadm init	Bootstraps the control plane	Init
--apiserver-advertise-address	The control-plane IP to advertise	Init flag
--pod-network-cidr	Pod IP range (match the CNI)	Init flag
/etc/kubernetes/pki	Generated CA + certs	Init output
admin.conf	The admin kubeconfig	Init output
CNI addon	DaemonSet giving pods IPs	CNI
kubeadm join	Enroll a worker	Join
token / discovery-token-ca-cert-hash	Authorize join / verify the control plane	Join
kubeadm reset	Undo a join/init on a node	Recovery

The unlock — three phases:

```
PREPARE every node:  runtime + cgroup driver + sysctls + swapoff + pinned tools
INIT one node:       kubeadm init → PKI + static-pod manifests + kubeconfig + join cmd
JOIN + NETWORK:      deploy CNI (→ Ready) ; kubeadm join workers (token + CA hash)
```

> ✅ **Check yourself before Rung 5:** Which command runs on the control plane and which on a worker: `kubeadm init`, `kubeadm join`? What does the token protect against, and what does the CA-cert hash protect against?

RUNG 5 — The Trace 🎬

Follow the bring-up end-to-end

Trace — `kubeadm init`, then a worker joins:

1. **Preflight:** kubeadm checks swap off, runtime up, cgroup driver, ports free. Any failure aborts with a clear message.
2. **PKI:** it generates the **cluster CA** and every component cert (apiserver serving/SANs, etcd, kubelet client…) under `/etc/kubernetes/pki`.
3. **Manifests:** it writes static-pod YAML for apiserver/etcd/scheduler/controller-manager to `/etc/kubernetes/manifests/`; the local **kubelet** sees them and starts the control plane.
4. **Bootstrap:** once the apiserver is up, kubeadm creates a **bootstrap token** and prints the `join` command + the kubeconfig setup. You copy `admin.conf` to `~/.kube/config`.
5. **NotReady:** `kubectl get nodes` shows the node `NotReady` — no CNI yet, so pods can't get IPs.
6. **CNI:** `kubectl apply -f <addon>` deploys a DaemonSet; the node flips **Ready**.
7. **Worker join:** on the worker, `kubeadm join` contacts `:6443`, **verifies the CA hash** (real control plane?), presents the **token** (authorized?), TLS-bootstraps the kubelet (gets a signed cert), and registers the node — `NotReady` until the CNI DaemonSet schedules a pod there, then **Ready**.

```
init: preflight ✓ → CA+certs → static-pod manifests → kubelet starts control plane → token+join
printed
      → NotReady → apply CNI → Ready
join: worker → :6443 (verify CA hash) → present token → kubelet TLS-bootstrap → node registers →
Ready
```

> ✅ **Check yourself before Rung 6:** At which step do the control-plane *certificates* get created, and at which step does a *worker* first prove it's contacting the genuine control plane?

RUNG 6 — The Contrast ⚖️

This…	vs that…	Difference
kubeadm	**minikube**	you provide VMs, multi-node vs it makes one VM, single-node
kubeadm	**"the hard way"**	automates PKI + manifests vs everything by hand
kubeadm	**managed (EKS/GKE)**	you run the control plane vs the cloud runs it
`kubeadm init`	**`kubeadm join`**	create the control plane vs enroll a worker
before CNI	**after CNI**	node `NotReady` (no pod IPs) vs `Ready`

When NOT to: don't reach for kubeadm to *learn basic kubectl* (minikube is faster); don't skip the prereqs (swap/runtime/cgroup/sysctls) — they're the top cause of `init` failures; don't reuse a >24h-old token (regenerate it).

One-sentence "why this over that":

> Use kubeadm to bootstrap a real multi-node cluster you control — it automates the PKI and static-pod manifests — after you've prepped every node's runtime, cgroup driver, sysctls, and swap.

> ✅ **Check yourself before Rung 7:** A worker's `kubeadm join` fails with "token expired." Why do tokens expire, and what's the one-line fix on the control plane?

RUNG 7 — The Prediction Test 🧪

Prediction 1 — Full bring-up: init → CNI → join

> **My prediction:** "If I `kubeadm init` then apply a CNI then join a worker, then both nodes go `Ready` — *because* init creates the control plane, the CNI gives pods IPs (flipping NotReady→Ready), and join enrolls the worker."

```
# control plane:
sudo kubeadm init --apiserver-advertise-address=$(ip -4 -o addr show eth0 | awk '{print $4}' | cut -
d/ -f1) \
  --pod-network-cidr=10.244.0.0/16
mkdir -p ~/.kube && sudo cp /etc/kubernetes/admin.conf ~/.kube/config && sudo chown $(id -u):$(id -g)
~/.kube/config
kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
kubeadm token create --print-join-command      # copy → run on the worker with sudo
kubectl get nodes                              # both Ready
```

Verify: both `Ready`, CNI DaemonSet has a pod per node. `NotReady` before the CNI is normal.

Prediction 2 — A missed prereq blocks `init`

> **My prediction:** "If swap is on, the runtime is down, `SystemdCgroup` is false, or a sysctl is missing, then `kubeadm init` fails preflight with a specific message — *because* the kubelet/runtime can't operate under those conditions."

```
sudo kubeadm init ...            # read the preflight error
free -h                          # swap on? → sudo swapoff -a
sudo systemctl status containerd # down? → start/restart
grep SystemdCgroup /etc/containerd/config.toml   # false? → set true + restart
sysctl net.ipv4.ip_forward       # 0? → apply /etc/sysctl.d/k8s.conf
```

Verify: fixing the flagged prereq lets `init` complete. These four are the classic blockers.

Prediction 3 — An expired token needs regenerating (and reset before re-join)

> **My prediction:** "If a node's join token expired (24h), a fresh `kubeadm token create --print-join-command` works; a previously-broken node must `kubeadm reset` first — *because* tokens are short-lived and a half-joined node has stale state."

```
# control plane:
kubeadm token create --print-join-command
# worker (if previously joined/broken):
sudo kubeadm reset -f
sudo kubeadm join 192.168.56.11:6443 --token <new> --discovery-token-ca-cert-hash sha256:<hash>
```

Verify: the node re-appears `Ready`. Reusing an old token fails; regenerate rather than reuse.

Prediction: "If I do X, then Y, because [mechanism]"	Ran?	Right?	Miss?
1.			

🎁 CAPSTONE — Compress It

One sentence, no notes:

> After preparing every node's runtime, cgroup driver, sysctls, and swap, `kubeadm init` generates the PKI and writes the control plane as static pods, a CNI addon makes the node Ready, and `kubeadm join` enrolls workers with a token + CA-cert hash.

Explain it to a beginner in 3 sentences:

> 1. First you prep each machine — install a container runtime with the matching cgroup driver, turn off swap, and set a couple of kernel networking options — or the kubelet won't run pods.
> 2. `kubeadm init` on the first node creates all the certificates and starts the control plane, then prints a join command.
> 3. You deploy a pod-network add-on (which turns the node Ready) and run the join command on each worker, which uses a token and a CA fingerprint to enroll securely.

Which rung to revisit hands-on? Rung 3A + Prediction 2 — the **prereqs** (cgroup driver, sysctls, swap) cause most real `init` failures, and the fixes are muscle memory worth building.

🎯 CKA exam tips & quick notes

- **Order:** runtime → kubeadm/kubelet/kubectl → `kubeadm init` → CNI → `kubeadm join`.
- **Prereqs that block init:** swap on, runtime down, wrong **cgroup driver** (`SystemdCgroup = true`), missing sysctls (`br_netfilter`, `ip_forward`).
- **init flags:** `--apiserver-advertise-address` (right NIC), `--pod-network-cidr` (match CNI), `--control-plane-endpoint` (HA), `--upload-certs` (multi-CP).
- **kubeconfig:** copy `/etc/kubernetes/admin.conf` → `~/.kube/config`.

- Node **NotReady** right after init = **deploy a CNI**.
- **Regenerate the join command** with `kubeadm token create --print-join-command` (24h tokens); `kubeadm reset -f` before re-joining a broken node.
- Control-plane components are **static pods** in `/etc/kubernetes/manifests/`; the **kubelet is a systemd service** you keep running.

📌 Command cheat sheet

```
# PREREQS (all nodes)
sudo swapoff -a
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml && sudo
systemctl restart containerd
# CONTROL PLANE
sudo kubeadm init --apiserver-advertise-address=<ip> --pod-network-cidr=10.244.0.0/16
mkdir -p ~/.kube && sudo cp /etc/kubernetes/admin.conf ~/.kube/config && sudo chown $(id -u):$(id -g)
~/.kube/config
kubectl apply -f <cni>.yaml
kubeadm token create --print-join-command
# WORKER
sudo kubeadm join <ip>:6443 --token <t> --discovery-token-ca-cert-hash sha256:<hash>
```

Related sections

- Section 9 — Design & Install a Kubernetes Cluster — the HA/topology design you're implementing.
- Section 5 — Cluster Maintenance — upgrading the cluster you just built.
- Section 8 — Networking — CNI / pod-CIDR details for the network addon step.
- Section 6 — Security — the PKI kubeadm generates under `/etc/kubernetes/pki`.
- Section 13 — Troubleshooting — when control-plane static pods won't start.
- ../../Linux/14-cgroups.md · ../../Linux/24-kernel-tuning-boot.md — the cgroup driver and sysctls/modules you configure in prereqs.

✅ Answers — "Check yourself before Rung N"

Before Rung 2

Q: Name two things `kubeadm init` generates for you that would be tedious and error-prone to build by hand. (Hint: TLS, and control-plane process definitions.)

A: (1) The entire PKI — the cluster CA plus roughly ten component certificates (apiserver serving cert with its SANs, etcd certs, kubelet client certs, etc.) under `/etc/kubernetes/pki`; done by hand, one wrong SAN means broken TLS. (2) The static-pod manifests for the control-plane processes — apiserver, etcd, scheduler, and controller-manager — written to `/etc/kubernetes/manifests/` with every component's flags correctly wired to the right cert paths and etcd endpoints. Building either manually is "the hard way": days of work with a hundred ways to typo it.

Before Rung 3

Q: Why must the container runtime's cgroup driver match the kubelet's? What kind of failure do you get if they disagree?

A: The cgroup driver is how the runtime and the kubelet account for and enforce resource limits on containers; if the two use different drivers (e.g. containerd on `cgroupfs` while the kubelet uses `systemd`), they manage two conflicting views of the cgroup hierarchy on the same host. The result is instability: the kubelet and runtime fight over resource management, so pods fail to start or the node becomes flaky, and `kubeadm init` can fail its preflight/bring-up. That's why on a systemd host you set `SystemdCgroup = true` in `/etc/containerd/config.toml` yourself — kubeadm ≥1.22 already defaults the kubelet to the systemd driver.

Before Rung 4

Q: After `kubeadm init` succeeds, `kubectl get nodes` shows `NotReady`. Is that a bug? What single action fixes it, and why was the node not Ready before?

A: It's not a bug — it's the expected state right after `init`. The single fix is to deploy a CNI network addon (e.g. `kubectl apply -f kube-flannel.yml`), after which the node flips to `Ready`. The node was `NotReady` because without a CNI plugin pods can't get IPs, so the kubelet reports the node's network as not ready; the addon runs as a DaemonSet (one agent per node) and owns the pod CIDR that must match `--pod-network-cidr`.

Before Rung 5

Q: Which command runs on the control plane and which on a worker: `kubeadm init`, `kubeadm join`? What does the token protect against, and what does the CA-cert hash protect against?

A: `kubeadm init` runs on the control-plane node (it bootstraps the control plane); `kubeadm join` runs on each worker (it enrolls that node). The token authorizes the join — it proves to the control plane that this worker is allowed to join the cluster (and it expires after 24h, limiting the window of abuse). The CA-cert hash protects the worker in the other direction: it lets the worker verify it is talking to the genuine control plane and not a man-in-the-middle impersonating it.

Before Rung 6

Q: At which step do the control-plane *certificates* get created, and at which step does a *worker* first prove it's contacting the genuine control plane?

A: The certificates are created in step 2 of the trace — right after preflight passes, kubeadm generates the cluster CA and every component cert (apiserver serving/SANs, etcd, kubelet client, etc.) under `/etc/kubernetes/pki`, before the static-pod manifests are written. A worker first verifies it's contacting the genuine control plane in step 7, at the start of `kubeadm join`: it contacts `:6443` and checks the discovery-token-ca-cert-hash against the control plane's CA, and only then presents its token and TLS-bootstraps the kubelet.

Before Rung 7

Q: A worker's `kubeadm join` fails with "token expired." Why do tokens expire, and what's the one-line fix on the control plane?

A: Bootstrap tokens are deliberately short-lived — they expire after 24 hours — so a leaked or stale token can't be used indefinitely to enroll rogue nodes into the cluster. The one-line fix on the control plane is `kubeadm token create --print-join-command`, which mints a fresh token and prints the complete join command (including the CA-cert hash) to run on the worker. Don't reuse an old token — regenerate it; and if the worker had a previous half-completed join, run `sudo kubeadm reset -f` on it first.

Helm, Climbed the Ladder 🪜

Section 11 of the CKA — deriving why a whole app can be one versioned package

> Helm — the **package manager for Kubernetes**. We climb from **the pain of managing dozens of manifests → the "an app is one versioned package" idea → the machinery →** then the commands as predictions. Each rung ends with a ✅ **Check yourself**. *Note: Helm is largely supplementary to the current CKA (Kustomize is the tested config tool — Section 12), but it's essential real-world knowledge, in the course, and* `helm.sh/docs` *is allowed in the exam.*

RUNG 0 — The Setup 🎯

What am I learning? How Helm bundles an entire application's Kubernetes objects into one installable, upgradable, rollback-able package.

Why did it land on my desk? It's in the CKA course and it's how real teams ship apps (WordPress, Prometheus, etc.). Even if the exam leans on Kustomize, knowing Helm's chart/release/revision model is table stakes on the job.

What do I already know? Maybe `helm install`. What's fuzzy: the chart-vs-release-vs-revision distinction, and why a rollback creates a *new* revision instead of deleting history.

RUNG 1 — The Pain 🔥

Why does Helm exist at all?

A real app is a dozen-plus objects (Deployments, Services, PVCs, Secrets, ConfigMaps):

```
MANAGING A REAL APP WITHOUT HELM (the pain)
  install:  kubectl apply -f  (×15 files, in the right order)
  tweak:    change PV size / replicas → hand-edit multiple YAMLs
  upgrade:  track all 15 objects, apply the changed ones, hope nothing drifts
  rollback: "what did it look like yesterday?" → no record
  remove:   kubectl delete ×15, and miss one
```

Before / without it: you `kubectl apply` each file, hand-edit to change one value, manually track everything to upgrade, and delete objects one-by-one — with no notion of "the app, version 3."

What breaks without it: atomicity (an app is many objects but should be managed as one), parameterization (changing one value shouldn't mean editing many files), and history (no clean upgrade/rollback of the whole app).

Who feels it most? Anyone deploying off-the-shelf or complex apps repeatedly across environments.

> ✅ **Check yourself before Rung 2:** Name two operations that are painful on a 15-object app without a package manager that Helm makes a single command. (Hint: install, and undo.)

RUNG 2 — The One Idea 💡

The single sentence everything hangs off

> **Helm treats a whole application as one *versioned package*: a chart is a set of templated manifests + default values, installing it creates a release, and every install/upgrade/rollback is a numbered revision you can move between — so you manage N objects as one unit.**

Derivations:

- *"chart = templated manifests + values"* → `templates/*.yaml` with `{{ .Values.x }}` placeholders filled from `values.yaml`; that's how one chart serves many configs.
- *"installing creates a release"* → the same chart can be installed many times (each a separate **release** with its own name), and Helm stores each release's state **as a Secret in the cluster** so the whole team sees it.
- *"every action is a numbered revision"* → `helm history` is an audit trail; a **rollback to revision 1 creates revision 3** (history only moves forward, never deletes).
- *"manage N objects as one unit"* → one `install` / `upgrade` / `uninstall` acts on the entire app.

> ✅ **Check yourself before Rung 3:** Chart, release, revision — say what each is in one phrase. If you install the same chart twice, how many releases and how many revisions exist?

RUNG 3 — The Machinery ⚙️

How it ACTUALLY works — go slow

🧸 Plain-English first (read this before the technical version)

Helm is like a recipe-book system for setting up complicated software. This section covers four things: the vocabulary, an old-vs-new design change, what's inside a recipe book, and the everyday commands.

- **(A) The vocabulary.** A **chart** is the recipe book itself — full instructions with fill-in-the-blank amounts. A **release** is one actual meal cooked from that book; you can cook the same book many times, and each meal gets its own name. A **revision** is a photo of the meal taken every time you change it — install it, update it, or undo an update, and a new numbered photo goes in the album. A **repository** is the library where recipe books are published (with a big catalog called Artifact Hub listing them all). Finally, Helm keeps its record of each release inside the cluster itself (stored as **Secrets** — Kubernetes' locked filing drawers), so the whole team sees the same history, not just the person who ran the command.
- **(B) Old Helm vs. new Helm (version 2 vs. 3).** Old Helm needed a butler program called **Tiller** living inside the cluster with master keys to everything — a security risk. New Helm fired the butler: it acts with *your* keys and *your* permissions, just like your normal tools. New Helm is also smarter about updates: before changing anything it compares **three** things — the old recipe, the new recipe, and what's *actually* running right now — so hand-made tweaks someone applied directly don't get silently steamrolled. (A version label in the recipe book's cover page tells you which era it belongs to: "v2" on the label, confusingly, means new Helm 3.)
- **(C) What's inside a chart.** Four parts: a cover page with metadata (including two different version numbers — the recipe book's own edition number, and the version of the app it cooks); a **values file** listing all the default fill-in-the-blank amounts (this is the file you usually edit); the **templates** folder of instruction pages with blanks in them; and a folder for sub-recipes the main recipe depends on.
- **(D) The commands, and who wins an argument.** There are one-line commands to subscribe to a library, search it, install, list, upgrade, view history, roll back, and uninstall. When you customize amounts, there's a pecking order: a value typed directly on the command line beats a value in your own custom file, which beats the book's printed default. Two gotchas: **undoing never erases history** — rolling back to photo 1 just takes a *new* photo 3 that looks like photo 1; and a rollback restores the *setup instructions*, not your data — like rebuilding a kitchen to an old floor plan without restoring what was in the fridge.

Now the original technical deep-dive — the same ideas, in precise form:

(A) The model

Term	What it is
Chart	The package — templates + default values + metadata
Release	One *installation* of a chart (many per chart, each named)
Revision	A snapshot; each install/upgrade/rollback makes a new one
Repository	Where charts are hosted (Bitnami…); **Artifact Hub** lists them all
Release state	Stored **as Secrets in the cluster** (team-visible)

(B) Helm 2 vs 3 (know the differences)

- **Tiller removed in Helm 3** — Helm 2 needed a cluster-side `Tiller` running in "god mode" (a security hole). Helm 3 talks straight to the API server using **your RBAC** (like kubectl).
- **Three-way strategic merge** — Helm 3 upgrades/rollbacks compare the **old chart, new chart, AND live cluster state**, so it preserves manual `kubectl` changes on upgrade and reverts them correctly on rollback (Helm 2 only compared charts).
- **`Chart.yaml apiVersion`:** `v1` = Helm 2; **`v2` = Helm 3** (adds `dependencies`, `type`).

(C) Chart structure

```
mychart/
├── Chart.yaml          # metadata (apiVersion v2, name, version, appVersion)
├── values.yaml         # DEFAULT config values (the file you usually edit)
├── templates/          # templated manifests ({{ .Values.x }})
└── charts/             # sub-charts (dependencies)
```

```
# Chart.yaml
apiVersion: v2            # v2 = Helm 3
name: wordpress
version: 15.2.5           # the CHART version
appVersion: "6.1.0"       # the APP version it deploys (informational)
```

`version` = chart version; `appVersion` = the app it ships. `{{ .Values.foo }}` pulls from `values.yaml`.

(D) Commands + override precedence

```
helm repo add bitnami https://charts.bitnami.com/bitnami && helm repo update
helm search hub wordpress          # Artifact Hub (all repos)   |  search repo = your repos
helm install my-site bitnami/wordpress      # <release> <chart>
helm list -A                                # all releases
helm upgrade my-site bitnami/wordpress
helm history my-site                        # revisions + action
helm rollback my-site 1                      # → a NEW revision with rev-1's config
helm uninstall my-site
```

Customize (precedence: `--set` > `--values` > chart `values.yaml`):

```
helm install my-site bitnami/wordpress --set wordpressBlogName="Helm Tutorials"
helm install my-site bitnami/wordpress --values custom-values.yaml
helm pull bitnami/wordpress --untar && vi wordpress/values.yaml && helm install my-site ./wordpress
```

> 🎯 A **rollback to revision 1 creates revision 3** — Helm never deletes history, it moves forward. `helm template` / `--dry-run` renders without installing. Rollbacks restore **manifests, not data** (a PV's DB isn't in a revision).

> ✅ **Check yourself before Rung 4:** If `values.yaml` sets `replicas: 1`, a `--values` file sets `3`, and `--set replicas=5`, how many replicas deploy — and why?

RUNG 4 — The Vocabulary Map 🏷️

Term	What it actually is	Which part
Chart	The package	Model
Release	One install of a chart	Model
Revision	A versioned snapshot	Model / history
Repository / Artifact Hub	Chart hosting / index	Distribution
values.yaml	Default config	Templating
template / `{{ .Values }}`	Manifest with placeholders	Templating
Chart.yaml (apiVersion v2)	Metadata; v2 = Helm 3	Metadata
version vs appVersion	Chart version vs app version	Metadata
Tiller	Helm 2's removed server component	Helm 2 vs 3
three-way merge	old chart + new chart + live state	Helm 3 upgrades
--set / --values	CLI override / file override	Customization
helm pull --untar	Download a chart locally	Customization

The unlock: a chart is a **template**; a release is a **rendered instance**; a revision is a **saved version of that instance**. Everything else (repos, values, history) supports that trio.

> ✅ **Check yourself before Rung 5:** Which stores where the app's *runtime data* lives — a Helm revision, or the PV? So what does a rollback actually restore?

RUNG 5 — The Trace 🎬

Trace — install → upgrade → rollback:

1. `helm install my-site bitnami/wordpress`: Helm **fetches the chart**, **renders** `templates/` with `values.yaml` into concrete manifests, **applies** them to the cluster, and records **release `my-site`, revision 1** as a Secret.
2. `helm upgrade my-site …`: Helm renders the new chart/values, does a **three-way merge** against live state, applies the diff, and records **revision 2**.
3. A problem appears. `helm history my-site` shows rev 1 and 2. `helm rollback my-site 1`: Helm re-applies rev 1's manifests and records **revision 3** (whose config equals rev 1's) — history moved forward, nothing was deleted.

4. `helm uninstall my-site` removes every object of the release in one shot.

```
install → render(chart+values) → apply → release/rev 1 (Secret)
upgrade → render → 3-way merge → apply → rev 2
rollback 1 → re-apply rev-1 manifests → rev 3 (=rev1 config)
```

> ✅ **Check yourself before Rung 6:** After the rollback, what revision number is live, and what config does it contain? Why isn't it "revision 1" again?

RUNG 6 — The Contrast ⚖️

This…	vs that…	Difference
Helm	**Kustomize**	Go-templating + values + releases vs plain-YAML overlays (no templating)
Helm	**raw `kubectl apply`**	whole-app package + rollback vs per-file, no history
Helm 3	**Helm 2**	uses your RBAC, no Tiller, 3-way merge vs Tiller in god-mode, 2-way
version	**appVersion**	the chart's version vs the app's version
`--set`	**`--values` / values.yaml**	inline override (highest) vs file / defaults

When NOT to: for the **CKA exam**, reach for **Kustomize** (it's the tested tool); don't expect Helm rollbacks to restore *data* (only manifests); don't hand-edit a release's live objects (the next upgrade's 3-way merge may fight you — change values instead).

One-sentence "why this over that":

> Use Helm to install and version *whole applications* (especially third-party ones) as parameterized packages with real upgrade/rollback; use Kustomize for plain-YAML, template-free per-environment tweaks — and that's what the CKA tests.

> ✅ **Check yourself before Rung 7:** Why is Helm 3 considered more secure than Helm 2 for a shared cluster? Name the component that went away and what replaced its permissions.

RUNG 7 — The Prediction Test 🧪

Prediction 1 — One command installs the whole app; one removes it

> **My prediction:** "If I `helm install`, then a Deployment + Service (and more) appear from a single command, and `helm uninstall` removes them all at once — *because* Helm manages the release as one unit."

```
helm repo add bitnami https://charts.bitnami.com/bitnami && helm repo update
helm install web bitnami/nginx
helm list ; kubectl get pods,svc          # created by one command
helm uninstall web                        # all gone
```

Verify: one install created multiple objects; one uninstall removed them. Compare to N `kubectl apply` / `delete`.

Prediction 2 — Rollback creates a new, higher revision

> **My prediction:** "If I install, upgrade, then `helm rollback … 1`, then `helm history` shows revision 3 labeled 'rollback to 1', not a return to revision 1 — *because* Helm's history only moves forward."

```
helm install nginx-release bitnami/nginx --version 13.2.10
helm upgrade nginx-release bitnami/nginx        # → rev 2
helm rollback nginx-release 1                    # → rev 3 (= rev 1 config)
helm history nginx-release                       # rev 3 "rollback to 1"
```

Verify: the live revision is 3 with rev-1's config. The pod image reverts, but the revision number climbed.

Prediction 3 — `--set` beats `--values` beats defaults

> **My prediction:** "If I override a value with `--set`, `helm get values` shows my override taking effect over the chart default — *because* `--set` has the highest precedence."

```
helm install my-site bitnami/wordpress --set wordpressBlogName="My Site" --set
wordpressUsername=admin
helm get values my-site                  # confirms the overrides
```

Verify: `helm get values` lists your overrides layered over defaults.

Prediction: "If I do X, then Y, because [mechanism]"	Ran?	Right?	Miss?
1.			

🎁 CAPSTONE — Compress It

One sentence, no notes:

> Helm packages a whole app as a chart (templated manifests + default values); installing it makes a release whose every install/upgrade/rollback is a numbered, cluster-recorded revision — so you manage many objects, and their history, as one unit.

Explain it to a beginner in 3 sentences:

> 1. Instead of applying and tracking a dozen YAML files, you install one chart, and Helm creates and manages all the objects together as a "release."
> 2. You customize a chart's values at install time, and each install/upgrade/rollback is a numbered revision you can move between.
> 3. Helm 3 talks to the cluster with your own permissions (no Tiller), and rollbacks restore the manifests — but not the app's data.

Which rung to revisit hands-on? Rung 3D + Prediction 2 — the **revision model** (rollback = new revision) and **override precedence** are the two things people get wrong.

🎯 CKA / real-world tips & quick notes

- **Helm 3 = no Tiller**, uses your **RBAC**; `apiVersion: v2` in `Chart.yaml`.
- **Chart vs release vs revision:** package / one install / a snapshot.
- **Commands:** `repo add/update`, `search hub/repo`, `install`, `list -A`, `upgrade`, `history`, `rollback <rev>`, `uninstall`, `pull --untar`.
- **Override precedence:** `--set` > `--values` > chart `values.yaml`.
- **Rollback = new revision** (forward-only) and restores **manifests, not data**.
- `helm template` / `--dry-run` renders without applying; release metadata is stored as **Secrets**.

📌 Command cheat sheet

```
helm repo add <name> <url> && helm repo update
helm search repo <chart>            helm search hub <chart>
helm install <release> <chart> [--set k=v] [--values f.yaml] [--version x]
helm list -A                        helm history <release>
helm upgrade <release> <chart>      helm rollback <release> <revision>
helm uninstall <release>            helm pull <chart> --untar
helm get values <release>           helm template <chart>      # render only
```

Related sections

- Section 12 — Kustomize Basics — the config tool that IS on the CKA (contrast with Helm).
- Section 4 — Application Lifecycle Management — rollouts/rollbacks Helm wraps.
- Section 1 — Core Concepts — the objects a chart bundles.
- Section 6 — Security — the RBAC Helm 3 relies on.

✅ Answers — "Check yourself before Rung N"

Before Rung 2

Q: Name two operations that are painful on a 15-object app without a package manager that Helm makes a single command. (Hint: install, and undo.)

A: (1) Install: without Helm you `kubectl apply -f` fifteen files in the right order; with Helm it's one `helm install <release> <chart>` that creates every object of the app. (2) Rollback (the "undo"): without Helm there's no record of "what did it look like yesterday," so undoing an upgrade is guesswork; with Helm it's one `helm rollback <release> <revision>` because every install/upgrade is a recorded revision. (Uninstall is the same story: one `helm uninstall` versus fifteen `kubectl delete`s where you'll miss one.)

Before Rung 3

Q: Chart, release, revision — one phrase each. If you install the same chart twice, how many releases and how many revisions exist?

A: A **chart** is the package — templated manifests plus default values and metadata. A **release** is one named installation of a chart in the cluster. A **revision** is a numbered snapshot of a release; every install/upgrade/rollback creates a new one. Installing the same chart twice (under two release names) gives you **two releases**, each currently at **revision 1** — so two revisions total, one per release, tracked independently.

Before Rung 4

Q: If `values.yaml` sets `replicas: 1`, a `--values` file sets `3`, and `--set replicas=5`, how many replicas deploy — and why?

A: Five replicas deploy. Helm's override precedence is `--set` > `--values` > the chart's own `values.yaml`, so the inline `--set replicas=5` wins over the `--values` file's 3, which itself would have beaten the chart default of 1. The chart's `values.yaml` only supplies defaults for anything not overridden higher up the chain.

Before Rung 5

Q: Which stores where the app's runtime data lives — a Helm revision, or the PV? So what does a rollback actually restore?

A: The runtime data lives in the PV — a Helm revision stores only the rendered manifests and the values used for that release, not the contents of any volume (a PV's database is not in a revision). Therefore a `helm rollback` restores **manifests, not data**: it re-applies the old revision's object definitions (images, replicas, config), while whatever the app has written to its PersistentVolume stays exactly as it is.

Before Rung 6

Q: After the rollback, what revision number is live, and what config does it contain? Why isn't it "revision 1" again?

A: After `helm rollback my-site 1` the live revision is **revision 3**, and its config is identical to revision 1's — Helm re-applied rev 1's manifests as a new snapshot. It isn't "revision 1" again because Helm's history only moves forward and never deletes: every action (install, upgrade, rollback) appends a new numbered revision, so `helm history` remains a complete audit trail showing rev 1, rev 2, and rev 3 labeled "rollback to 1."

Before Rung 7

Q: Why is Helm 3 considered more secure than Helm 2 for a shared cluster? Name the component that went away and what replaced its permissions.

A: Helm 2 required **Tiller**, a cluster-side server component that ran in "god mode" with broad permissions — a security hole, because anyone who could talk to Tiller could effectively do anything in the cluster. Helm 3 removed Tiller entirely; the client now talks directly to the API server using **your own RBAC credentials** (exactly like kubectl), so each user's Helm actions are limited to what their Kubernetes permissions actually allow.

Kustomize, Climbed the Ladder 🪜

Section 12 of the CKA — deriving template-free, per-environment YAML

> Kustomize — plain-YAML customization built into `kubectl` (`-k`), and **on the current CKA curriculum** (unlike Helm). We climb from **the pain of copying manifests per environment → the "one base, layered overlays" idea → the machinery** → then the commands as predictions. Each rung ends with a ✅ **Check yourself**.

RUNG 0 — The Setup 🎯

What am I learning? How to keep one set of YAML and produce per-environment variants (dev/staging/prod) by patching only what differs — no templating language.

Why did it land on my desk? Kustomize is the config tool the **CKA actually tests** (it's built into `kubectl`), and it appears in tasks like "add a label/namespace/replica count across these manifests."

What do I already know? Maybe `kubectl apply -f`. What's fuzzy: base vs overlay, when to use a *transformer* vs a *patch*, and the two patch styles.

RUNG 1 — The Pain 🔥

Why does Kustomize exist at all?

Three environments, each needing small differences (dev 1 replica, staging 2, prod 5):

```
CUSTOMIZING PER ENVIRONMENT BY HAND (the pain)
  copy the whole manifest set ×3 (dev/, staging/, prod/) → drift is guaranteed
  a shared change (new label) → edit it in THREE places, miss one
  hand-edit replicas/image per env → error-prone, no single source of truth
```

Before / without it: you duplicated manifests per environment (they drift apart) or edited them by hand each deploy. Helm solves this with *templating*, but templated YAML is no longer plain/valid YAML you can read and `kubectl apply` directly.

What breaks without it: a single source of truth (duplicates diverge) and safe per-env variation (manual edits are fragile).

Who feels it most? Platform/app teams promoting the same app across environments.

> ✅ **Check yourself before Rung 2:** Why is "copy the manifests into dev/, staging/, prod/ folders" a maintenance trap? What happens when you need to change something shared by all three?

RUNG 2 — The One Idea 💡

The single sentence everything hangs off

> **Kustomize keeps ONE base of plain, valid YAML and layers per-environment overlays that patch only what differs — with no templating language — so dev/staging/prod share a single source of truth and you never duplicate manifests.**

Derivations:

- *"one base... overlays patch only what differs"* → the base holds shared defaults; each overlay references the base (`resources: [../../base]`) and changes just its deltas.
- *"no templating language"* → everything stays **plain, valid YAML** (unlike Helm's `{{ }}`), so you can still `kubectl apply` a piece on its own and read it directly.
- *"patch only what differs"* → two tools for changing things: **transformers** (broad, e.g. label/namespace *everything*) and **patches** (surgical, e.g. *this* deployment's replicas).
- *(built-in)* `kubectl apply -k` renders base+overlay and applies — no separate binary needed.

> ✅ **Check yourself before Rung 3:** In one sentence, what's the difference in *purpose* between a base and an overlay? What does an overlay do to avoid duplicating the base?

RUNG 3 — The Machinery ⚙️

How it ACTUALLY works — go slow

🧸 Plain-English first (read this before the technical version)

Kustomize solves a simple problem: you run the same app in three settings (a practice kitchen, a rehearsal kitchen, and the real restaurant), and each needs tiny differences. Instead of keeping three full copies of the instructions — which slowly drift apart — you keep **one master copy plus a short "what's different here" note for each kitchen**. This section covers three things: the folder layout, broad changes, and surgical changes.

- **(A) The layout: base, overlays, and the instruction card.** The **base** folder holds the shared master instructions. Each **overlay** folder (one per environment — dev, staging, prod) doesn't copy anything; it just says "start from the master copy, then change these one or two things" (for example, run 2 copies of the app instead of 1). The tool only reads a specially named file, `kustomization.yaml` — think of it as the index card on the front of each folder listing (1) which instruction sheets belong here and (2) what tweaks to apply. One gotcha: the "build" command only *prints* the final combined instructions on screen — it doesn't do anything to your cluster. A separate apply command actually deploys them, and it's built into the normal Kubernetes tool, no extra software needed. The index card can also point at whole sub-folders, so one command can deploy everything at once.
- **(B) Transformers: broad, blanket changes.** A **transformer** stamps the *same* change onto *every* item at once — like a rubber stamp across a whole stack of paperwork. Examples: add the company name to every document (labels), add a prefix to every name, file everything under one department (namespace), or swap out which brand of ingredient is used everywhere (container images). Placement matters: a stamp on the top-level index card hits everything; a stamp on a sub-folder's card only hits that folder.
- **(C) Patches: surgical, one-item changes.** A **patch** edits one field on one specific item — tweezers instead of a rubber stamp. There are two writing styles:
  - **Strategic merge**: you write a mini-version of the document showing only the part that changes (plus the name, so the tool knows which one you mean). Easy to read — usually the go-to.
  - **JSON 6902** (a precise edit-list format): you write step-by-step edit instructions — "replace this exact field with this value," "add," or "remove." It shines when editing *lists*, because you can point at "item number 0" or say "append to the end."

The rule of thumb the section builds to: **blanket change for everything = transformer; one-off change to one object = patch.**

Now the original technical deep-dive — the same ideas, in precise form:

(A) base + overlays + `kustomization.yaml`

```
myapp/
├── base/                       # shared defaults
│   ├── kustomization.yaml
│   ├── deployment.yaml         # replicas: 1
│   └── service.yaml
└── overlays/
    ├── dev/kustomization.yaml       # resources: [../../base]  (uses default 1)
    ├── staging/kustomization.yaml   # + patch replicas: 2
    └── prod/kustomization.yaml      # + patch replicas: 5
```

Kustomize only reads a file named `kustomization.yaml` with two core parts: `resources` (the manifests) and **transformations**.

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- deployment.yaml
- service.yaml
commonLabels: { company: KodeKloud }     # a transformer
```

```
kustomize build k8s/                  # RENDERS final YAML to stdout (does NOT apply)
kubectl apply -k k8s/                 # built-in: render + apply
kubectl delete -k k8s/                # delete what it manages
```

> 🎯 `kustomize build` only **prints**; use `kubectl apply -k` to deploy. A root `kustomization.yaml` can list **directories** (each with its own kustomization) instead of files — one `apply -k` deploys everything.

(B) Transformers — broad changes

Apply the **same change across many resources**:

Transformer	Effect
`commonLabels` (newer: `labels`)	add a label to every resource
`commonAnnotations`	add an annotation to every resource
`namePrefix` / `nameSuffix`	prepend/append to every name
`namespace`	put every resource in a namespace
`images`	swap image name/tag

```
namespace: dev
namePrefix: kodekloud-
images:
- { name: mongo, newName: postgres, newTag: "4.2" }   # match on IMAGE name; quote numeric tags
```

> 🎯 **Scope:** a transformer in the **root** kustomization applies to **all** imported resources; the same in a **subdir's** kustomization applies **only** there. For `images`, match on the **image** name (not the container name).

(C) Patches — surgical changes

Change one field on one object. Two styles:

Strategic Merge Patch (partial YAML — usually easier): paste a snippet with only what changes; merged by name.

```
patches:
- path: replica-patch.yaml       # or inline with `patch: |`
```

```
# replica-patch.yaml — a partial Deployment
apiVersion: apps/v1
kind: Deployment
metadata: { name: api-deployment }   # identifies the target
spec: { replicas: 5 }                # the only change
```

JSON 6902 Patch (op/path/value — precise, best for lists):

```
patches:
- target: { kind: Deployment, name: api-deployment }
  patch: |-
    - op: replace                # add | remove | replace
      path: /spec/replicas
      value: 5
```

- `op:` `replace` / `add` / `remove`. `path:` `/spec/replicas`, `/metadata/name` …
- **Lists use an index**: `/spec/template/spec/containers/0`; `/-` = append to the end.

> 🎯 **Strategic merge** = partial-YAML-that-looks-like-the-resource (readable, adding/changing fields). **JSON6902** = `op/path/value` (best for lists — indexes/ `/-`, `remove`). Both can be inline (`patch: |-`) or a file (`path:`). Transformers = broad; patches = one-object surgery.

> ✅ **Check yourself before Rung 4:** You want to (a) put every resource in namespace `prod`, and (b) set *one* deployment's replicas to 5. Which is a transformer and which is a patch?

RUNG 4 — The Vocabulary Map 🏷️

Term	What it actually is	Which part
base	Shared default manifests	Model
overlay	Per-env layer referencing the base	Model
kustomization.yaml	The file Kustomize reads (`resources` + transforms)	Model
resources	Files or dirs to manage	kustomization
transformer	Broad change across resources	Transformers
commonLabels / namePrefix / namespace / images	Specific transformers	Transformers
patch	Change on one/few objects	Patches
strategic-merge	Partial-YAML patch, merged by name	Patches
JSON6902	`op/path/value` patch	Patches
op / path / `/-`	replace-add-remove / field path / append	JSON6902
kubectl apply -k	Render + apply	Build/apply
kustomize build	Render to stdout only	Build/apply

The unlock — two axes:

```
BREADTH:  transformer (many resources) ── vs ── patch (one object)
STYLE:    strategic-merge (partial YAML) ── vs ── JSON6902 (op/path/value)
STRUCTURE: base (shared) ── + ── overlay (per-env deltas)
```

> ✅ **Check yourself before Rung 5:** Sort these into transformer vs patch: `namePrefix: kk-` ; bump `api-deployment` replicas to 5; label everything `env=prod` ; add a sidecar container to one pod.

RUNG 5 — The Trace 🎬

Trace — `kubectl apply -k overlays/prod` :

1. Kustomize reads `overlays/prod/kustomization.yaml` .
2. Its `resources: [../../base]` pulls in the **base** manifests (deployment `replicas: 1` , service).
3. **Transformers** in the overlay run across those resources (e.g. `namespace: prod` , `namePrefix: prod-` , a `commonLabels`).
4. **Patches** run on their targets — e.g. a JSON6902 `replace /spec/replicas → 5` on the `web` Deployment.
5. Kustomize emits the **final rendered YAML** (base + transforms + patches); `kubectl` applies it.
6. The **base is untouched** (`replicas: 1`); only the rendered prod output has `5` . Swapping to `overlays/dev` renders the default `1` — same base, different result.

```
apply -k prod → read overlay → import base(replicas:1) → transformers(ns/prefix/labels)
             → patches(replicas→5) → final YAML → kubectl apply   (base stays 1)
```

> ✅ **Check yourself before Rung 6:** After applying `overlays/prod`, what's `replicas` in the *base* deployment.yaml file on disk? Why didn't it change?

RUNG 6 — The Contrast ⚖️

This…	vs that…	Difference
Kustomize	**Helm**	plain-YAML overlays, no templating, built into kubectl vs Go-templating + releases
transformer	**patch**	same change across many resources vs one-object surgery
strategic-merge	**JSON6902**	partial YAML by name vs `op/path/value` (best for lists)
`kubectl apply -k`	`kustomize build`	render + apply vs render to stdout only
root-scope transformer	**subdir-scope**	applies to all imported resources vs only that subdir

When NOT to: don't use Kustomize when you need loops/conditionals/complex templating (that's Helm); don't forget it's `apply -k` to deploy (`build` alone only prints); don't reach for a patch when a transformer covers it (label/namespace everything → transformer).

One-sentence "why this over that":

> Use Kustomize for template-free, per-environment YAML that stays a single readable source of truth (and it's what the CKA tests); use Helm when you need full templating and packaged releases.

> ✅ **Check yourself before Rung 7:** You need to append a container to the *end* of one pod's container list. Which patch style, and what does the `path` look like?

RUNG 7 — The Prediction Test 🧪

Prediction 1 — One `apply -k` deploys multiple directories

> **My prediction:** "If a root `kustomization.yaml` lists `api/` and `db/` (each with its own kustomization), then one `kubectl apply -k` creates everything — *because* Kustomize recurses into listed directories."

```
cat > k8s/kustomization.yaml <<'EOF'
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources: [api/, db/]
EOF
kubectl apply -k k8s/ ; kubectl get pods      # api + db from one command
```

Verify: one `apply -k` created resources across subdirectories — no per-folder apply.

Prediction 2 — Overlays change replicas without touching the base

> **My prediction:** "If the base sets `replicas: 1` and the prod overlay patches it to 5, then `apply -k overlays/prod` deploys 5 while the base file still reads 1 — *because* the overlay patches the *rendered* output, not the base file."

```
# overlays/prod/kustomization.yaml: resources:[../../base] + a replace patch to 5
kubectl apply -k overlays/prod
kubectl get deploy web -o jsonpath='{.spec.replicas}{"\n"}'   # 5
grep replicas base/deployment.yaml                            # still 1
```

Verify: prod = 5, base file = 1. Switch to `overlays/dev` → default 1.

Prediction 3 — Transformers relabel/namespace/retag everything at once

> **My prediction:** "If I set `namespace`, `namePrefix`, `commonLabels`, and `images` in the root kustomization, then every object lands in that namespace, gets the prefix + label, and the image tag changes — *because* transformers apply broadly."

```
# root kustomization.yaml
namespace: staging
namePrefix: kk-
images: [{ name: nginx, newTag: "1.26" }]
commonLabels: { env: staging }
```

```
kubectl apply -k k8s/
kubectl get deploy -n staging                              # names prefixed kk-, label env=staging
kubectl get deploy kk-web -n staging -o jsonpath='{..image}{"\n"}'   # nginx:1.26
```

Verify: every object got the namespace, prefix, label, and new tag — no manifest edits.

Prediction: "If I do X, then Y, because [mechanism]"	Ran?	Right?	Miss?
1.			

🎁 CAPSTONE — Compress It

One sentence, no notes:

> Kustomize renders a single base of plain YAML plus per-environment overlays that use transformers (broad changes) and patches (surgical changes) to alter only what differs — deployed with `kubectl apply -k`, no templating.

Explain it to a beginner in 3 sentences:

1. You keep one shared "base" of normal YAML and, per environment, a small overlay that changes only what's different.
2. Broad changes (namespace, labels, image, name prefix) are "transformers"; one-off changes (this deployment's replicas) are "patches."
3. `kubectl apply -k <dir>` renders base + overlay and applies it — and it's all valid YAML you can still read and apply by hand.

Which rung to revisit hands-on? Rung 3C — the **two patch styles** and JSON6902 list paths (`/containers/0`, `/-`) are the fiddly, testable bit.

🎯 CKA exam tips & quick notes

- **Deploy with `kubectl apply -k <dir>`** (or `kustomize build <dir> | kubectl apply -f -`). `build` alone only prints.
- **`kustomization.yaml`** must be named exactly that: `resources:` + transformers/patches.
- **Transformers** (`commonLabels`/`labels`, `namePrefix`/`nameSuffix`, `namespace`, `commonAnnotations`, `images`) = broad; **patches** = per-object.
- **Two patch styles:** strategic-merge (partial YAML, by `name`) and JSON6902 (`op/path/value`, `target`); lists use **index** or `/-`; quote numeric image tags.
- **Scope:** root transformer → all resources; subdir → only that subdir.
- **base + overlays:** overlay's `resources: [../../base]`, then patch the deltas. `kubectl delete -k` tears it down.

📌 Command cheat sheet

```
kubectl apply -k k8s/                  # deploy a kustomization
kubectl delete -k k8s/                 # remove it
kustomize build k8s/                   # render only (stdout)
kustomize build k8s/ | kubectl apply -f -
# keys: resources, commonLabels/labels, namePrefix, nameSuffix, namespace,
#       commonAnnotations, images, patches (strategic-merge or JSON6902)
```

Related sections

- Section 11 — Helm Basics — the templating alternative (contrast: templates vs plain-YAML overlays).
- Section 4 — Application Lifecycle Management — the manifests Kustomize customizes.
- Section 1 — Core Concepts — declarative `kubectl apply` that `-k` extends.
- Section 7 — Storage / Section 6 — Security — resources you'd patch per environment.

✅ Answers — "Check yourself before Rung N"

Before Rung 2

Q: Why is "copy the manifests into dev/, staging/, prod/ folders" a maintenance trap? What happens when you need to change something shared by all three?

A: Because duplicated manifest sets are guaranteed to drift apart — there is no single source of truth, and each folder gets hand-edited independently until the environments quietly diverge. When you need a shared change (say, a new label), you must make the same edit in three places, and sooner or later you'll miss one, leaving the environments inconsistent in ways nobody notices until something breaks.

Before Rung 3

Q: In one sentence, what's the difference in *purpose* between a base and an overlay? What does an overlay do to avoid duplicating the base?

A: The base holds the shared default manifests common to every environment, while an overlay is a per-environment layer whose only job is to change the deltas (dev's 1 replica vs prod's 5). To avoid duplicating the base, the overlay doesn't copy any manifests — it *references* the base with `resources: [../../base]` in its `kustomization.yaml`, then applies only its own transformers and patches on top of the imported base at render time.

Before Rung 4

Q: (a) Put every resource in namespace `prod`, and (b) set one deployment's replicas to 5. Which is a transformer and which is a patch?

A: (a) is a transformer: `namespace: prod` in the kustomization applies the same change broadly, across every resource it manages. (b) is a patch: bumping a single deployment's `spec.replicas` to 5 is one-object surgery, done either as a strategic-merge patch (a partial Deployment naming `api-deployment` with `spec: { replicas: 5 }`) or a JSON6902 patch (`op: replace, path: /spec/replicas, value: 5`). The rule from the file: transformers = broad, patches = surgical.

Before Rung 5

Q: Sort into transformer vs patch: `namePrefix: kk-`; bump `api-deployment` replicas to 5; label everything `env=prod`; add a sidecar container to one pod.

A: `namePrefix: kk-` — transformer (prepends to every resource's name). Bump `api-deployment` replicas to 5 — patch (one object's field). Label everything `env=prod` — transformer (`commonLabels`, applied to all resources). Add a sidecar container to one pod — patch (surgery on a single object's container list; a JSON6902 `add` with path `/spec/template/spec/containers/-` is the natural fit for appending to a list).

Before Rung 6

Q: After applying `overlays/prod`, what's `replicas` in the *base* deployment.yaml file on disk? Why didn't it change?

A: Still `replicas: 1`. Kustomize never modifies source files: `apply -k` imports the base, runs the overlay's transformers and patches in memory, and emits final rendered YAML that kubectl applies — the `5` exists only in that rendered prod output. That's the whole point of the model: the base stays the untouched single source of truth, so applying `overlays/dev` against the very same base renders the default `1`.

Before Rung 7

Q: You need to append a container to the *end* of one pod's container list. Which patch style, and what does the `path` look like?

A: Use a JSON6902 patch — it's the style built for list operations, using `op/path/value` with list indexes. The operation is `op: add` and the path ends in `/-`, which means "append to the end of the list": `path: /spec/template/spec/containers/-` (for a Deployment's pod template), with `value:` holding the new container's YAML. Strategic merge is the readable choice for changing named fields, but precise list positioning (`/containers/0`, `/-`, `remove`) is JSON6902 territory.

Troubleshooting, Climbed the Ladder 🪜

Section 13 of the CKA — deriving a method so you never guess

> The exam's highest-value skill (**30% of the CKA**): given a broken cluster, find and fix the fault — across **application**, **control-plane**, **worker-node**, and **network** failures. We climb from **the pain of guessing under a clock** → **the "walk the path until a check fails" idea** → **the four checklists** → then real diagnoses as predictions. Each rung ends with a ✅ **Check yourself**.

RUNG 0 — The Setup 🎯

What am I learning? A repeatable diagnostic method — and the four fixed checklists (app / control-plane / node / network) that turn "something's broken" into a fast, ordered hunt.

Why did it land on my desk? Troubleshooting is the **single largest CKA domain at 30%**, and it's where a method beats knowledge: the same commands, applied in the right order, crack almost every broken-cluster task.

What do I already know? Probably `kubectl describe` and `logs`. What's fuzzy: *which* to reach for first, how to read a symptom to the right layer, and what to do when `kubectl` itself is dead.

RUNG 1 — The Pain 🔥

Why does a troubleshooting method exist at all?

You're handed a broken cluster and a 2-hour clock. Without a method:

```
GUESSING UNDER PRESSURE (the pain)
  change a random field → didn't help → change another → now TWO things are wrong
  read the wrong logs   → the real error was in the PREVIOUS container
  fix the app           → but the fault was the control plane the whole time
  20 minutes gone, cluster more broken than before
```

Before / without a method: you poke at whatever's nearest, make changes you can't reason about, and lose track of what you've touched — the worst way to debug against a clock.

What breaks without it: *time* (the exam's scarcest resource) and *reversibility* (undisciplined changes compound the fault).

Who feels it most? You, in the exam — and every on-call engineer at 3 AM.

> ✅ **Check yourself before Rung 2:** Why is "start changing things and see what helps" a losing strategy on a timed, multi-fault cluster? What does it cost you beyond time?

RUNG 2 — The One Idea 💡

The single sentence everything hangs off

> **Troubleshooting is walking the path a request (or a control-signal) takes, hop by hop, until a check fails — the symptom localizes the layer (application / control-plane / worker-node / network), and each layer has a fixed checklist, so you diagnose by elimination instead of guessing.**

Derivations:

- *"walk the path hop by hop"* → for an app: `user → Service → Pod → backend Service → Pod`; check each link (endpoints, ports, logs) until one fails.
- *"the symptom localizes the layer"* → **Pending** pod ⇒ scheduler; **no scaling/self-heal** ⇒ controller-manager; `kubectl` **dead** ⇒ apiserver/etcd; **node NotReady** ⇒ kubelet; **DNS/connectivity** ⇒ CNI/kube-proxy/CoreDNS.
- *"each layer has a fixed checklist"* → you don't invent steps under pressure; you run the layer's list.
- *"diagnose by elimination"* → every check either clears a hop or names the fault — no wasted moves.

The golden rule: **draw the map, then walk every link.**

> ✅ **Check yourself before Rung 3:** Match symptom → layer: (a) pods stuck Pending, (b) `kubectl` returns connection refused, (c) node shows NotReady, (d) `Endpoints: <none>` on a Service.

RUNG 3 — The Machinery ⚙️

The four checklists — the most important rung. Go slow.

🧸 Plain-English first (read this before the technical version)

Think of a detective investigating a building where "something is wrong," ruling out suspects floor by floor. There are four floors — the app, the management office, the worker rooms, and the phone system — and each floor has its own fixed checklist, so you never guess.

- **(A) The app floor: follow the customer's path.** A customer's request travels a known route: front door → receptionist → the app itself → its assistant → the filing room (database). You check each handoff in order until one fails. The classic culprits are all mix-ups: the receptionist's contact list points at nobody (the **Service** — the internal switchboard — has labels that don't match any app, so its list of destinations is empty); someone wrote down the wrong department name; the call is forwarded to the wrong extension (port number); or the database password on file is wrong. When a program keeps crashing over and over, the crucial trick is reading the *previous* attempt's diary (logs), because the current one hasn't failed yet.
- **(B) The management office: the symptom names the culprit.** Kubernetes has a few manager programs, and each failure style points at exactly one. New work sits waiting and never gets assigned to a room? The **scheduler** (the room-assigner) is down. Nothing scales up or heals itself? The **controller manager** (the supervisor who notices gaps and fills them) is down. Your own command tool won't connect at all? The **API server** (the front desk everything talks through) is down. Twist: if the front desk is dead, your usual tool is blind — so you use a backstage tool (`crictl`) that talks directly to the machinery running the programs. The managers are started from plain instruction files in one folder; fix a typo in the file and the manager restarts itself automatically.
- **(C) The worker rooms: go there in person.** If a whole machine reports "NotReady," check its status report first, then walk to the machine itself (log in remotely) and interrogate its caretaker program, the **kubelet**: Is it even running? What do its diary entries (system logs) say? Its mistakes live in one of exactly two settings files — one governing *how it behaves*, one holding *the address it uses to phone headquarters* — and the error message tells you which file to open.
- **(D) The phone system: three suspects only.** Network trouble comes down to a trio: the wiring installer that gives every app an internal number (the CNI), the call-router that forwards switchboard calls to the right app (kube-proxy), and the phone book that turns names into numbers (CoreDNS, the cluster's directory service). Everything unreachable right after setup? Wiring never installed. Name lookups failing everywhere? Phone book down. Apps fine but the switchboard number dead? Call-router.

Method over memory: match the symptom to the floor, then run that floor's checklist top to bottom.

Now the original technical deep-dive — the same ideas, in precise form:

(A) Application failure — walk the request map

```
user ──▶ web Service (NodePort) ──▶ web Pod ──▶ db Service ──▶ db Pod
         └── check each hop, both directions ──┘
```

```
curl http://<node-ip>:<nodePort>          # 1) reach the app
kubectl describe svc web-service          # 2) Endpoints: <none> = selector/label MISMATCH
kubectl get pods                          # 3) STATUS + RESTARTS
kubectl describe pod <pod>                #    Events (scheduling/pull/probe)
kubectl logs <pod> --previous             # 4) the CRASHED container (CrashLoopBackOff)
kubectl describe deploy <deploy>          # 5) env: DB_HOST / DB_USER / DB_PASSWORD
```

The classic app faults:

Symptom	Root cause	Fix
`Endpoints: <none>`	Service **selector** ≠ pod **labels**	fix the selector
"name does not resolve"	wrong **Service name** (e.g. `mysql` vs `mysql-service`)	rename / fix env
"connection refused"	Service **targetPort** ≠ container port (8080 vs 3306)	fix `targetPort`
"access denied"	wrong **DB user/password** env	fix env / secret
timeout on the node port	wrong **nodePort**	fix `nodePort`

> 🎯 Set the namespace once: `kubectl config set-context --current --namespace=<ns>`. Immutable field edit → `kubectl replace --force -f`.

(B) Control-plane failure — symptom names the component

Control-plane components are **static pods**. The symptom tells you which:

Symptom	Broken component
pods stuck **Pending** (`Node: <none>`)	**kube-scheduler**
scaling / self-heal / new ReplicaSets don't happen	**kube-controller-manager**
`kubectl` itself fails ("connection refused")	**kube-apiserver** (or etcd)

```
kubectl get nodes ; kubectl get pods -n kube-system
kubectl describe pod kube-scheduler-controlplane -n kube-system   # Events + Last State
kubectl logs kube-controller-manager-controlplane -n kube-system
# if kubectl is DOWN (apiserver broken), use the runtime directly:
crictl ps -a | grep apiserver ; crictl logs <container-id>
ls /etc/kubernetes/manifests/             # fix the YAML → kubelet auto-restarts
```

Common breaks (in `/etc/kubernetes/manifests/*.yaml`): wrong `command`/args (`executable file not found`), wrong file path (`no such file or directory`), wrong volume `hostPath` (`unable to load client CA file`).

> 🎯 Editing a static-pod manifest **restarts that component** (kubectl may blip ~30–60s). When kubectl is fully dead, **`crictl ps -a` + `crictl logs`** are your eyes.

(C) Worker-node failure — nodes → conditions → kubelet

```
kubectl get nodes                          # NotReady?
kubectl describe node node01               # Conditions: Memory/Disk/PIDPressure, Ready=Unknown
```

`Ready=Unknown` + stale heartbeat = the kubelet stopped talking to the API server. Go **onto the node**:

```
ssh node01
systemctl status kubelet                   # active? inactive? activating(exit 255)?
sudo systemctl start kubelet               # or restart
journalctl -u kubelet -f                   # the real error
```

Log message	Cause	Fix location
kubelet `inactive (dead)`	just stopped	`systemctl start kubelet`
`unable to load client CA file … no such file`	wrong `clientCAFile`	**`/var/lib/kubelet/config.yaml`**
`connection refused` to apiserver `:6553`	wrong apiserver port	**`/etc/kubernetes/kubelet.conf`** (should be `:6443`)

> 🎯 Two kubelet configs: **`/var/lib/kubelet/config.yaml`** (behavior — `clientCAFile`, `staticPodPath`) and **`/etc/kubernetes/kubelet.conf`** (the kubeconfig — apiserver address:port). Start at `kubectl get nodes`, then SSH and check **service → logs → config**.

(D) Network failure — the kube-system trio

```
kubectl get pods -n kube-system              # CNI DaemonSet, kube-proxy, coredns all Running?
kubectl logs -n kube-system <kube-proxy-pod> # proxy mode / errors
kubectl exec <pod> -- nslookup <svc>.<ns>    # DNS working? CoreDNS up?
```

- **NotReady right after install** → CNI not deployed (`kubectl apply -f <addon>`).
- **DNS failing cluster-wide** → CoreDNS down/misconfigured.
- **Service unreachable, pods fine** → kube-proxy issue.

> ✅ **Check yourself before Rung 4:** A Service returns `Endpoints: <none>`. Which layer, which single check confirms it, and what's the root cause? And if `kubectl` itself won't respond — what tool replaces it?

RUNG 4 — The Vocabulary Map 🏷️

Term	What it actually is	Which layer
Endpoints	The pod IPs behind a Service	App (selector match)
selector / labels	Service's pod query / pod tags	App
targetPort	The pod port a Service forwards to	App
CrashLoopBackOff / ImagePullBackOff	Container keeps crashing / can't pull	App
`logs --previous`	The crashed container's logs	App
static-pod manifest	Control-plane pod YAML in `/etc/kubernetes/manifests`	Control plane
crictl	Runtime CLI when kubectl is dead	Control plane
node Conditions	Ready / *Pressure flags	Node
kubelet / journalctl	Node agent / its logs	Node
kubelet.conf / config.yaml	kubeconfig / behavior config	Node
CNI / kube-proxy / CoreDNS	Pod net / service net / DNS	Network

The unlock — symptom → layer → checklist:

```
app error (curl fails)        → APP:      map → endpoints → ports → logs --previous → env
Pending / no-scaling / no-API → CONTROL:  get pods -n kube-system → logs/crictl → fix manifest
node NotReady                 → NODE:      describe node → kubelet service → journalctl → config
DNS / connectivity            → NETWORK:   kube-system trio (CNI/kube-proxy/CoreDNS)
```

> ✅ **Check yourself before Rung 5:** Which layer's checklist starts with `kubectl get pods -n kube-system`, and which starts with `kubectl describe node`?

RUNG 5 — The Trace 🎬

Follow ONE real diagnosis end-to-end

Trace — "the app shows a database connection error":

1. **Reach it:** `curl -s localhost:30081 | grep -i error` → "Can't connect to `mysql-service` ... name does not resolve." That's an **app-layer** symptom pointing at the db hop.
2. **Walk the map to the db Service:** `kubectl get svc` → the actual service is named `mysql`, but the app is configured for `mysql-service`. Mismatch found at the name hop.
3. **Decide the fix:** either rename the Service to `mysql-service` or point the app's `DB_HOST` env at `mysql`. (If instead the name resolved but connection *refused*, you'd check `describe svc` for `Endpoints` / `targetPort`; if *access denied*, the DB creds env.)
4. **Apply + verify:** fix, then re-`curl` → the page turns green. One hop identified, one change, verified.

```
curl → error names the failing hop (db name) → get svc (name mismatch)
     → fix name/env → curl again → SUCCESS
```

The discipline: each command either **cleared a hop** (front-end reachable) or **named the fault** (wrong service name) — no random edits.

> ✅ **Check yourself before Rung 6:** In this trace, what told you the fault was at the *db* hop and not the web hop? What would `Endpoints: <none>` have pointed to instead of a name mismatch?

RUNG 6 — The Contrast ⚖️

Symptom…	vs…	Tells you
Pending pod (`Node: <none>`)	**NotReady** node	scheduler down vs kubelet down
no scaling / self-heal	**Pending**	controller-manager vs scheduler
`kubectl` **connection refused**	**app curl fails**	apiserver/etcd vs application
`kubectl logs`	`kubectl describe`	app errors (stdout) vs Events/Last State (scheduling/runtime)
`kubectl`	`crictl`	works when API is up vs the fallback when it's down
`config.yaml`	`kubelet.conf`	kubelet behavior vs its apiserver connection

When to use each tool: `describe` first (Events explain most failures); `logs --previous` for crash loops; `crictl` only when the API is unreachable; SSH + `journalctl` for node/kubelet faults.

One-sentence "why this over that":

> Read the symptom to the layer, then run that layer's checklist top-to-bottom — the map for apps, `kube-system` static pods for the control plane, the kubelet service for nodes, and the CNI/kube-proxy/CoreDNS trio for networking.

> ✅ **Check yourself before Rung 7:** Two clusters: one has pods stuck Pending; the other can't scale a Deployment. Different control-plane component each — name them.

RUNG 7 — The Prediction Test 🧪

Prediction 1 — App failure: the map leads straight to the broken Service

> **My prediction:** "If the app shows a DB error, then walking the map (curl → service name/endpoints/targetPort → env) finds one broken link, and fixing it turns the page green — *because* the fault is one hop in a known chain."

```
kubectl config set-context --current --namespace=alpha
curl -s localhost:30081 | grep -i error       # names the failing hop
kubectl get svc ; kubectl describe svc mysql-service   # name/selector/endpoints/targetPort
# fix the name/selector/port/env, then:
curl -s localhost:30081                       # SUCCESS
```

Verify: the page turns green after one targeted fix. Walk the map in order — don't skip to guessing.

Prediction 2 — Pending pods ⇒ scheduler; the fix is in its manifest

> **My prediction:** "If a pod is Pending with `Node: <none>` and the scheduler pod is CrashLoopBackOff, then `describe` shows a bad command in `kube-scheduler.yaml`, and fixing the manifest reschedules the pod — *because* Pending = nothing is assigning nodes."

```
kubectl get pods                              # app pod Pending, Node: <none>
kubectl get pods -n kube-system               # kube-scheduler-controlplane CrashLoopBackOff
kubectl describe pod kube-scheduler-controlplane -n kube-system   # "executable file not found"
sudo vi /etc/kubernetes/manifests/kube-scheduler.yaml   # fix the mangled command
kubectl get pods                              # pod schedules → Running
```

Verify: scheduler returns to Running, the Pending pod gets a node. `Pending + Node: <none>` ⇒ scheduler.

Prediction 3 — Worker NotReady ⇒ kubelet; the log names the file

> **My prediction:** "If a node is NotReady and its kubelet is failing on a CA file, then `journalctl` names the exact bad path in `/var/lib/kubelet/config.yaml`, and fixing it + restarting kubelet returns the node to Ready — *because* the kubelet is the node's agent."

```
kubectl get nodes                             # node01 NotReady
ssh node01
sudo systemctl status kubelet                 # activating (exit 255)
sudo journalctl -u kubelet | tail             # "unable to load client CA file … /WRONG-ca"
sudo vi /var/lib/kubelet/config.yaml          # fix clientCAFile → /etc/kubernetes/pki/ca.crt
sudo systemctl restart kubelet
exit; kubectl get nodes                       # node01 Ready
```

Verify: kubelet goes `active (running)`, node returns Ready. The error message names the exact file to fix.

Prediction: "If I do X, then Y, because [mechanism]"	Ran?	Right?	Miss?
1.			

🎁 CAPSTONE — Compress It

One sentence, no notes:

> Read the symptom to a layer (app / control-plane / node / network), then walk that layer's fixed checklist hop by hop — map→endpoints→logs for apps, `kube-system` static pods (with crictl as backup) for the control plane, kubelet service→logs→config for nodes, and the CNI/kube-proxy/CoreDNS trio for networking — fixing the one link that fails.

Explain it to a beginner in 3 sentences:

> 1. Don't guess — figure out which layer is broken from the symptom (a Pending pod is the scheduler; a NotReady node is the kubelet; a failing app is the request path).
> 2. Then follow that layer's checklist step by step until a check fails: for an app, trace user → service → pod → database, checking endpoints, ports, logs, and env at each hop.
> 3. When `kubectl` itself is down, use `crictl` on the node to read the control-plane containers' logs, and fix the static-pod manifest they came from.

Which rung to revisit hands-on?

- **Rung 3A (the app map)** — `Endpoints: <none>` and the name/targetPort faults are the most common tasks. Fix: Prediction 1, twice.
- **Rung 3B/C (crictl + kubelet configs)** — the "kubectl is dead" and "which kubelet file" moments. Fix: Predictions 2 and 3 on a real node.

🎯 CKA exam tips & quick notes

- **Draw the map, walk every link.** `Endpoints: <none>` = Service↔Pod mismatch (selector/ports), then pod status/logs.
- **`kubectl logs --previous`** for `CrashLoopBackOff`; **`kubectl describe`** for Events.
- **Control plane** = static pods in `/etc/kubernetes/manifests/`; symptom→component (Pending=scheduler, no-scaling=controller-manager, kubectl-dead=apiserver/etcd). kubectl down → **`crictl ps -a`** / **`crictl logs`**; fix the manifest (command/path/`hostPath`).
- **Worker node** = `get nodes` → `describe node` (Conditions) → SSH → **kubelet service → journalctl → config** (`/var/lib/kubelet/config.yaml`, `/etc/kubernetes/kubelet.conf` apiserver `:6443`).
- **Network** = CNI/kube-proxy/CoreDNS in `kube-system`; NotReady after install = deploy a CNI.
- Set the namespace once; `alias k=kubectl` + completion to save time.

📌 Command cheat sheet

```
# APP
kubectl describe svc <svc>          # Endpoints? selector?
kubectl logs <pod> --previous       # crashed container
kubectl get pods -o wide            # pod IPs / nodes
# CONTROL PLANE
kubectl get pods -n kube-system
kubectl logs <cp-pod> -n kube-system   |   crictl ps -a ; crictl logs <id>
ls /etc/kubernetes/manifests/
# WORKER NODE
kubectl describe node <node>        # Conditions
systemctl status kubelet ; journalctl -u kubelet -f
# fix: /var/lib/kubelet/config.yaml  |  /etc/kubernetes/kubelet.conf
```

Related sections

- Section 1 — Core Concepts — the object map (Service→Pod) you walk.
- Section 5 — Cluster Maintenance — recovering etcd/control plane; node drain.
- Section 6 — Security — cert/`crictl` control-plane debugging.
- Section 8 — Networking — CNI/kube-proxy/CoreDNS fault-finding.
- Section 3 — Logging & Monitoring — `logs` / `top` as first-line tools.
- ../../Linux/16-systemd-services.md — `systemctl` / `journalctl` for kubelet debugging.

✅ Answers — "Check yourself before Rung N"

Before Rung 2

Q: Why is "start changing things and see what helps" a losing strategy on a timed, multi-fault cluster? What does it cost you beyond time?

A: Because random changes aren't reasoned about: you change a field, it doesn't help, you change another — and now two things are wrong instead of one, or you fix the app when the real fault was in the control plane all along. Beyond time (the exam's scarcest resource), it costs you **reversibility**: undisciplined edits compound the fault and you lose track of what you've touched, so the cluster ends up more broken than when you started. A method — walking a fixed checklist by elimination — means every check either clears a hop or names the fault, with no wasted or destructive moves.

Before Rung 3

Q: Match symptom → layer: (a) pods stuck Pending, (b) `kubectl` returns connection refused, (c) node shows NotReady, (d) `Endpoints: <none>` on a Service.

A: (a) Pods stuck Pending → control-plane layer, specifically the **kube-scheduler** (nothing is assigning nodes). (b) `kubectl` connection refused → control-plane layer, the **kube-apiserver** (or etcd behind it). (c) Node NotReady → worker-node layer, the **kubelet** (it stopped talking to the API server). (d) `Endpoints: <none>` → application layer: the Service's **selector doesn't match the pod labels**, so no pods back the Service.

Before Rung 4

Q: A Service returns `Endpoints: <none>`. Which layer, which single check confirms it, and what's the root cause? And if `kubectl` itself won't respond — what tool replaces it?

A: `Endpoints: <none>` is an **application-layer** fault; the single confirming check is `kubectl describe svc <svc>` — compare the Service's **selector** against the pods' **labels**. The root cause is a selector/label mismatch, so the Service matches zero pods; the fix is to correct the selector. When `kubectl` itself is dead (apiserver broken), you fall back to the container runtime CLI: **`crictl ps -a`** to find the control-plane containers and **`crictl logs <container-id>`** to read their errors, then fix the static-pod YAML in `/etc/kubernetes/manifests/` (the kubelet auto-restarts it).

Before Rung 5

Q: Which layer's checklist starts with `kubectl get pods -n kube-system`, and which starts with `kubectl describe node`?

A: The **control-plane** checklist starts with `kubectl get pods -n kube-system` — the control-plane components are static pods there, and the failing one (scheduler, controller-manager, apiserver) shows up immediately. The **worker-node** checklist starts with `kubectl get nodes` then `kubectl describe node <node>` — reading the Conditions (Ready=Unknown, Memory/Disk/PIDPressure) before SSHing in to check the kubelet service, its journalctl logs, and its config files.

Before Rung 6

Q: In this trace, what told you the fault was at the *db* hop and not the web hop? What would `Endpoints: <none>` have pointed to instead of a name mismatch?

A: The curl itself succeeded in reaching the front end and returned a page whose error text named the failing hop: "Can't connect to `mysql-service` ... name does not resolve" — the web Service and web pod hops were already cleared (the request got through them), and the error explicitly pointed at the database service name. If instead `kubectl describe svc` had shown `Endpoints: <none>`, that would have pointed to a **selector/label mismatch** — the Service exists and resolves, but its selector matches no pod labels, so no pod IPs sit behind it — fixed by correcting the selector rather than the service name or `DB_HOST` env.

Before Rung 7

Q: Two clusters: one has pods stuck Pending; the other can't scale a Deployment. Different control-plane component each — name them.

A: Pods stuck Pending (with `Node: <none>`) means the **kube-scheduler** is broken — nothing is assigning pods to nodes. A Deployment that won't scale (no new ReplicaSets, no self-healing) means the **kube-controller-manager** is broken — it runs the controllers that create/scale ReplicaSets and replace pods. Both are static pods in `/etc/kubernetes/manifests/`, so the fix path is the same: `kubectl get pods -n kube-system`, read logs/Events, and repair the component's manifest.